

需求和架构 (1) [06–Raw/26生成式软件工程/NJU]

原视频: <https://www.bilibili.com/video/BV1Rch76WEfQ> · 课程合集:
<https://space.bilibili.com/202224425/lists/8942017> · 讲者: 蒋炎岩 (NJU 《生成式软件工程》2026)

全课主线

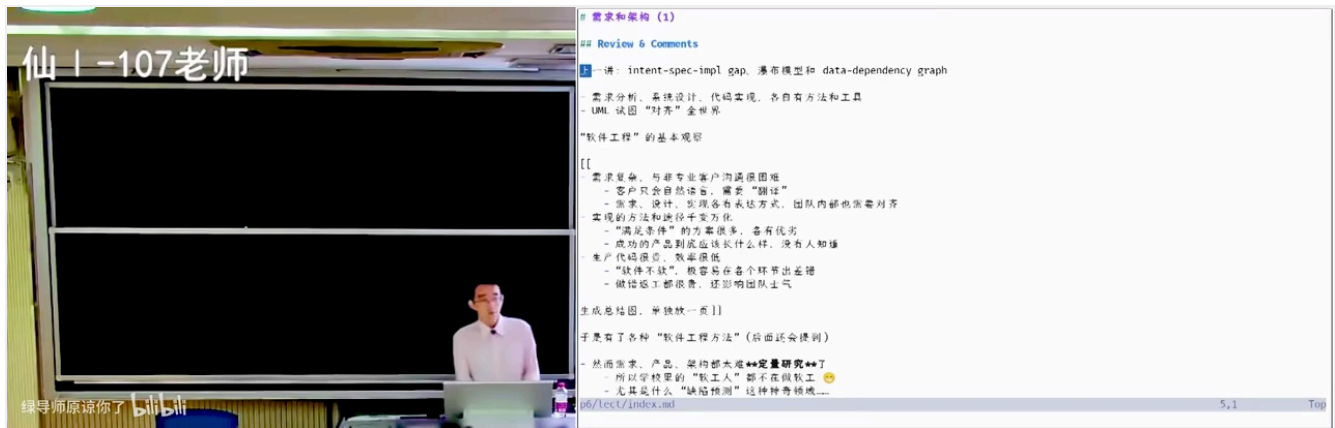
这节课的核心主张是：软件工程真正难的不是把代码写出来，而是在 intent 到 spec 阶段把需求、边界和验证讲清楚，并用架构控制复杂性、承接需求变化。AI 时代目标明确的代码生成变便宜，反而更需要需求工程、traceability、架构和从 first principle 出发的判断。讲者想让听众相信：只做 MVP、堆功能或依赖 AI 直接生成会积累技术债；正确做法是做能抵抗变化、保存历史状态的最小架构。

1. 软件危机使软件开发难以为继，于是需要工程化方法；Royce 式链条背后是 data dependency 网络，后置错误会沿依赖回溯并污染下游，因此软件工程诞生。(第 1 部分)
2. 为解决人对齐和追溯问题，出现 traceability 与 UML；而软件是纯逻辑制品、有形式语义，细微差错会改变行为，因此软件并不“软”，这是工程困难的一个本质原因。(第 1、2 部分)
3. 软件工程关乎全生命周期，但训练必须从底层代码开始；今天主题是从 intent 到 spec，是决定软件系统从零走向的最关键部分。(第 3 部分)
4. AI 让目标明确的代码变便宜，人的关键工作前移到把目标和边界讲清楚，因此从 intent 到 spec 更值得讲。(第 3 部分；整理者串联)
5. 信任 agent 的前提是边界清楚和 validation 明确；MVP 能快速走完一个切片但不完备，AI 不会自动从需求、产品设计和架构角度选择，早期决定进入上下文后变成技术债。(第 4 部分)
6. 做 MVP 的决策来自需求；需求、产品设计、价格是撬动软件生产力和技术债的杠杆，若不被明确要求，AI 也会留下日志缺失这类技术债。(第 5 部分)

7. 教务系统说明“需求爆炸”和功能叠加不等于需求被满足：做了需求分析不等于优先序对，需求叠加会制造史诗级 bug 和边界限制；学生不敢承包真实客户系统，agent 却可被派去探索并生成需求说明书。(第 6、7、8 部分)
8. 需求工程是通过无尽沟通把客户想要什么想清楚，需要同理心和领域经验；真实需求报告中的术语、形式语义、ID、验收场景、需求清单和 traceability 能对齐客户、管理团队，也让 AI 可依据说明书实现原型。(第 8、9、10 部分)
9. 需求永远不能完全说清且会变化，第一版一实现就背上技术债；改需求真正破坏的是整体一致设计，裂缝会传递成“一团乱麻”，真实系统不是课程作业。(第 11、12 部分)
10. AIGC 时代做 MVP 容易，但会产生驾驭 AI 的错觉；软件工程没有银弹，早期决策会持续变成技术债，因此第一步要的不只是 MVP，而是能承受需求变化的最小架构。(第 13、14 部分)
11. 架构本质是控制复杂性、解开耦合；从小代码例子到中间语言方案，都在讲抽象、分解和跳出问题看问题，以便承载后续需求变化。(第 14、15、16、17、18、19、20、21 部分)
12. AI 生成的系统即使工程细节成熟，跳出来仍可能只是 CRUD；教务系统这类需求要保存历史、面对物理世界未建模变化，当前状态式 CRUD 会死，于是转向状态/操作/Delta 等价、先记录事实再回放投影的 event sourcing。(第 22、23、24、26、27、28 部分)

回到 first principle 看清问题根源，即使没听过这些模式名，也能做出正确架构设计，而不是只堆功能或依赖 AI 直接生成。

第 1 部分



仙 I -107老师

需求和架构 (1)

Review & Comments

讲一讲 intent-spec-impl gap, 瀑布模型和 data-dependency graph

- 需求分析、系统设计、代码实现, 各自有方法和工具
- UML 试图“对齐”全世界

“软件工程”的基本观念

[[需求复杂、与非专业客户沟通困难

- 客户只会自然语言, 需要“翻译”
- 需求、设计、实现各有表达方式, 团队内部也需要对齐

实现的方法和建设平台变化

- “需求事件”的方案很多, 各有优劣
- 成功的产品到底应该长什么样, 没有人知道

生产代码很苦, 效率很低

- “软件不烂”, 很容易在各个环节出差错
- 做给老板都很贵, 还影响团队士气

生成总结图, 单独放一页]]

于是有了各种“软件工程方法”(后面还会提到)

- 然而需求、产品、架构都太难**定量研究**了
- 所以学校里的“软工人”都不在做软工
- 尤其是“缺陷预测”这种神乎其技...

06/lect/index.md 5,1 Top



仙 I -107老师

Review & Comments

上一讲的三个关键词

- Intent-Spec-Impl Gap
- 瀑布模型
- Data-Dependency Graph

各有方法, 也需要对齐

- 需求分析、系统设计、代码实现, 各自有方法和工具
- UML 试图“对齐”全世界

1/4

软件危机逼出工程化方法：软件工程要解决“开发不下去”

- 上一课讲软件工程的来龙去脉：当时遇到**软件危机**，软件开发难以为继，因此需要一种工程化方法来解决软件中的问题。
- 讲者回顾 Royce 的老 paper：其理想大致是 `intent` → `spec` → `implementation` → **软件产品**；但实现后还需要维护，软件生命周期没有结束。
- 所以自然地形成一条向下的、类似**瀑布**的过程。

Royce 的瀑布链背后是 data dependency 网络，这就是软件工程起点

- 在 `intent`/设计/需求阶段，会得到对软件的一些认识；这些认识逐渐进入 **specification space**，并形成依赖关系。
- 到系统实现时，会出现模块、函数、代码、测试等各种 **software artifacts**。

- 这些 artifacts 之间存在直接和间接的 **data dependency**，最后形成错综复杂、交错的网络。
- 讲者说：这就是软件工程的一个起点。

Royce 批判这个模式：后置错误会沿依赖回溯，污染大片下游

- Royce 的分析来自 **data dependency**：如果在后面发现某个东西错了，错误会反推、回溯到前面某个东西错了。
- 如果前面的东西事后被证明是错的——例如设计不合理、实现达不到预期、需求或设计做错——那么后面很大一部分、被污染/触及的部分都会失效。
- 后果是可能需要重做，并带来大量成本。
- 逻辑链：后置阶段发现错误 → 沿 **data dependency** 回溯到前置需求/设计错误 → 下游 **artifacts** 大面积失效 → 重做 → 成本飙升。
- 讲者说，在这样的背景下，软件工程诞生了。

软件工程研究从需求、设计到实现的各种方法

- 软件工程研究的是：做需求分析、系统设计、系统实现时，有什么样的方法。
- 既包括有人文关怀的方法：用 **checklist**，讨论产品经理应该怎么管人；讲者认为这也是软件工程的一部分。
- 也包括代码层面：写代码时应该怎么写测试、能否做**自动化测试**、**自动化分析**。
- 由此展开软件工程的一整个故事。

traceability：因为人难对齐、只剩代码难追溯

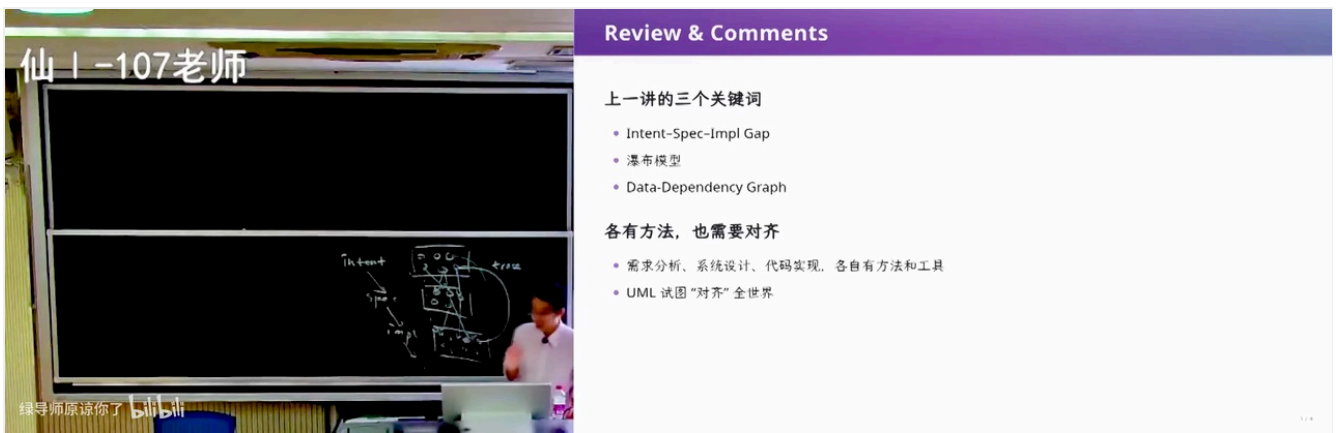
- 讲者提到一个在 [APRAI] 时代的有趣尝试（录音如此）。原因包括：人很难对齐，人和人很难对齐；如果不写文档、只剩一份代码，出错了甚至不知道该追溯到过去的什么时候。
- 所以软件工程提出重要概念：**traceability**（可追踪性）。
- 理想情况下，每一个测试用例或代码片段都能关联、trace 到一个过去发生过的事情、做过的设计。
- 作用：维护“整个系统里到底发生了什么”“我改它会影响谁”“我改设计会影响哪些测试用例”。

结构化、有语义的表示需求催生 UML

- 如果有一个语言，能够结构化、有语义地表示这些关系和 artifacts，那显然是更好的。
- 于是有了 UML；讲者说，它试图去对齐整个世界的软件工程实践。

小结：软件危机与瀑布式依赖风险推动软件工程；traceability 与 UML 是对齐人和实践、表达依赖的早期尝试。

第 2 部分



仙 1-107 老师

Review & Comments

上一讲的三个关键词

- Intent-Spec-Impl Gap
- 瀑布模型
- Data-Dependency Graph

各有方法，也需要对齐

- 需求分析、系统设计、代码实现，各自有方法和工具
- UML 试图“对齐”全世界

结构化、有语义的语言 → UML 试图对齐软件工程实践

- 讲者先承接前文：用结构化、有语义的语言来表示它，显然更好；所以有了 UML。
- 讲者认为，UML 试图去对齐世界上整个软件工程实践；这些实践/尝试本身是“有智慧”的。

软件工程困难的本质原因之一：软件其实一点都不“软”

- 讲者把“软”的日常含义具体化：像教室里的椅背，它有弹性；按下去会瘪，坐后弹回来，下次还是好的。
- 软件不同：它是纯粹的逻辑制品，不是物理材料。这个逻辑制品容不下任何细微差错；作为程序，它有形式语义。
- 讲者的链：纯逻辑制品 + 形式语义 → 细微差错会改变程序行为，不能像椅背那样靠弹性恢复 → 软件不“软”，这是软件工程困难的一个本质原因。

bit flip 例子：N=1 变巨数，程序彻底 collapse

- 设想程序从 `stdin`（原音：SDE）输入 N。输入 N=1 时，循环只从输入（数字数组/输入流）读一个元素到 `buffer`。
- 若软件运行在卫星上，宇宙射线击中内存，这个 `unsigned integer` 的最高位变成 1；N 从 1 突然变成巨大无比数。
- 后果：程序无限制从输入读数；没输入就等；读了一百、一千个数字后产生 `memory error`，整个程序彻底 collapse。
- 讲者总结：所以软件其实一点都不软。这个例子比较极端，因为运行环境不可预见。

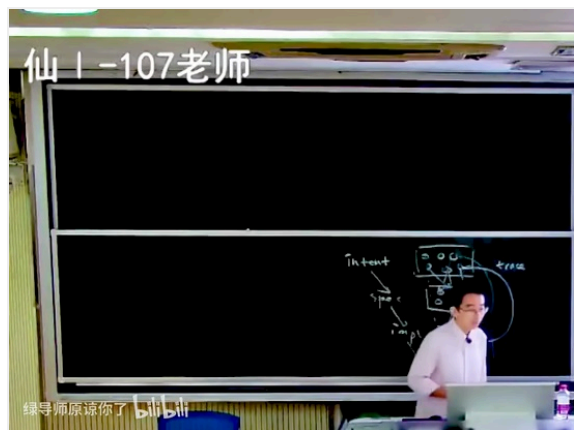
数据中心也有 silent error：硬件缺陷同样能击穿逻辑正确性

- 讲者补充：不只在太空极端环境；在 `data center` 也能观察到类似问题。
- 随着处理器制程越来越先进，在热噪声下，某些密度特别大的指令可能产生 `silent error`（原音近似：silence error）。
- 同一条指令在这台机器上执行和在那台机器上执行，结果可能不一样；甚至要服务器运行一段时间后才出现，属于制造时的缺陷。
- （整理者串联）这说明：即使程序逻辑上写对，执行它的物理硬件也可能引入细微错误，因此“纯粹逻辑制品容不下细微差错”不是抽象口号，而是工程风险。

软件既要集成人类智慧，又要承载客户需求；引出 Java 系统与屎山

- 讲者认为，软件是集成了人类智慧的东西，同时又要承载客户来的需求。
 - 由此他今天会花时间讲 `Java 系统`：应该怎样设计、怎样防止它堆积成屎山。
 - 在这个背景下，讲者观察到：学计算机专业的同学肯定要上软件工程课程；软件工程是关乎整个软件生命周期的学科，但学生往往被迫只能从“底下这个地方”开始学。
-

第 3 部分



仙一-107老师

软件工程方法与研究

于是有了各种“软件工程方法”

- 然而需求、产品、架构都太难定量研究了
 - 所以学校里的“软工人”都不在做软工 😊
 - 尤其是什么“缺陷预测”这种神奇领域.....

一个来自城市规划的参照

- 困难的不仅是“怎么测”：不同参与者，可能连“什么算成功”都没有共识。参见城市规划中的 [Wicked Problems](#) (Rittel & Webber, 1973)。

绿导师原谅你了 Bilibili

软件工程是全生命周期，但训练必须从底层代码开始

- 讲者主张：**软件工程**是一个关乎整个软件生命周期的学科。
- 但你们被迫从最底下开始学。讲者给出的理由链是：
 1. 如果你不理解代码是什么；
 2. 如果你写不出代码；
 3. 如果你没有体会过码农开发软件的痛苦；
 4. → 那你不可能做好一个**产品经理**。
- 进一步说：没有一手经验，你确实做不到判断“这个事情应该可以办到”“这个事情花多少成本可能可以办到”。
- 所以课程先学 **C 语言**：一个最基础的语言。讲者评价它“老、已经过时、没人用”，但也还算有一些人用。
- 在这个基础上不断产生新知识：知道世界是怎么失败的、新语言是怎么被创造出来的、为什么要有它，然后一点一点往上走。

今天的主题：从 intent 到 spec，是决定软件从零走向的最关键部分

- 今天要讲的是**需求、产品架构**，处在软件的设计阶段，也就是从 **intent** 到 **spec** 的阶段。
- 讲者认为，这其实是决定一个软件系统从零开始走向的**最关键部分**。
- 前面上课先讲**版本管理**，讲者解释：因为你总是要从一个空的软件项目出发；“大B1”（原转写）时总是从一个空项目出发。先讲如何从空出发，软件怎样往后走。

- 下一步要**把控软件执行的方向**：你的船到底往东南西北哪个方向开。一开始方向选了以后，对工程的影响非常大。
- 逻辑链是：空软件项目 → 先讲版本管理/如何从空出发 → 下一步确定方向 → 方向选择强烈影响整个工程。

需求与产品架构难做量化研究，学术软工多集中在 implementation

- 讲者吐槽：像现在的**需求、产品架构**，都很难做出量化研究（原转写“两个研究”）。
- 常见做法要么是案例式地说“今年没办法，我只能这样做案例”，要么按某个科学基金概念刷一个题目：需求要做 A/B/C，架构/产品要满足什么什么。
- 他提到：学校里有很多做软件工程的人；我们号称**南京大学软件工程世界第一**，在 **CSRankings**（原转写“cs making”）里数 **ICSE**（原转写“ECISE”）论文的话。
- 但讲者认为，他们大部分都集中在和编码相关的 **implementation** 相关东西上。
- 例子：有些做 **NLP** 的人会说，**traceability** 在软件世界里非常重要，能不能训练一个神经网络模型，直接从数据当中推导出 **traceability**。讲者回应：也许用 **GPT-6**（原转写“GP6”）就可以做到。
- 但讲者今天还是要讲 **intent→spec** 这个主题。

AI 让“目标明确的代码”变便宜，反而更需要划清边界/spec

- 讲者认为，尤其在 **AI 时代**，生产一个**目标明确的代码**已经变得非常便宜。
 - 个人经验：让 AI 生成代码，只要把它的**边界划定清楚**，他其实真的已经不看它的**实现**了。
 - 例子：他一会要展示 **GPT-6**（原转写“GB6”）昨天写的代码；其实现已经超越人类能够理解的范畴，但是 **so far**（原转写“三炮”）它是对的。只要边界划得清楚，他还能比较好地信任它。
 - 他最后说：并且我能够知道它怎样做一个好的.....（转录中断）。
 - （整理者串联）这里的隐含链条是：AI 能便宜地生成目标明确的代码 → 人的关键工作前移到把目标、边界、**intent** 讲清楚 → 因而从 **intent 到 spec** 的阶段更值得讲。讲者明确说自己今天仍要讲这个主题，但“关键前移”这一连接由整理者补出。
-

第 4 部分



能信任 agent 的前提：边界清楚 + validation 明确

- 讲者先提到：三炮（音）是对的，他还能比较好地信任它。
- 但信任有条件：
 1. 把它的边界划定清楚；
 2. 知道他怎样做一个好的 validation。
- 满足这些条件后，讲者认为可以把 agent 的 scope（原转写为 swap）铺开，让它帮忙实现软件，而且仍有一定质量保证。
- 逻辑链：边界清楚 + validation 明确 → 可以较好信任 agent → 可以把 agent 的 scope 铺开 → 由 agent 实现有质量保证的软件。

MVP 的价值：快速走完整切片，但天然不完备

- 讲者认为，做软件系统的压力，或者说能决定软件系统最后能走多远的，是剩下的部分。
- 做 MVP（minimum viable product）没有问题：它能让你快速找到一个从 intent → specification → implementation 的切片；在这个切片上，可以走完整套软件流程。
- 但 MVP 不完备。
- 做 MVP 时，一定先做最重要的功能，不会先想按钮样式长什么样。
- 原因链：按钮样式依赖整个界面设计，界面设计又依赖所有功能的实现。如果上层设计还不确定，就先去调按钮样式，当上面的设计发生改变后，按钮样式很可能就不再适用/作废。

- 所以人会有一种倾向：**不做无用功**，不研究一个即将被扔掉的东西。讲者认为这是每个人都有的本能。
- 如果确实有这种本能，就应该知道做 MVP 时选哪些最重要的功能，从而得到一个**可运行版本**。AI 会给你一个。

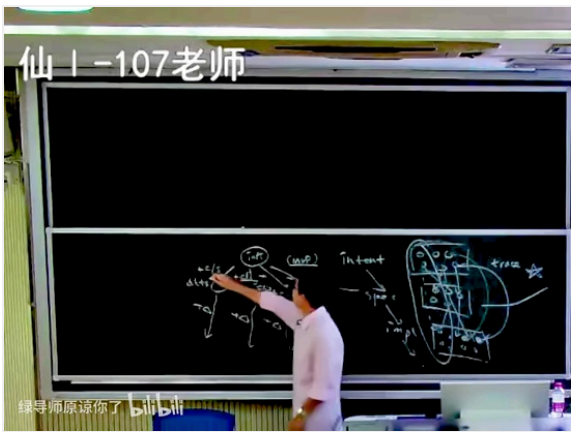
今天的 AI 不会自动从需求、产品设计、架构角度做选择

- 讲者认为，至少今天的 AI，不会主动从**需求、产品设计、架构**的角度考虑，除非你显式用 **prompt** 告诉它，甚至可能只是几个字。
- 从零开始的项目中，同一个 MVP 需求定下来，如果让它用 **client-server** 架构，或让它做 **CLI** 架构，加上几个字的 prompt，就会得到完全不同的 MVP。
- 它们功能上可能差不多，看起来都实现了需求，例如做一个 **coding agent**，或做讲者提到的 **live1**（音）学生的 **agent**。
- 但几个字的 prompt 会形成/膨胀出不同结构：
 - 如果先要求 **CLI**：里面可能有 **cli.py**，或依赖一个 JavaScript 里像 **COCODE**（音）用的命令行库。
 - 如果先要求 **client-server**：它可能先设计数据存储，然后给你选一个数据库。
- 关键机制：一旦某个东西被写出来，比如 **db.ts**，它就会进入 **agent** 的上下文；之后每当你继续做下一个需求，都会带上之前所有历史。

技术债：早期决定进入上下文后，变成后续每步都背着债

- 软件工程里的术语是 **技术债**（technical debt）。
 - 讲者说：你在做任何决定的时候，就背上了这个债务。
 - 因为在做 MVP 时，你并不知道这个决定最终对不对、是否真的需要。
 - 你的决定来自需求：比如对讲者来说，到底要一个什么**功能代理**。
 - 逻辑链：早期架构/prompt 选择 → 生成具体文件/结构（如 **db.ts**） → 进入 agent 上下文 → 后续需求携带全部历史 → 形成技术债。
-

第 5 部分



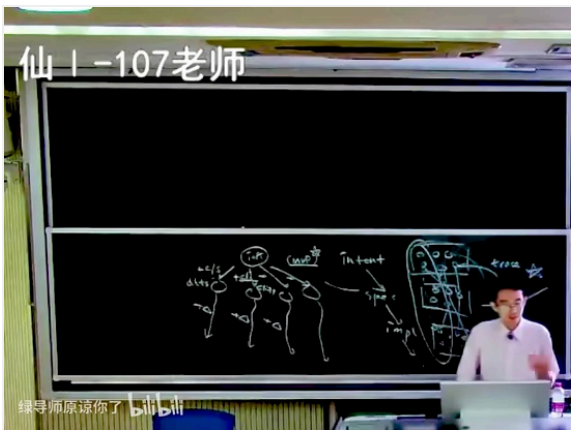
Agentic AI：软件生产力的杠杆

生产目标明确的代码，极为便宜

- 甚至拥有 Agent Swarm 并行的能力
- Agent 也能写测试用例、断言、甚至是证明

需求、产品设计、架构

- **需求、产品设计、架构**成为撬动软件生产力的杠杆
 - 如果不认真对待，就会成为 AI Slop 的堆积



需求与架构 (1)

Review & Comments

上一讲：intent-spec-impl gap, 瀑布模型和 data-dependency graph

需求分析、系统设计、代码实现，各自有方法和工具

- UML 试图“对齐”全世界

“软件工程”的基本观感

[[

需求复杂，与专业客户沟通困难

- 客户只会自然语言，需要“翻译”

- 需求、设计、实现各有表达方式，团队内部也需要对齐

实现的方法和途径千变万化

- “满足条件”的为数很多，各有优劣

- 成功的产品到底应该长什么样，没有人知道

生产代码痛苦，效率很低

- “软件不烂”，极容易在各个环节出差错

- 做错了工都很贵，还影响团队士气

生成总图，单独放一页]]

于是有了各种“软件工程方法”（后面还会提到）

- 然而需求、产品、架构都太难**定量研究**了

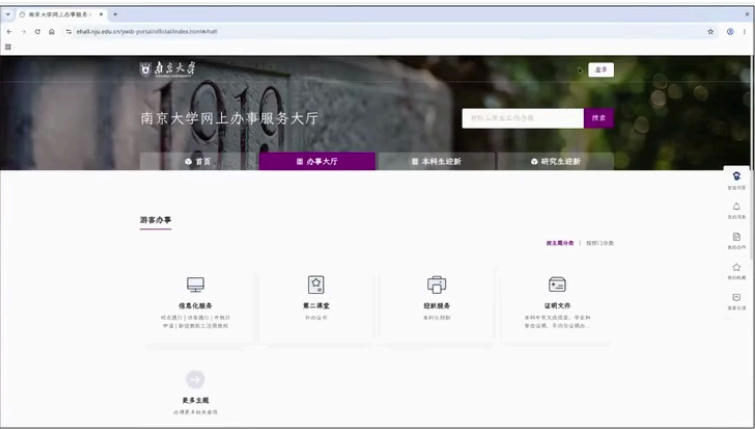
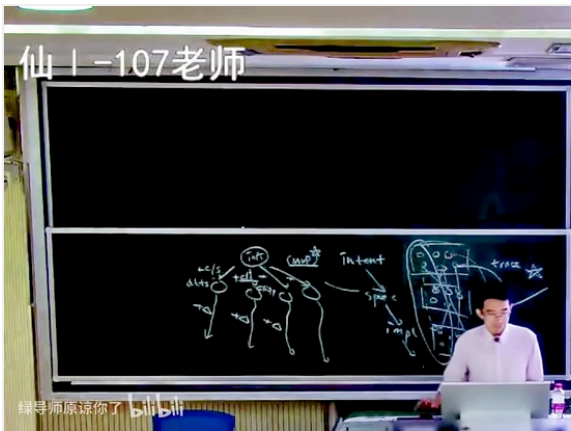
所以学校里的“软工人”都不在做软工

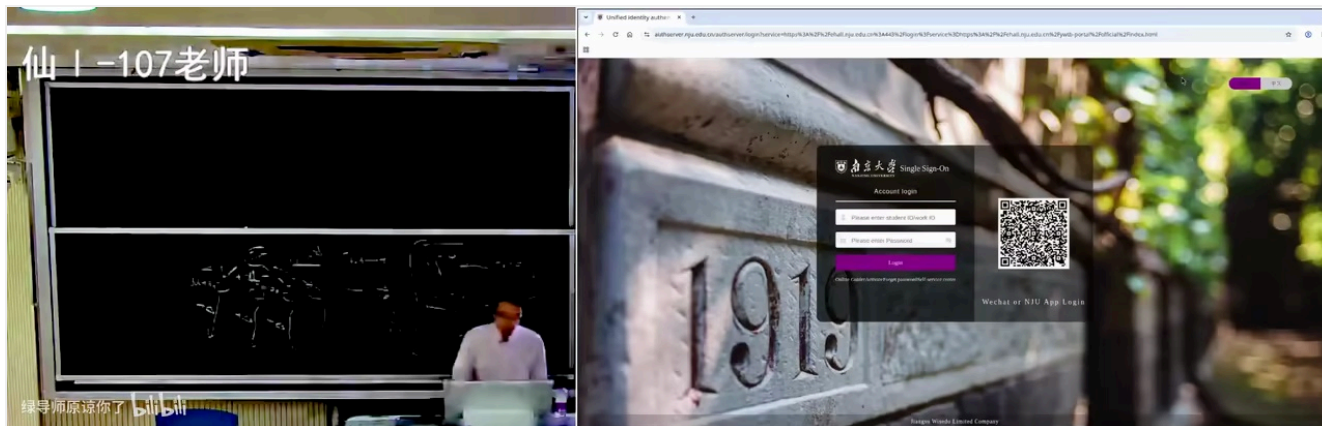
- 尤其是“缺陷检测”这种神秘领域...

86/Lect/index.md

5/1

Top



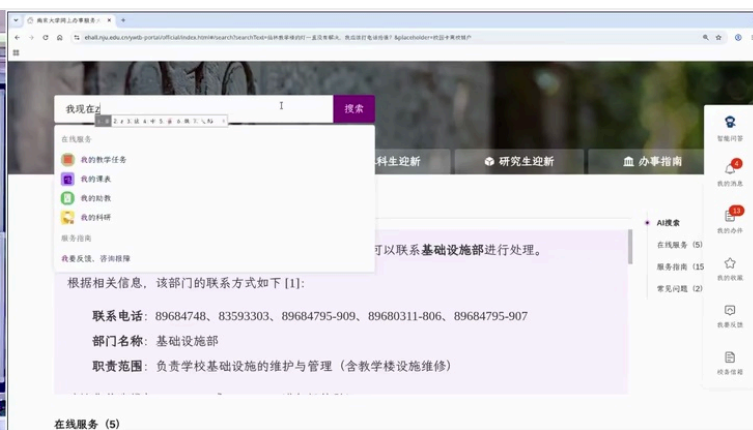
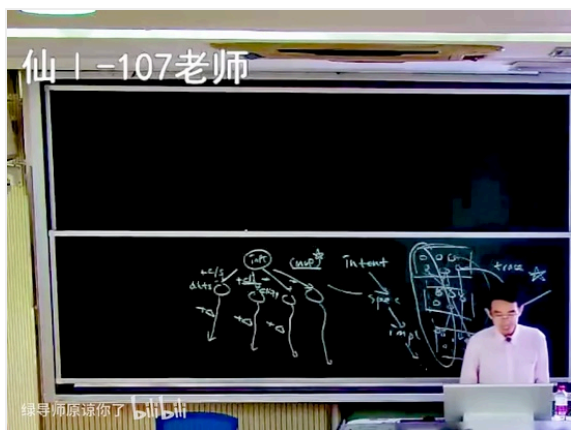
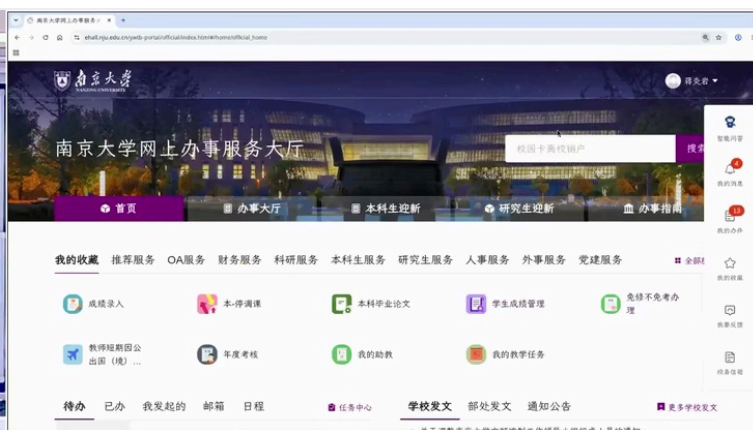
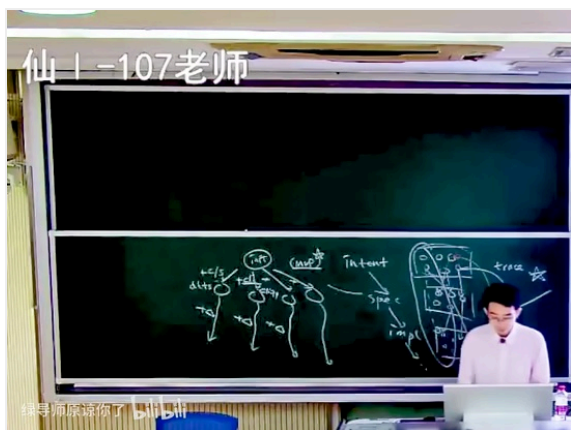
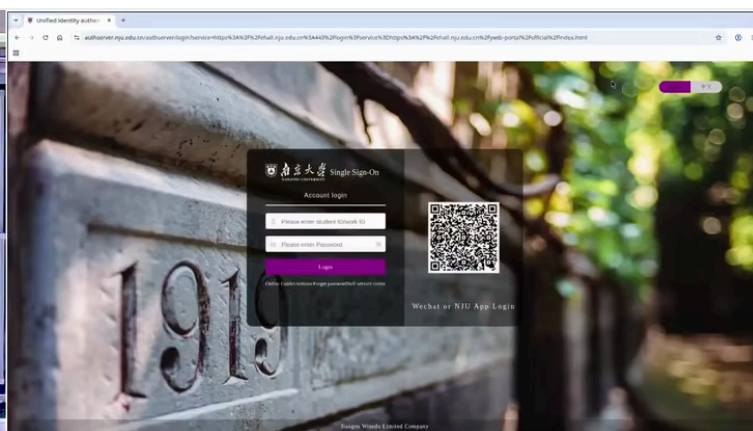
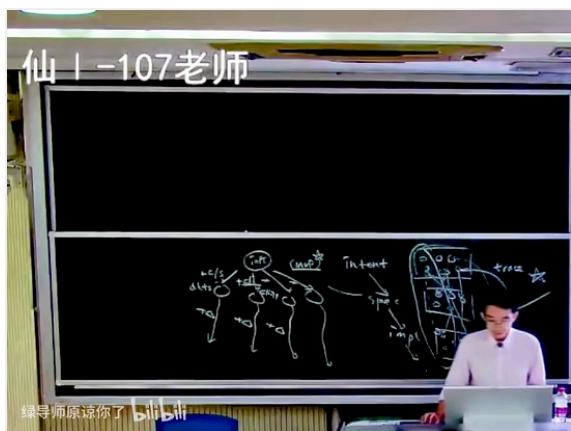
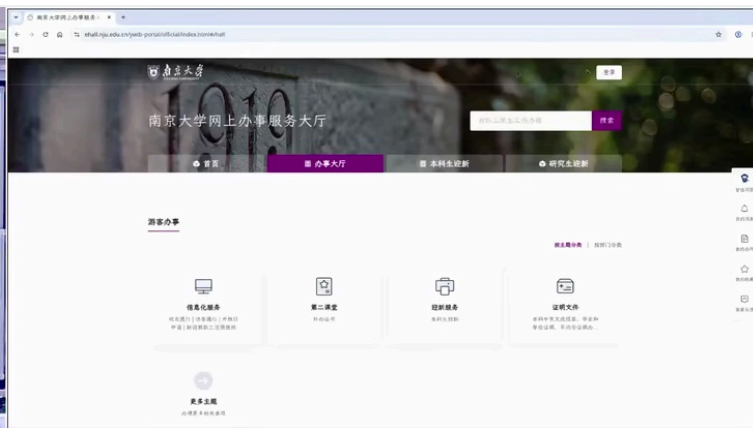
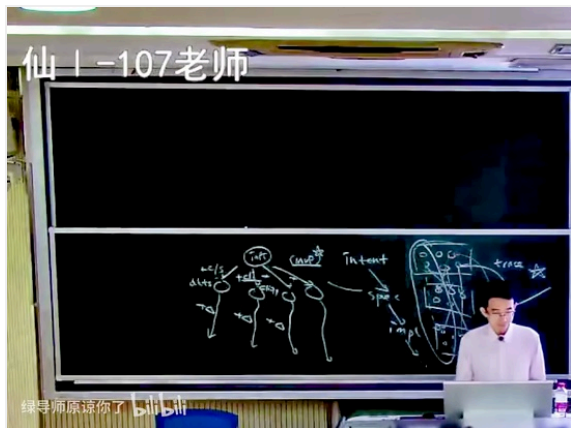


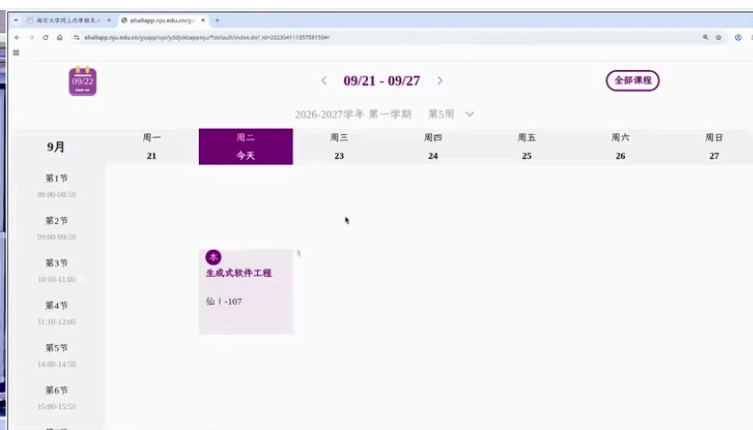
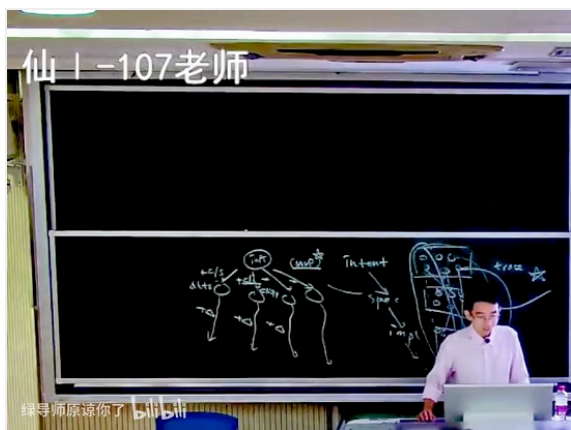
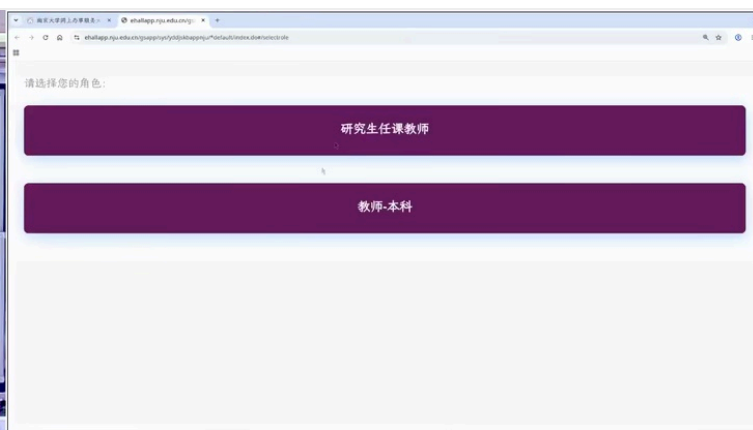
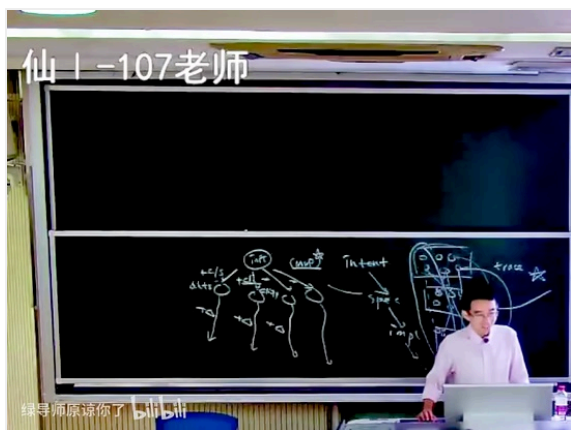
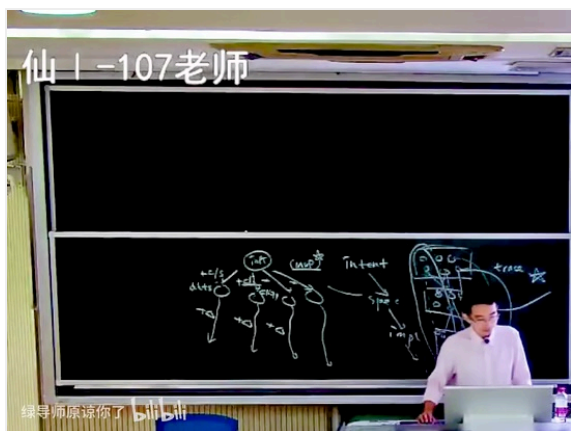
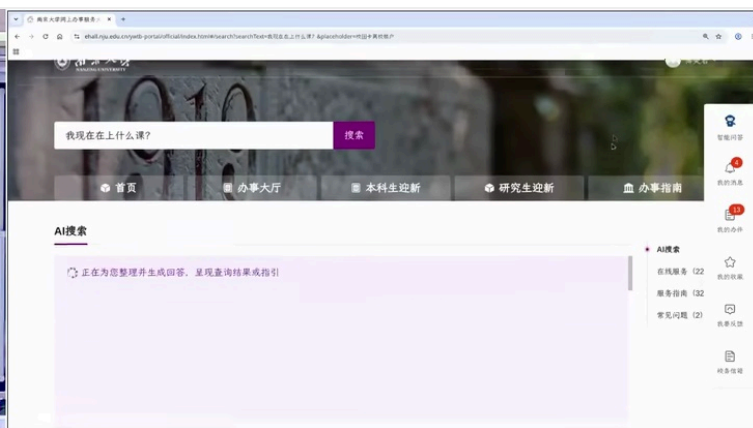
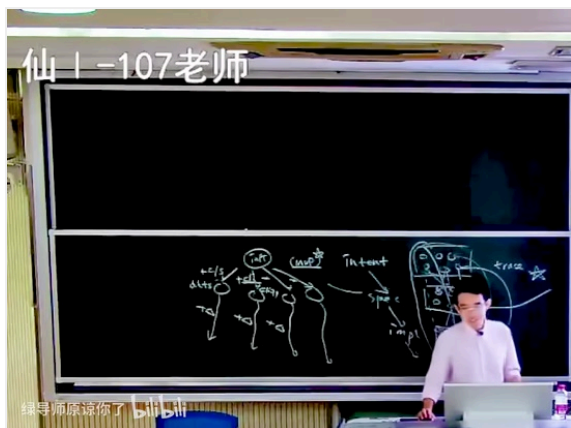
做 MVP：决策来自需求，不同需求把软件带向不同方向

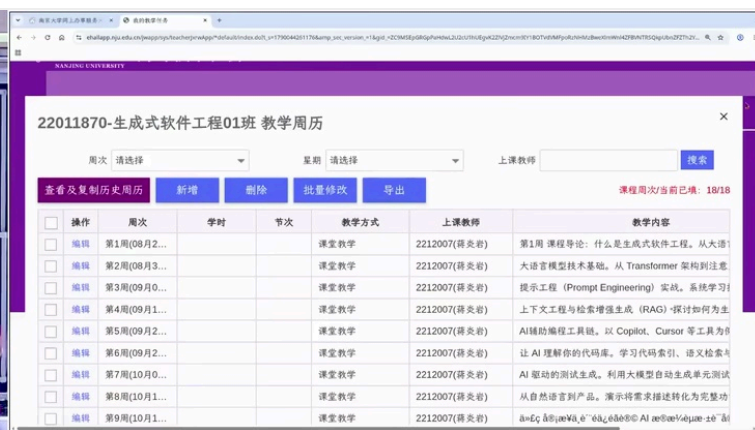
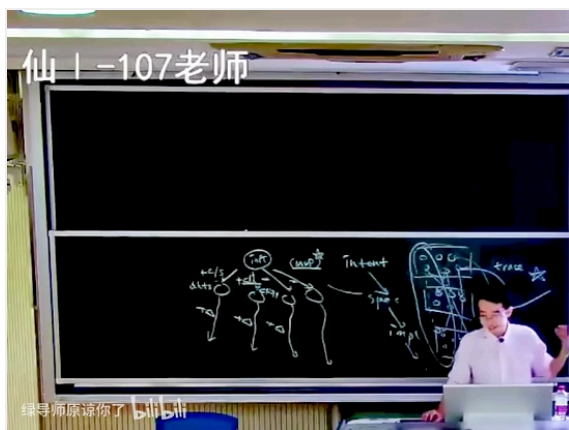
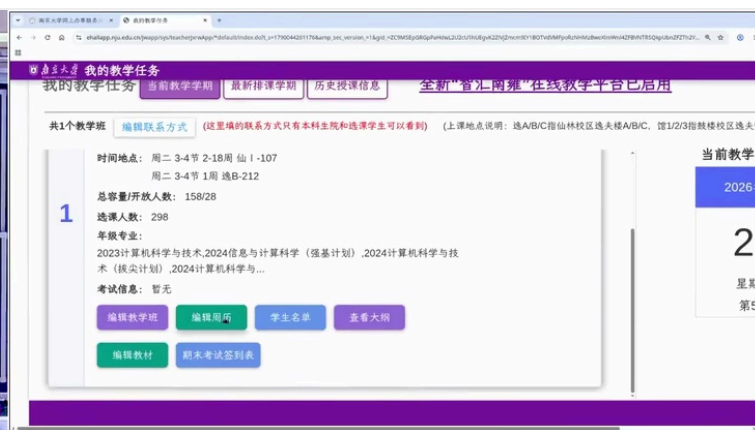
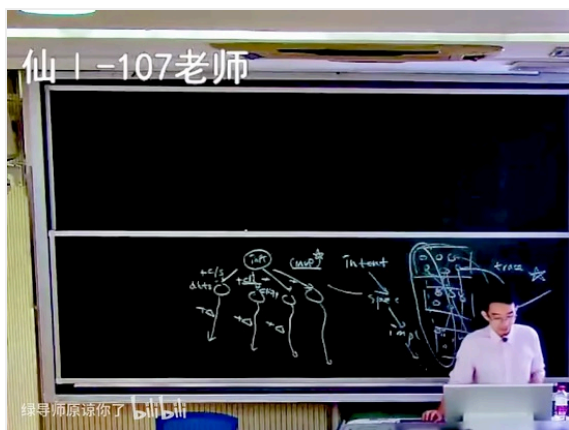
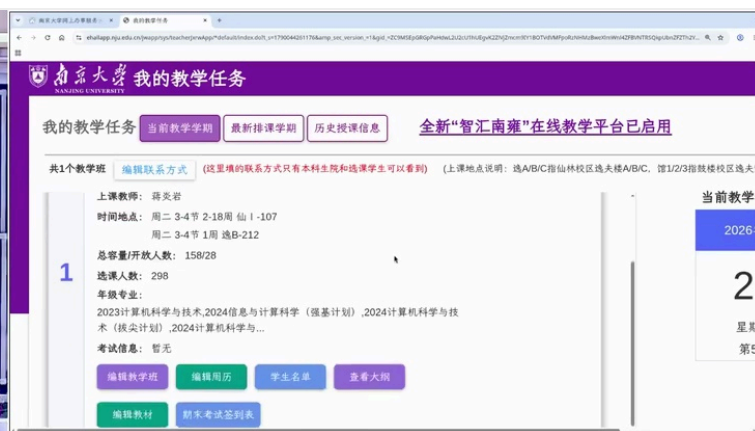
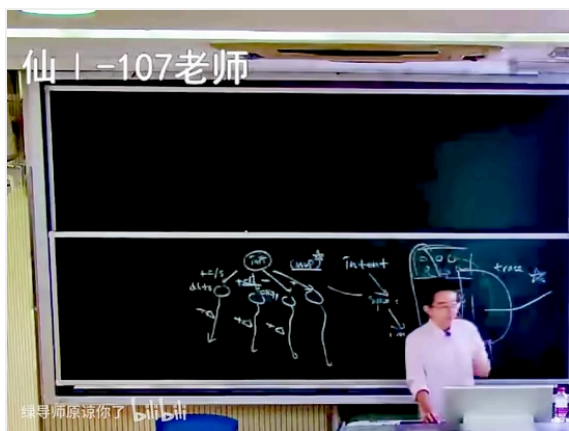
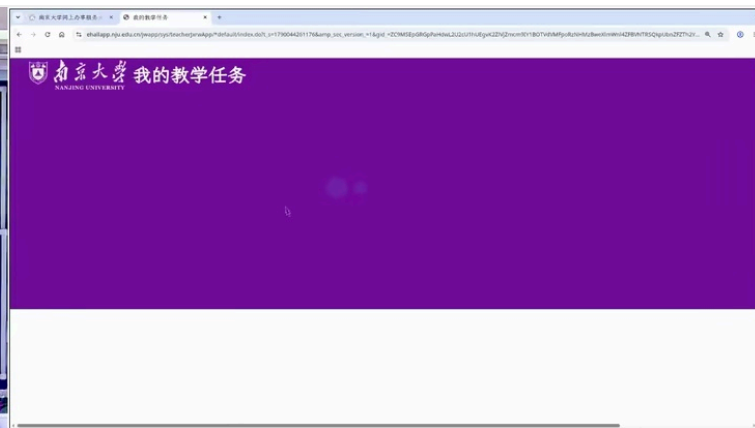
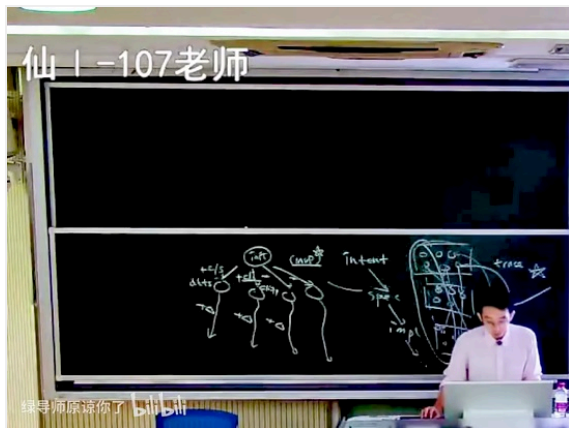
- 讲者指出：做 MVP 时，决策依据来自需求。
- 需求要写成“自己的版本”：
 - 例如想做一个帮助自己学习的东西；
 - 或者已经进入课题组，对读研究论文有很强需求。
- 链条：不同需求 → 软件走向不同。讲者说，在不同需求下，软件的走向肯定不同。

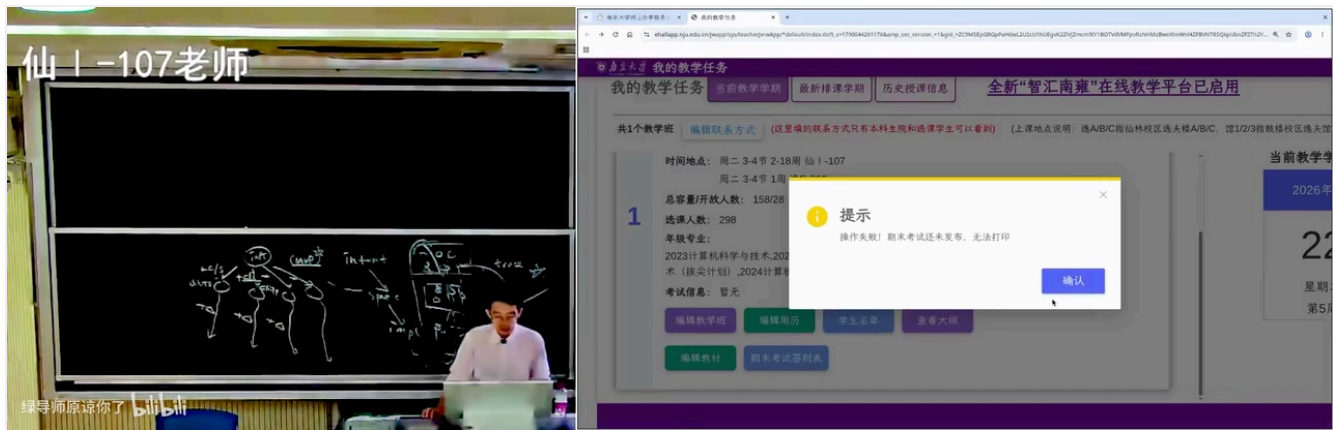
需求、产品设计、价格：三段杠杆

- 在需求之外，还要加产品设计。
- 讲者提醒直面最终产品：它可能比你想象的更接近“只有你自己用”。
- 自己用顺手 ≠ 别人用也顺手：讲者说自己有很多用得很顺手的软件，但想给另一个人用时，对方就用得不舒服；因为讲者有自己的知识和偏好，别人可能受不了、不习惯。
 - 例如个人偏好的命令/工作方式：内行觉得顺手，普通人肯定不要。
- 还有一个隐藏项：价格。价格会影响“这个软件会变成什么样”。
- 讲者认为，需求、产品设计、价格这三段会成为撬动软件生产力的真正杠杆；会对 AI 生成的 [stop]（疑为 startup）的走向、技术债的累积带来非常大影响。
 - （整理者串联：讲者所谓“这三段”承接上文，指需求、产品设计、价格。）
- 因此讲者要花两节课讲需求和价格。









所谓“史诗级更新”是自然语言搜索接入了登录上下文，但入口习惯会遮蔽它

- 讲者登录教务系统后，演示现在可以直接搜索/提问，并称之为“史诗级的更新”。
- 例子一：他问“仙林教学楼的灯一直没有解决，我应该打电话给谁”。系统出现“我正在理解您的问题”一类响应，随后话题落到**基建处**。但讲者补充：他已经在平台上提过，基建处把单子打回来，说“这个事情不归我们管”，所以他打算回头再打电话。
 - **（整理者串联）** 这个例子说明：问答入口能帮助定位部门，不等于实际报修闭环已经解决。
- 例子二：他问“我现在在上什么课”。他不确定系统有没有载入自己的上下文，但猜测可以，因为“我登录了”。结果系统返回**教师课表（本科）**，显示他的课表。
 - **逻辑链**：登录 → 系统可能载入个人上下文 → 问“我现在上什么课” → 返回教师课表（本科）。讲者明确说这是猜测，不是已确认机制。
- 讲者说，这是一个“史诗级更新”，但大家可能不知道，因为功能不知何时加上。他自己以前不会去搜索，因为总在**快捷搜索栏**里操作，等下一个界面弹出就直接点。因此功能虽在，入口习惯让他一直没用。

讲者判断：这种“需求爆炸”的系统，不如把预算变成 token 补贴

- 讲者认为，这样的系统是典型的需求爆炸系统：功能不断叠加，系统肯定花了很多钱。
- 他的理由是：
 1. 系统要购买算力；
 2. 要购买开发；

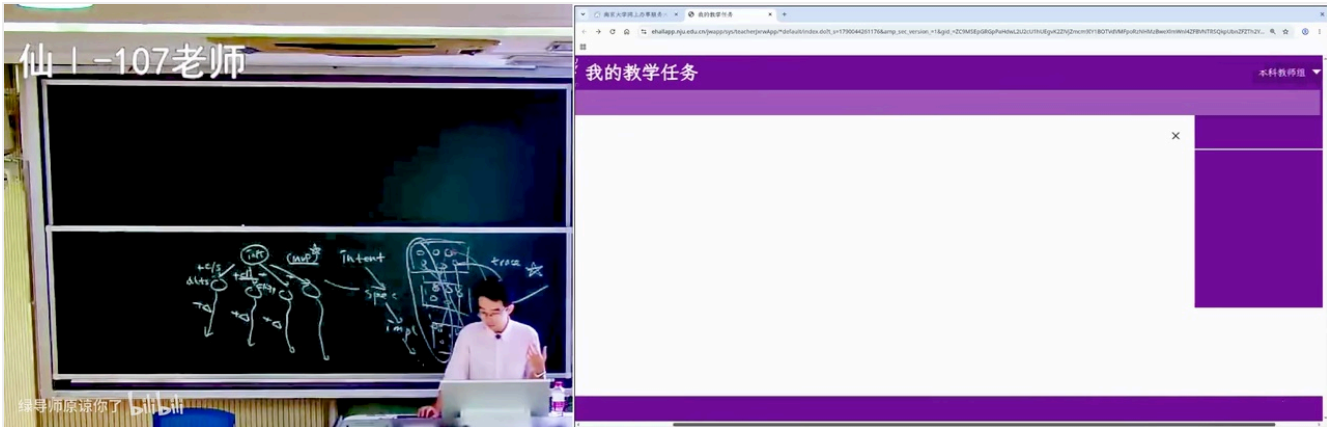
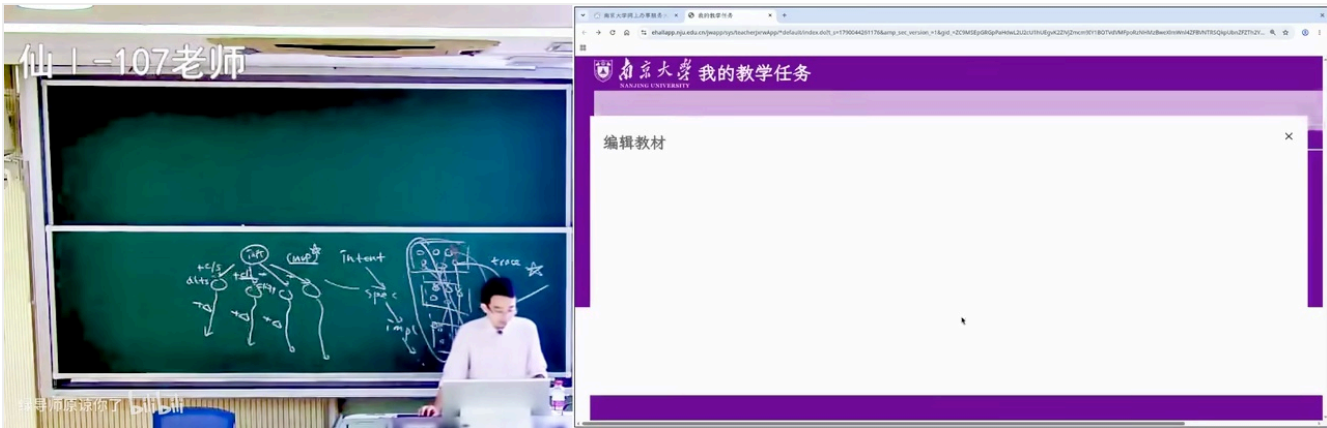
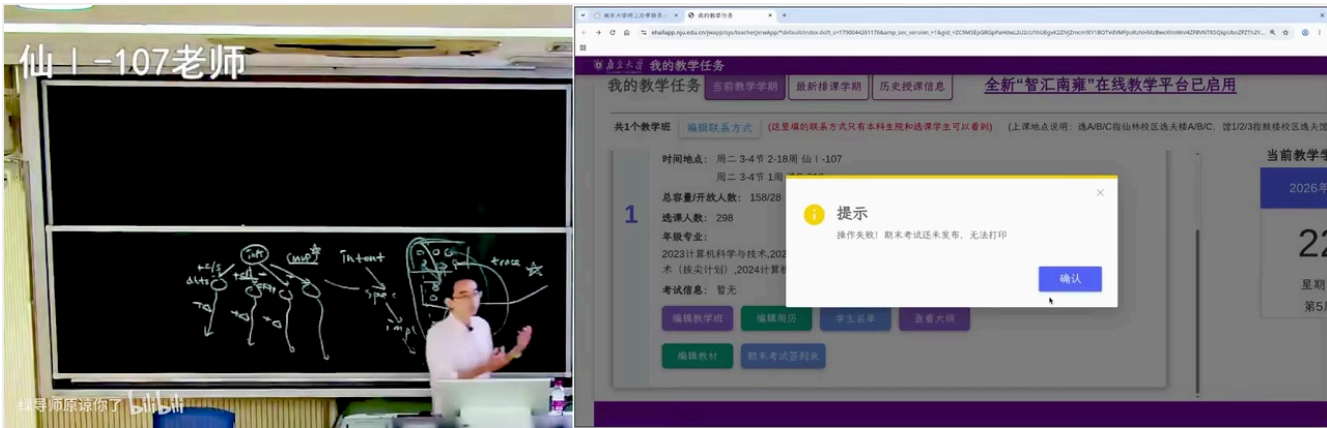
3. 如果他用 [PAGENT] (原词; 疑为某 Agent 工具) 去找一下, 可能只要一毛钱两毛钱。
- 因此他提出一个反事实判断: 把 token 补贴送给大家, 然后不要实现任何功能, 可能更便宜。这是讲者的个人观点, 不是系统方声明。
 - 讲者明确说, 这里是在讲需求。

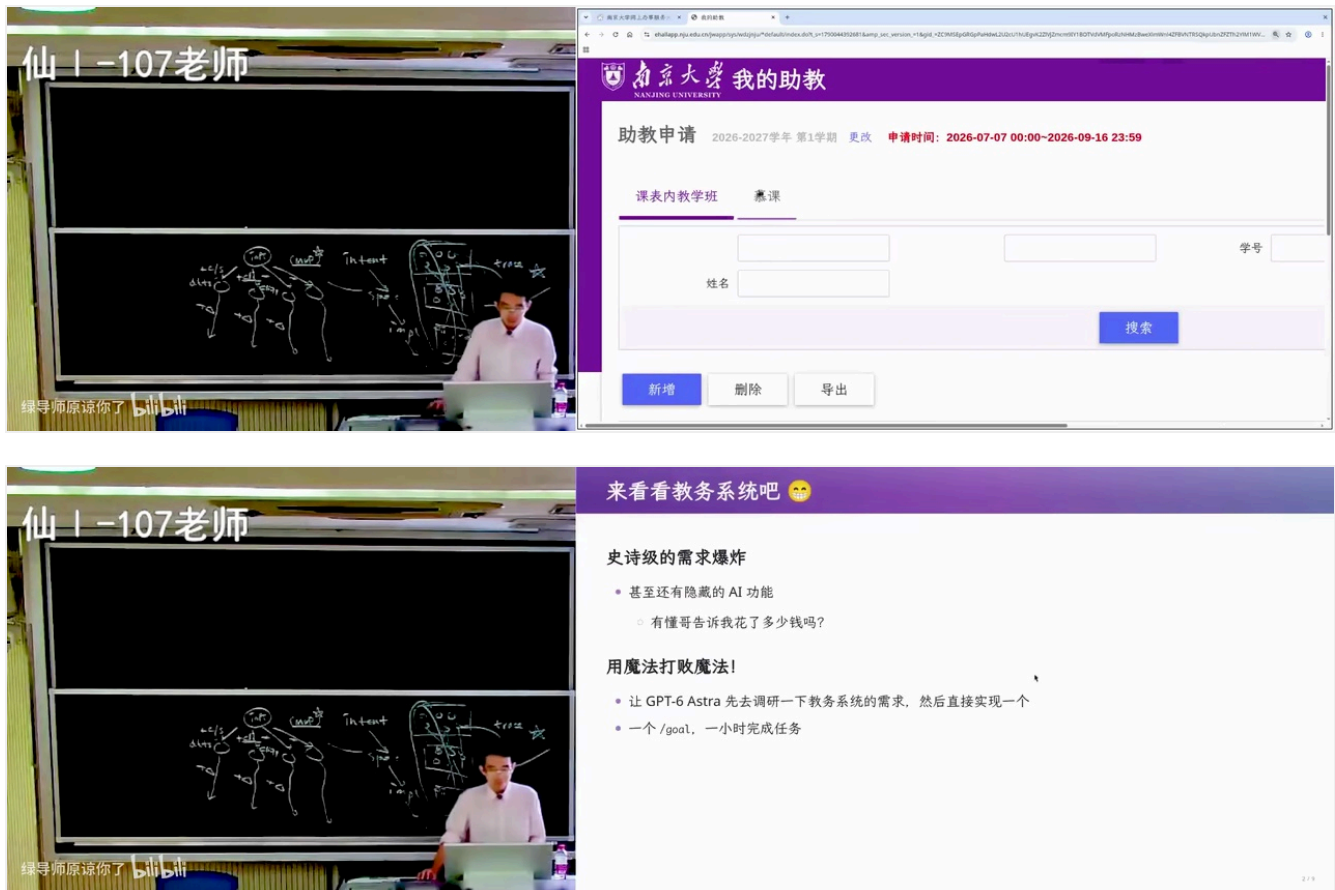
典型业务系统会堆出无穷功能; 需求分析做了, 不等于优先序对

- 讲者说, 这样的系统就是非常典型的业务系统。除了查课表, 他还应该能查教学任务, 也能看到他收藏的内容。
- 在教学任务/名单相关信息里可以看到大家的信息。讲者不点开, 因为点开可能显示电话号码之类。他直接说这是一个比较糟糕的 feature。学生名单里也能看到电话号码。
- 教学周历方面, 他上一次演示过: 直接用 AI/slop 来填满教学周历 (原音近“教学周底”)。讲者说“没有人管我”“没有任何人来审计我填的任何内容, 至少还没有”。
- 讲者认为, 他们更应该把教室的灯修好。
- 界面里有“无穷多的功能”, 例如:
 - 编辑联系方式;
 - 当前教学学期;
 - 最新排课学期;
 - 历史授课信息: 历史授课信息里要选择学年学期; 讲者觉得“不太好用, 又还蛮好用”;
 - 讲者认为这说明他们还是做了需求分析。
- 另一个功能点是期末考试签到表: 期末考试期间允许点击这个按钮; 如果期末考试还没发布, 就会弹窗告诉你无法打印。
- 还有例如编辑教材。
- (整理者串联) 从这些例子看, 做了需求分析、实现了功能点, 不等于需求优先序正确: 电话号码暴露、教学周历无人审计、教室灯不修, 都是讲者用来质疑“功能堆叠”价值的证据。

小结: 讲者借教务系统说明, 自然语言入口和无穷功能本身不等于需求被满足; 当实现成本高、审计缺失、隐私暴露、基础设施未修时, 不如把资源转为 token 补贴, 并优先处理真正的问题。

第 7 部分





现场演示教务系统：需求叠加会制造“史诗级 bug”

讲者这一段的推理链：

1. 教务系统里的功能不是一次设计完成的，而是需求不断叠加出来的。
2. 讲者把这种叠加与助教功能里的“史诗级 bug”放在一起讲；**（整理者串联）**这说明需求叠加会让系统复杂、边界条件增多，容易出 bug。
3. 这解释了为什么真实客户系统很难承包：要签合同，逾期交付、达不到功能/非功能要求都会被追责。
4. 学生不敢承包，agent 敢；所以讲者让 GPT/agent 去探索教师相关功能。
5. 结果得到《南京大学本科教学服务平台需求说明书》，作为真实需求报告案例。

期末考试打印按钮：先看正常约束与弹窗

- 讲者演示一个按钮：在期末考试期间允许点击。
- 如果还没有到期末考试发布的时候，系统要弹出一个窗口，告诉用户无法打印。

编辑教材：点击后跳转/弹窗，“曾经好像可以”，讲者判定发现 bug

- 进入“编辑教材”后进入另外一个页面，并有一个弹出窗口。
- 讲者说“再试一次编辑教材”，又回忆“曾经好像是是可以的”。
- 随后讲者说：“我们已经成功发现了一个 bug。”

查看大纲：刷新慢、只展示最新教学班，可能不满足功能性需求

- 查看大纲时，大纲显示的是教学安排，并且仅展示最新一个教学班。
- 页面等了很久才刷新出来。
- 讲者认为这个表现可能不满足它的功能性需求，并提到“状态正常”这一要求。

其他点击：讲者坚持“古法”手动操作

- 讲者说还有没有别的可以点的，自己不用 A 键的方式，而是用“古法方式”来点。
- 看到的是“我的教学任务”，以及和本科生相关的功能。
- 还有很多功能涉及助教。

助教功能把“需求叠加史”暴露出来

助教查询：本课无助教，课表内教学班找不到助教

- 讲者点进“我的助教”，说“我这门课好像没有助教”。
- 他尝试搜索，先问“我能搜我的名字吗”，随后发现“不对这是学号”。
- 页面显示“这是我班上的助教”。
- 接着出现“找不到课表内教学班没有助教”。
- 在相关页面里，讲者看到一些 JX，感叹教学班名称“堪比 agent”。
- 讲者从搜索/查看中看出：这个需求是不断叠加的。

需求叠加史：疫情前无慕课 → 疫情加慕课 → 加慕课助教 → 慕课助教特殊化 → 史诗级 bug

讲者给出的具体历史链：

1. 曾经没有慕课这样的功能；毫无疑问，疫情之前没有慕课功能。
2. 疫情期间，领导说需要为老师添加慕课。

3. 添加完慕课以后，需要慕课助教。
4. 慕课助教不能当成普通教学班的助教，要把它当成慕课助教。
5. 于是就有了一个“史诗级 bug”。
 - 讲者说：“我不知道为什么，但是这是一个史诗级的。”

删除/新增助教：可维护时段与当前不可申请

- 讲者尝试点“删除可以吗”，又点“新增可以吗”。
- 系统提示：“请在可维护时段内操作课内教学班”。
- 新增时提示：“新增请在可维护”。
- 讲者总结：现在已经不能申请助教了。
- 讲者说：“Anyway，这就是你看到的教务系统。”

从教务系统推到承包责任：学生不敢，agent 敢

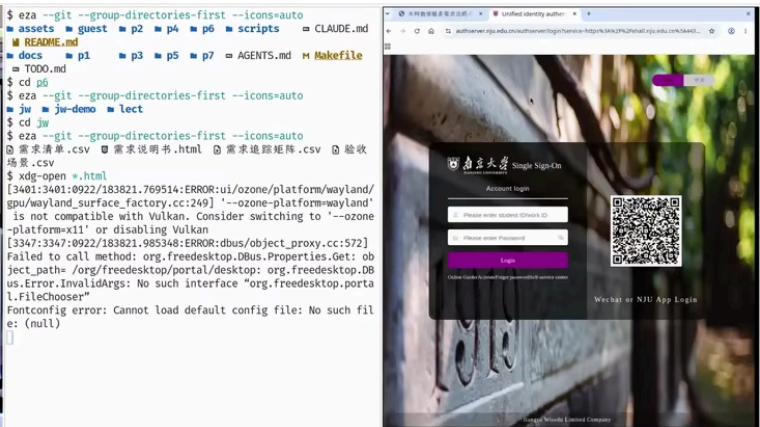
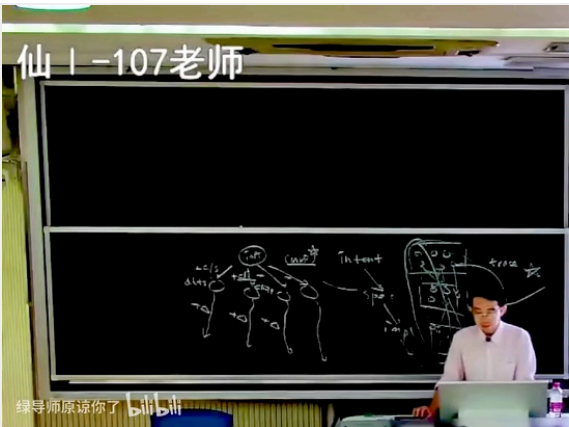
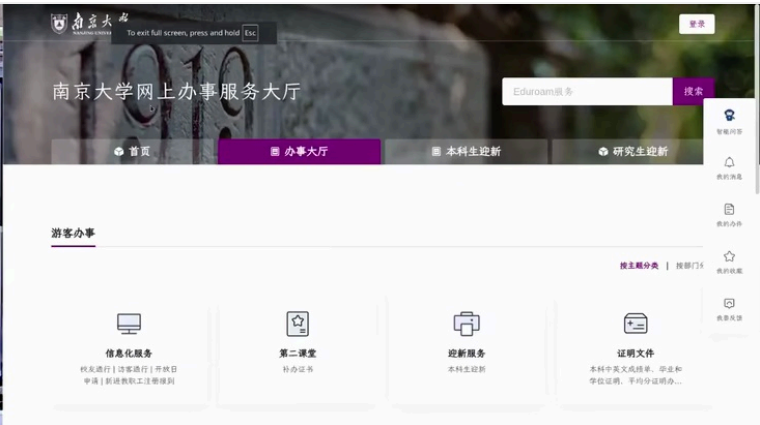
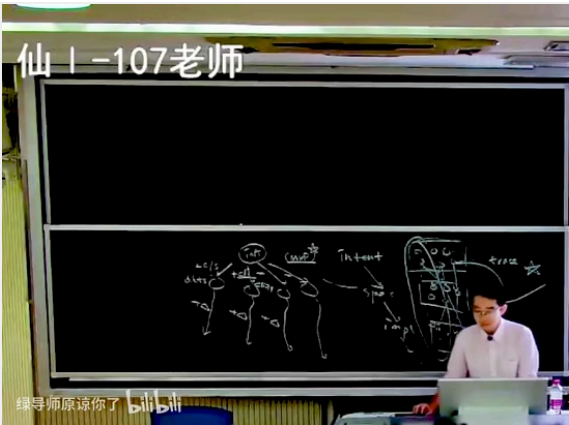
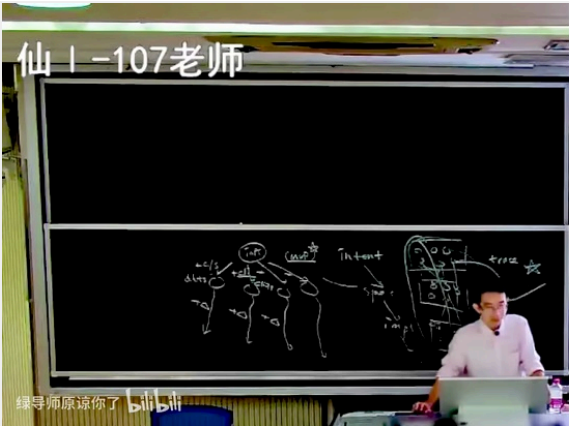
- 讲者由此让听众理解上一页说的“曾经很难讲”。
- 学生不会见到真正的客户，也可能不敢拍胸脯说：“给我 20 万，我找几个同学，把南大教务系统包了。”
- 因为一旦签合同，如果逾期交付，或者达不到**功能要求**、达不到**非功能要求**，就承担不了责任，会傻眼。
- 讲者说：成年人当然可以成立公司，去和学校签订这样的协议。
- 但“你们不敢不要紧，agent 他敢就行”。

让 agent 生成真实需求报告案例

- 讲者因此做了一件事：让 GPT 帮忙，派 agent 去爬/探索教务系统里面与**教师相关**的功能。
- 讲者特别强调：**注意只是教师相关的功能**。
- 目标是得到一个**需求说明书**。
- 结果显示这份需求说明书还是很不错的：《**南京大学本科教学服务平台需求说明书**》。
- 讲者说：这就是一个**真实的需求报告案例**，可以在软件工程课上看到这样的案例；软件工程课上也可能会展示一些来自.....

小结：需求不断叠加会让教务系统出现“史诗级 bug”和边界限制；真实客户系统的合同与功能/非功能责任让学生不敢承包，而讲者让 agent 去探索并生成了《南京大学本科教学服务平台需求说明书》。

第 8 部分



真实需求报告：把身边案例直接变成需求说明书，不是课堂节选

- 讲者展示的是一份**真实需求报告案例**。软件工程课上可能看到经典案例，或真实企业需求说明书的一部分，但讲者强调“这个就是真的”。
- 讲者认为 **AI 很好用**：要教需求工程时，当然可以找经典案例，也可以把**身边的案例直接变成需求说明书**。
- 目录里项目很多：**调研方法、证据边界、业务目标、成功标准、详细用例规约、功能需求目录**，最后还有**术语表/术语文档与配套文件**。
- 讲者说 **GPT（听写原文为“GBT”）懂软件工程**：初看像已经做好系统的 **slop**，突然写了很多东西；但它懂得，真正面对客户做工程时，**术语很重要**——客户看不懂时可以查术语，从而对齐理解。
- 讲者判断：即使不是 UML、仍处在早期意图沟通阶段，这些内容显然也有用处。
- （整理者串联）这里的**关系是：真实工程面对客户 → 需要共同语言 → 术语表和配套文件让沟通可对齐；不是单纯堆文档**。

形式语义与 ID 是 traceability 的基础

- 讲者看“需求功能”目录，指出它遵循较好的软件工程规范，并且有 **traceability**。
- 每个需求点前有语义字段，如**角色/参与者/业务目标**等；这些字段提供一种**形式语义**。
- 对比：自然语言可以说“教学班和学生管理有关”，但用**有形式意义的编号/ID**，就能在一定程度上帮助建立 **traceability**。
- 讲者明说：这比只靠自然语言描述更好。
- （整理者串联）逻辑链：**需求有角色、目标、ID 等形式语义 → 需求之间可编号、可引用 → 形成 traceability**。

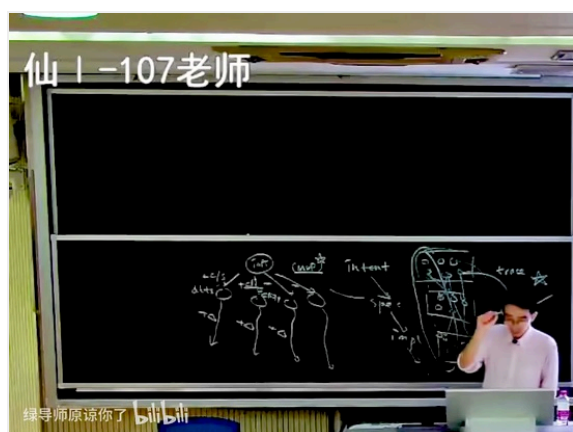
功能需求例子：身份选择不是多余，而是真实软件复杂性的显式化

- 讲者查看功能需求，看到“**门户与身份提供分类服务 / 检索和应用入口**”等条目。
- 具体需求写到：当一个服务支持多种已分配身份时，要提供**身份选择**，并显示**当前身份**。
- 讲者强调：真正做软件时，复杂性就在这里。你可以不写这些，但“有”和“到底有没有”差别很大。
- 他用实际观察佐证：在“本科生/教师课表”等界面切换时，确实看到系统让他选择身份；他选了其中一个身份。
- 需求还涉及：在多种身份下，服务页面要明确当前**学期、学年或业务批次**，并允许在**授权范围内切换**。
- 讲者认为这也是一种设计：在意图阶段，不光领导，一线教师也会关心“打开页面后什么样的界面比较舒适”。交互系统有一定设计感，会预设用户做这件事时想做什么。

- 因此系统会留一个“job box”（听写原词；从上下文看是选择控件），用户可选学期等；讲者顺带举到“教学周/课程与大纲/教材选用”这类内容，说明这种设计意图。
- （整理者串联）逻辑链：真实用户有多个身份和业务上下文 → 界面需要身份选择与当前上下文显示 → 需求文档要把这些显式写出来 → 否则实现时“有/没有”会造成巨大差异。

小结：讲者用真实需求报告说明，目录、术语表、形式语义、ID 和身份选择等细节，都是为工程沟通、客户对齐与 traceability 服务的。

第 9 部分

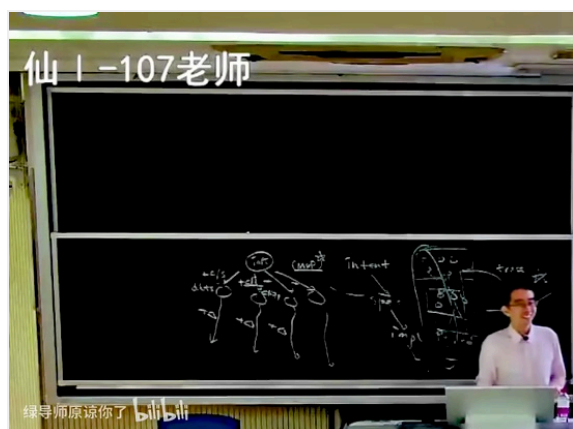


6.1 门户与身份

ID / 优先级	系统应提供的行为	候选验收口径	依据
FR-001 / M	提供本科教学服务分类、服务检索和应用入口。	教师可定位六类本科服务，并从入口到达对应应用。	O - E01
FR-002 / M	当一个服务支持多种已分配身份时，提供身份选择并显示当前身份。	选择本科教师组后页面显示该角色；未分配角色不得成为可选项。	O - E12/E13
FR-003 / M	服务页面明确当前学期、学年、批次或业务批次，允许在授权范围切换。	改变上下文后列表页同步更新；无记录时保留所选上下文。	O - E02/E06/E09/E21
FR-004 / M	在门户与子应用之间复用统一身份认证，并妥善处理认证失败。	跨应用免重复登录；会话失效时说明原因，未提交内容按约定恢复且不得丢失。	P - E01/E12

6.2 教学任务

ID / 优先级	系统应提供的行为	候选验收口径	依据
FR-005 / M	分别展示当前教学学期、最新排课学期和历史授课任务。	三个入口可区分：历史任务按指定学期及课程条件查询。	O - E02
FR-006 / M	按教学班展示课程、学分、任务状态、教师、周次、节次、地点、容量、选课人数及考试信息。	一个教学班包含多段排课时分别展示；空考试信息明确为暂无。	O - E02
FR-007 / M	提供教学班信息维护，包括考核方式、成绩标志、授课类型、语言、成绩构成及备注。	打开表单显示现值；取消不写入；合法保存后重新打开能看到更新。	O - E02
FR-008 / M	提供教师联系与方式维护，并在输入时说明可见人群。	页面说明仅本科教师与选课学生可见；链接读取验证使用。	O - E02



教务系统里的教学任务与教材选用：典型“TDS”低收益事务

- 讲者描述操作：在界面里选 combo box，再选一个学期或某个对象，看到 教学任务、课程与大纲、教材选用 等条目。
- 讲者判断：这就是一件“非常 TDS”的事情，TDS 到他根本不想干。TDS 的具体含义他没有展开，保留原词。

- 理由链：填这些条目 → 对获得学分、获得长进没有本质好处 → 不如去做以前没想过的算法题 → 能提高算法能力 → 在保研机试里多拿分。
- 因此，一条条往后写这种内容，讲者认为非常 nonsense，自己不会想写。

但需求文档写作会“量变引起质变”

- 讲者把话题转到课程作业里的 **需求文档**：数字逻辑电路课作业、计算机系统基础课作业。
- 他的转折是：单个条目无聊、低收益，但需求文档/规格说明写多了会“量变引起质变”。
- 例子落在计算机系统基础课：写 NEMU 时“挺好玩的”，这是学生见到的第一个比较长的需求文档。

从 OJ 短 spec 到 ICS 长 specification：系统变大，训练相通

- 在设计计算机系统基础课之前，OJ 的需求文档很短：明确输入接口、输出接口；接口通常很简单，一般就是输入一个整数或空格分隔整数，用 `cin` 就能读。
- 计算机系统基础课第一次给出较长的 **specification guide**：要求实现 NEMU。第一个 lab 可能是实现调试器，不需要实现指令解释；第二个实验需要实现指令解释。
- 关键链：任务表面上只有“实现指令解释”一句话 → 但“到底什么是指令”写在文档里 → 指令行为在文档里 → 学生得到另一个更大的需求文档。
- 讲者结论：系统大到一定程度后，做的训练都是相通的；学计算机系统基础课，一定程度上也学了 **软件工程**。

需求工程的管理价值：验收场景、需求清单、traceability

- 讲者说这是需求工程里的内容，“你为了挣钱”需要它。
- 具体结构：**验收场景**（讲者重复强调）、**需求清单**；这是一个非常好的结构化 checklist。
- 管理链：
 1. checklist 可拆分：前 20 个需求分给一个团队，后 20 个需求分给另一个团队；
 2. 可以 **track** 每个人的进度；
 3. 能看出哪个团队落后于实现需求、落后于进度；
 4. 有了 **traceability**，作为管理者就能把团队管起来。
- 讲者补充：这些内容本科上课时其实不怎么学。

AI 的魅力时刻：只给需求说明书，让 GPT sub agent 实现假 Java 系统

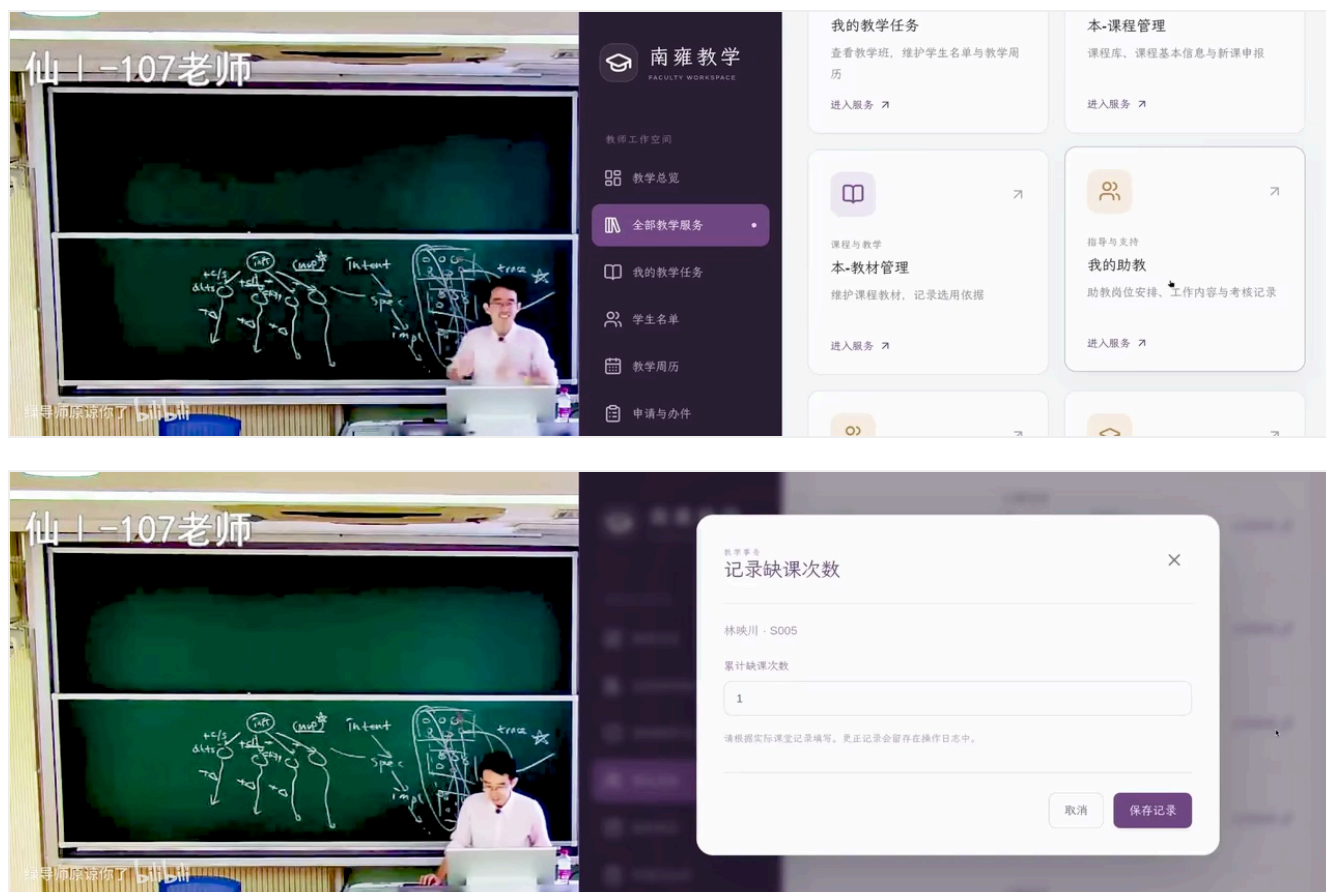
- 讲者说他“又干了这样一件事”：让 GPT 起一个 sub agent。

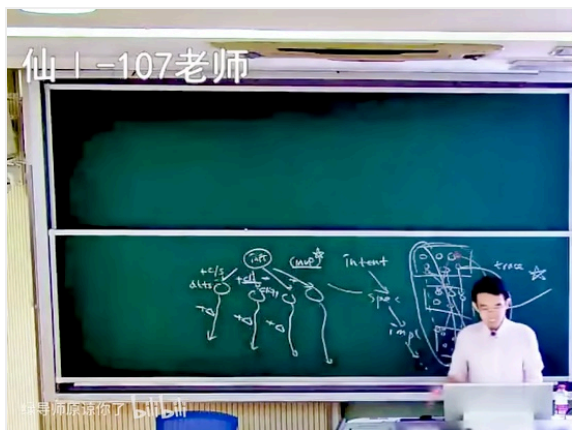
- 约束：这个 sub agent 不要看任何其他教务系统，只根据需求说明书实现一个假的 Java 系统。
- 结果：讲者觉得“突然又有信心了”，觉得“好像我也可以”，并开玩笑说可以 20 万跟学校谈一个合同。他还说生成出来的东西跟他刚才看的那个挺像。
- 这里他没有展开技术细节，重点是 AI 依据需求说明书做原型，带来信心和“原来我也能做”的感觉。

回到教学任务页面：教材管理与《软件工程实践者的研究方法》

- 末尾讲者返回的入口/路径是：全部教学任务 → 我的教学任务 → 本科教材管理，并提到“软件工程实践者的研究方法”。

第 10 部分





来看看教务系统吧 😊

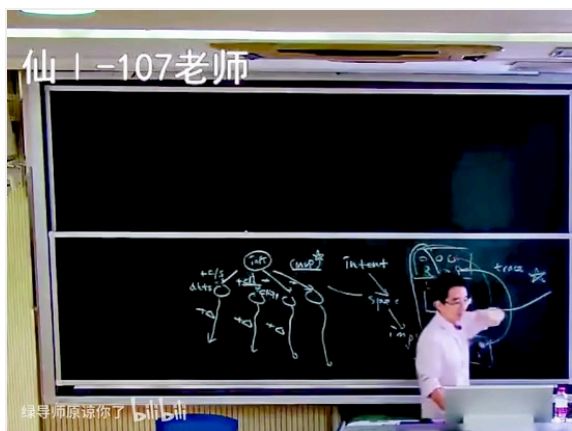
史诗级的需求爆炸

- 甚至还有隐藏的 AI 功能
 - 有懂哥告诉我花了多少钱吗？

用魔法打败魔法！

- 让 GPT-6 Astra 先去调研一下教务系统的需求，然后直接实现一个
- 一个 /goal，一小时完成任务

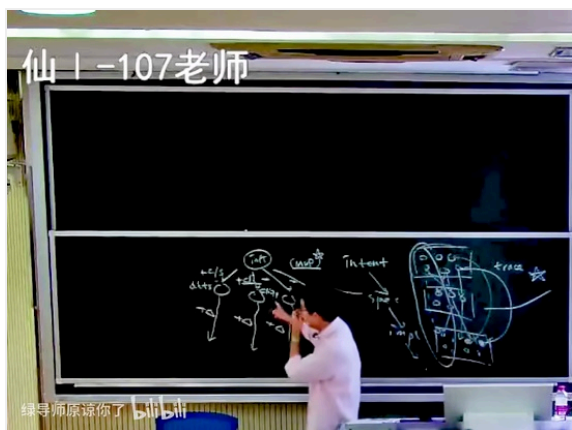
2 / 3



教务系统：总需求分解，从入学到毕业



3 / 3



教务系统的需求泥潭

按照经典软件工程做产品的思路

- 把需求方访谈一遍，期末考试都结束了
- 学生、老师、教务员、教务处、财务、辅导员.....各有各的需求
- 更糟糕的是：客户说不清楚明确的需求
 - 现实世界的奇葩规则：必修课二选一、准出规则、先修规则.....
 - 永远存在的例外：转专业、交换生、缓考、补考、课程替代.....

说清楚了，也未必能达成一致

- ③ 学生要选上课，教师要控制班额，教务员要保证毕业：冲突由谁裁决？
- ③ 城市规划早已遇到这种困境：问题怎样定义，也取决于各方的利益与价值判断。Rittel & Webber, 1973

4 / 3

讲者演示：不合理缺课记录也能保存，系统没有阻止

- 讲者回到全部教学任务 / 我的教学任务，尝试编辑记录，并演示保存；系统提示记录已保存 / 成功保存，还能看到学生名单。
- 讲者调侃系统里的命名带有 GPT 起名字的偏好：偏文雅，但也有自己的“偏执”。
- 他演示记录一个缺课数据：数值超过该学期上课次数的总和，系统没有阻止；讲者认为这条记录其实不合理。

- 随后他转向“软件工程的魅力时刻”：当需求清楚、需求文档写到那个程度时，**编码、可追溯性 (traceability)、测试**，AI 都能帮你搞定；你只需要一份**需求**。所以这是**需求工程**，后面还会回到这里。

需求工程：通过无尽沟通把客户想要什么想清楚

- 讲者展示了一个从入学到毕业的教务系统，包括权限规则、版本、数据同步、操作留痕等各种功能。
- 由这个例子说明：需求无非是通过**无尽地和客户沟通**，把客户想要什么想清楚。
- 小到界面要不要放**当前学期和学年**、某个地方要不要有**修改按钮**，都要和用户对齐。
- 原因：**用户不是你自己**。设计、尤其是做需求时，这是普遍问题；CS 同学常缺这种**软技能**，需要**同理心**。

同理心的具体对象：53 岁教务员，而不是会写程序的 geek

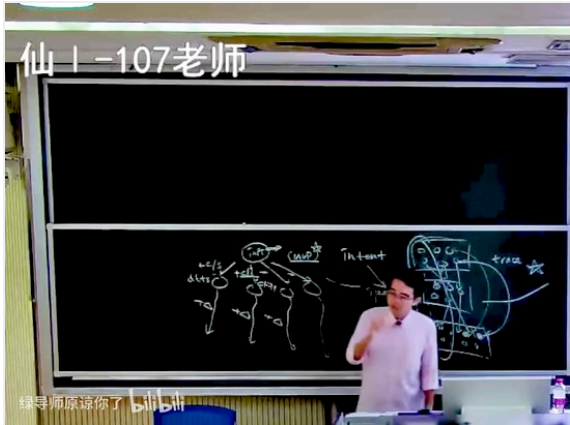
- 讲者反对“我是 geek，就假设所有人都会写程序”的思路。
- 设计者要考虑的教务员可能是：**本科学历、53 岁、还有两年退休、一位优雅的女士**；她上大学时可能甚至没操作过电脑。
- 系统既要为她设计，也要为讲者自己、为年轻人设计。
- 讲者指出：怎么得到“**那一张表**”完全是另一个问题，不是这门课重点。

课程重点：长需求列表已经进入泥潭，领域经验决定需求偏差

- 讲者认为：当教务系统已经像刚才那样有一长串需求列表时，其实已经进入**泥潭**。
- 如果真要做交互系统，你被迫**从零开始**。
- 这是**行业壁垒和经验**：如果有行业资质，或在行业摸爬滚打 **20 年**，公司就有很多**领域知识**，知道教务员想要什么。
- 本科生/毛头小伙子不知道教务员到底想要什么，所以设计出的软件、做出的需求很有可能有**偏差**。
- 逻辑链：缺领域知识、缺对真实用户的理解 → 不知道用户要什么 → 需求有偏差 → 软件有偏差。

小结：讲者把重点从“AI 能写码”拉回需求工程：需求清楚则 AI 可执行；需求脱离真实用户，长列表就会变成泥潭。

第 11 部分



教务系统的需求泥潭

按照经典软件工程做产品的思路

- 把需求方访谈一遍，期末考试都结束了
- 学生、老师、教务员、教务处、财务、辅导员.....各有各的需求
- 更糟糕的是：客户说不清楚明确的需求
 - 现实世界的奇葩规则：必修课二选一、准出规则、先修规则.....
 - 永远存在的例外：转专业、交换生、缓考、补考、课程替代.....

说清楚了，也未必能达成一致

- <3> 学生要选上课，教师要控制班额，教务员要保证毕业：冲突由谁裁决？
- <3> 城市规划早已遇到这种困境：问题怎样定义，也取决于各方的利益与价值判断。Rittel & Webber, 1973

有行业经验的人能把学位授予流程翻译成需求；本科生容易把需求做偏

- 讲者指出：一个毛头小伙子、本科生，不知道教务员到底想要什么，所以设计出来的软件、做出的需求很可能有偏差。
- 讲者认为：自己有这个行业的经验，对学位授予流程比本科生更清楚，知道流程应该怎样变成需求文档列表。
- 讲者补充：某种程度上，做软件都是被迫的。

空项目没有漂移，但第一版需求一被实现就背上技术债

1. 刚开始是一个空项目，没有任何东西在漂移。
2. 一旦写出一个需求列表，不管是 AI 还是人类，马上就会给你实现一版，就像前面看到的教务系统一样。
3. 实现出版本以后，你就背上了技术债。
4. 讲者说，这是因为他们的定律是：需求永远不可能真正说清楚；其次，需求不是一成不变的。

案例：离散数学拆成两门，旧的毕业规则立刻失效

- 讲者说，这是 AI 帮他虚构的一个案例，但他大概能体会，实际可能在学校里发生。
- 做法是：把离散数学课拆开。原因是讲者他们觉得第一门离散数学没有教会大家数学的语言，所以把它拆开。
- 拆开以后带来非常糟糕的后果：
 - 准出结果变了；
 - 保研要求变了；
 - 新旧要求衔接不上。
- 旧系统里的规则是：计算机系或计算机学院的学生如果要毕业，必须选修离散数学。讲者认为这合理。
- 新情况是：离散数学从上学期一门课，变成了上下学期两门课。
- 于是需要在系统里增加规则：允许毕业，要么选修了离散数学一……讲者强调这里形成一个或的关系。（转录中另一支选项未完整展开。）

第一版“毕业要求”界面在当时合理，但无法容纳“或”

- 最早第一版沟通需求时，肯定没有想过离散数学会被拆开，所以系统里没有对应规则。
- 当时写需求 checklist 时，大概会这样写：
 - 界面上有一个叫“毕业要求”的地方；
 - 在毕业要求上，要给教务员一个下拉框（讲者推测操作者应该是教务员）；
 - 有一个“增加一条”按钮；
 - 点“增加一条”后，可以选一门课程。
- 讲者判断：在刚开始做需求的那个时间点，这个需求非常合理。
- 但一旦这个需求固定下来，离散数学拆成两份后，就完蛋了：这个设计不能适应新的需求。
- 对应动作只能是：重构。把需求文档再调出来，原来地方不能这样了，要在这里加一个“或”，下拉框里也加一个“或”。
- 每叠加一次这种东西，屎山就开始累积。讲者说，写过软件的人都懂。

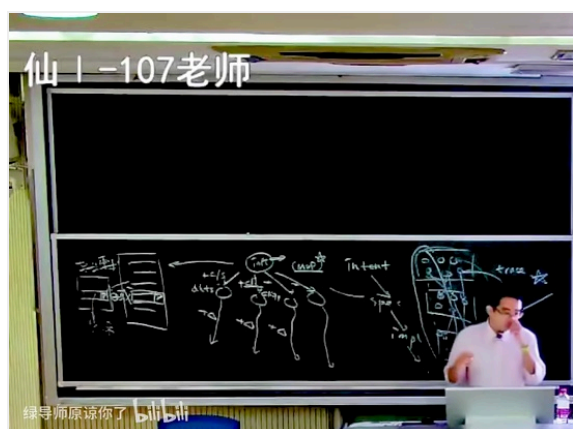
改需求真正破坏的是整体一致的设计，屎山由此累积

- 讲者个人意见：你让我做个东西可以，但改需求真的受不了。

- 原因：改需求会破坏你原本做的**整体上一致**的设计。
- 一旦需求中间产生一个裂缝以后.....
- （整理者串联）这个案例的链条可概括为：

第一版需求合理 → 被实现并固定 → 真实需求变化（离散数学拆分） → 旧规则和界面不能适应 → 重构、加“或” → 整体一致的设计被破坏 → 技术债/屎山累积。

第 12 部分



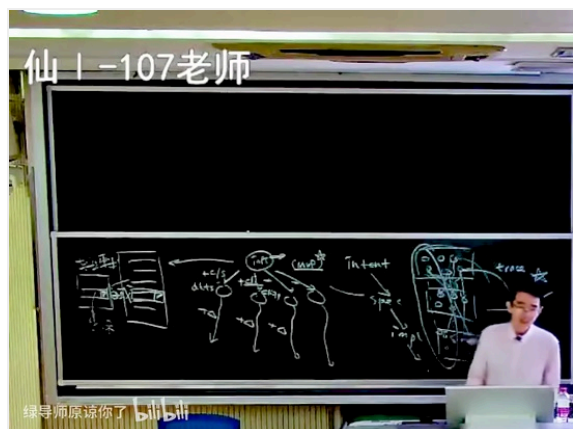
培养方案：学分够了，就能毕业吗？

一个虚构但可能发生的案例

- 2024 级李同学在 2026 年转入计算机专业，适用目标专业的 2024 版方案。
- 毕业至少 160 学分：通识 40、专业必修 72、专业选修 28、实践 20，各类分别达标。
- 数学必修组二选一：高数 A（5 学分），或高数 B（4 学分）+ 桥接课（1 学分）。
- 算法课须先修数据结构：缓考生可暂选，开课前仍未通过则撤销。
- 两门文换课程获批准抵算法课，认定 4 学分：不能再计入专业选修。
- 重修学分只计一次，成绩单保留历史记录，绩点按较高成绩算。

最后的“暗箱操作”

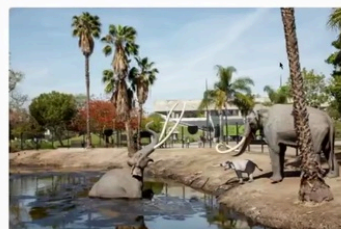
- 管理员凭线下审批，点“视为满足”覆盖指定学生的未达标项；记录理由、批准人和适用方案。



勉强实现需求，然后进入焦油坑

经常需要工程师 Hack 数据库

- 上课第一天：绕过前端校验，把一位同学的缺课次数改为 -1
- 结果：教务系统所有能选出这位同学选课记录的部分全部瘫痪



改需求为什么让人受不了：它破坏整体一致性，并让裂缝传递

讲者认为，改需求真正麻烦的地方是：它不只是改一个点，而是破坏原本“整体上一致的设计”。逻辑链是：

1. 团队先围绕需求做了一个整体一致的设计。
2. 需求中途变化，等于在这个一致结构里产生一道裂缝。
3. 裂缝会传递到系统里的各个部分。

4. 于是一个看似很小的需求变更，会牵动整个系统的各个部分。
5. 结果就是“一团乱麻”。

所以，问题不是单点修改，而是一致性被破坏后的连锁传播。

课程替代/抵扣：一旦面对开放世界，规则就无穷多

讲者用教务系统举例，提到**重修、交换、抵扣**，以及有些课程可以抵扣。具体情形是：

- 有一个同学从别的院系转过来。
- 别的院系又有一门**离散数学**课；软院好像也有对应的数学课（转录作“E3数学课”）。
- 那这门课能不能抵扣计算机学院的离散数学？
- 是抵扣**离散数学一和二**，还是只抵扣**离散数学一**，还要求再上**离散数学二**？
- 讲者说，这样追问下去就“无穷多”。

他的结论是：一旦面对开放的物理世界，系统就没办法穷举所有情况。而且到最后，系统里没有覆盖的东西还是可以“暗箱操作”，最终总还有一个 **override**。

系统里没有的，最后仍有 **override**：毕业条件可以被改掉

讲者举了一个毕业案例：

- 某同学已经到第 6 年，因为离散数学拿不到本科学位。
- 这时离散数学课被拆成两门。
- 时间上已经来不及：例如已经到了秋季/春季学期，马上就要毕业，不可能再去修那门（只在春季学期开的）课。
- 最后领导/教学主任说：他认为这两门课之间存在替代关系。
- 系统里最终怎么办？打电话给工程师，在数据库里把这个同学的毕业条件改成**满足**，就过去了。
- 讲者强调：这还是**档案可查**的，层层审批流程都会在。最终是**教学院长**认为，这个同学学过的旧数学课可以抵扣新的离散数学。

讲者认为，这在“人类世界”的意义上是合理的：制度上有一个合理解释。也正因如此，软件系统才复杂。

所以软件系统复杂：做需求分析就背上了技术债

讲者收束说：软件系统的复杂在于，任何时候实现需求、做需求分析，都会给自己“背上技术债”。需求不是孤立地被满足，而是进入一个已经一致、又不断被现实撕开裂缝的系统。

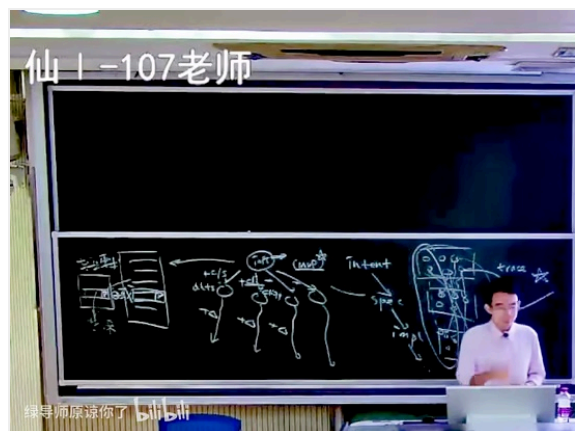
2018 年第一次用教师端：缺课次数功能为何被讲者认为没必要

讲者回忆：

- 开始上课第一天是 **2018 年**，很多年以前。
- 当时还是老版教务系统，和他自己上学时的教务系统差不多。
- 第一天他发现这个系统自己没用过：以前都是学生端，第一次用**老师端**。
- 他觉得很好玩：可以给同学设置**缺课次数**。
- 但他不知道为什么要做这个需求。讲者认为，如果由他来做这个需求，他不会把**缺课次数**这个 feature 放进教务系统。
- 理由是：同学缺课就缺课；不需要在系统里留证据。老师有签到表/点名表留证据，扫描一下，就可以开除？学生手册里有写。

（整理者串联）这个例子补充说明：在讲者看来，教务系统里的需求并不都来自不可省的必要性，有些 feature 本身就可以被质疑。

第 13 部分

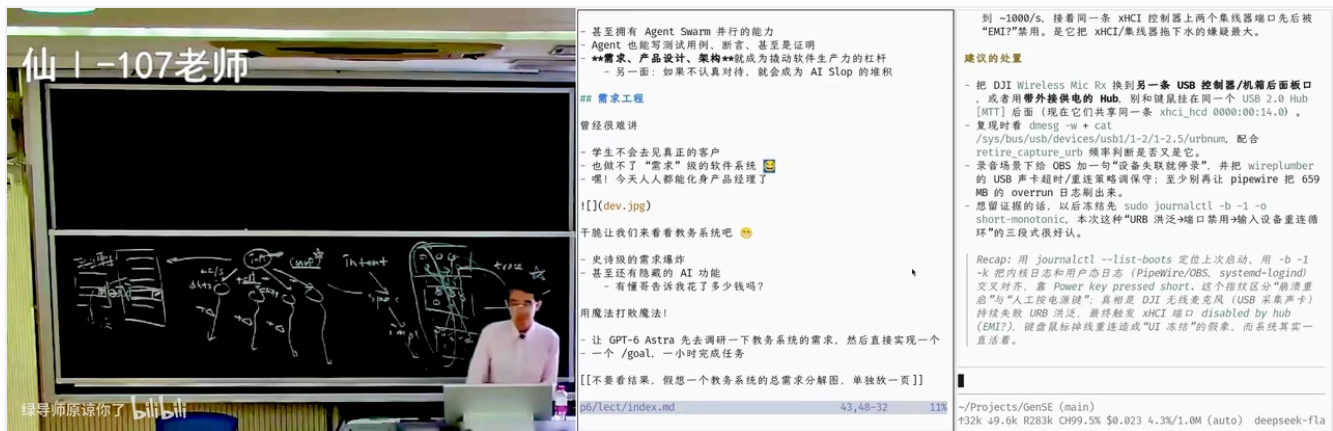


勉强实现需求，然后进入焦油坑

经常需要工程师 Hack 数据库

- 上课第一天：绕过前端校验，把一位同学的缺课次数改为 **-1**
- 结果：教务系统所有能选出这位同学选课记录的部分**全部瘫痪**





一次“0 改成 1”的事故：真实软件系统不是课程作业

缺课次数不是普通字段：它连着制度后果

- 讲者先给出场景：老师有签到表 / 点名表，扫描一下就留下证据。
- 学生手册里写了：无故缺课达到一定次数，就可以开除。
- （整理者串联）因此，缺课次数不是一个随便显示的字段，而是会触发真实制度动作的业务数据。

绕过前端校验失败，但小改一条数据让系统 500

- 讲者试着绕过前端校验。
- 他输入一个非常大的缺课次数，系统直接拒绝了。讲者认为刚才那个 GPT（转写为“GP”）实现的质量不错，前后端应该都做了校验。
- 但讲者把一位同学的缺课次数从 0 改成 1。他说这看起来也不是什么有害操作。
- 结果：所有能选出这位同学选课记录的页面全部 crash，直接 500。
- 讲者只好打电话给教务，说教务系统（原音转写为“交互系统”）崩溃了。

- 第二天，可能工程师登录数据库，把“1”的条目修复了。
- 讲者由此说：这就是真实软件系统，是你们真实面临的软件系统需求。

需求只是一个 lesson，本课只能带过

- 讲者说，这意味着“需求只是一个 lesson”（原词保留）。
- 他说明：自己没法在这门课上讲需求和产品设计；那可能应该是另外一门产品课。
- 这门课讲软件工程，不适合展开，所以只能这样带过。

AIGC 时代：MVP 空前容易，但会产生“驾驭 AI”的错觉

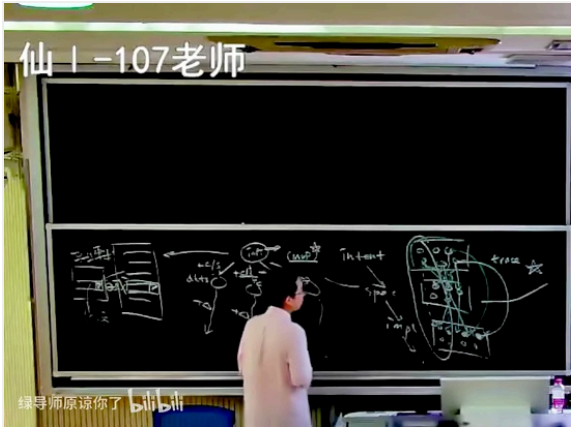
- 讲者认为，在 AIGC 时代（原音转写为“OGACAI”），你做 MVP 没问题、正确，而且前所未有地容易。
- 做法是：优先实现最重要的功能。
- 你看到一个产品，马上就知道哪个地方设计错了；这个时候推导成本比较低。
- 但这也会产生一种驾驭 AI 的错觉：
 - AI 确实可以往后修；
 - 你迭代三个版本，AI 都可以往后修。
- 可一旦在真实系统里运行足够长的时间，比如“计算机系改成计算机学院”，原来的设计不起作用时，就会面临麻烦。

软件工程没有银弹：早期决策会变成技术债

- 讲者说，这也是所有软件工程都面临的问题：there is NO silver bullet，没有银弹。
- 他解释“银弹”的典故：吸血鬼和狼人没法普通杀死，需要一颗 silver bullet；但软件工程里没有银弹。
- 随着软件增长，你在最早时刻制定的那些决策会不断产生影响，因为软件要生存、挣钱。
- 如果这只是课程作业，讲者说随时可以清零、从零重做。
- 但软件一旦上线，就开始和物理世界产生交火，必须持续背着 technical debt 维护下去。
- 成本会因为前面做得不够前瞻而不断变糟糕。
- 讲者说这是需求问题；解决方式是架构。他之前已经透露了一点：刚开始时你可能有需求，或处在 MVP 阶段，还没试图把所有要做的事情都放进来。

小结：讲者把问题从一次小数据事故带到架构：MVP 容易，长期演化难，早期决策会变成技术债。

第 14 部分



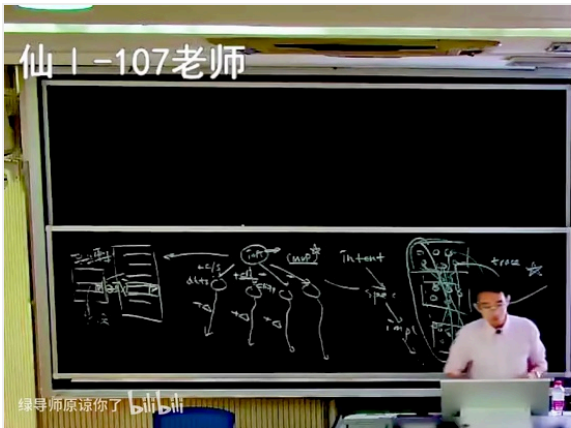
软件架构：真正决定软件品味和能走多远的决定性因素

驾驭复杂性，应对变化

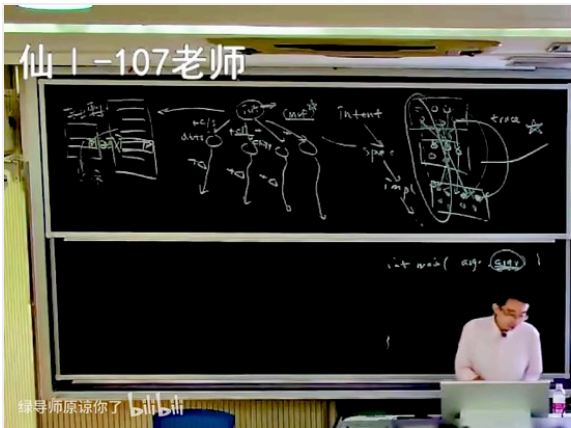
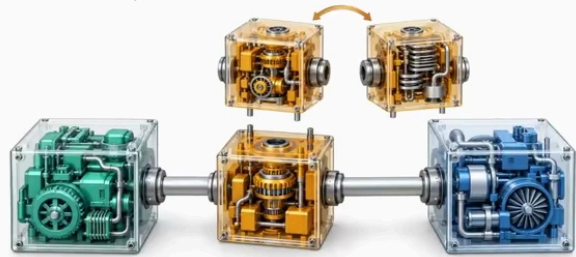
- 架构使我们驾驭问题的复杂性
 - 能简化问题、安放复杂性、减少依赖
 - 能应对需求的变化、降低维护成本
- 甚至再小的代码都有“架构”

把变化安放在边界内

- 把容易变化的设计决策藏在模块内部，让一次变化尽量只影响一个地方。——Parnas, 1972



好的架构：复杂性有归处，变化有边界



```
$ cd ..
$ eza --git --group-directories-first --icons=auto
├── jw ── jw-demo ── lect
├── cd jw-f
├── zoxide: no match found
├── cd jw-demo/
├── eza --git --group-directories-first --icons=auto
├── data ── node_modules ── scripts ── src ── index.html ── package.json ── tsconfig.app.json ── tsconfig.node.json
├── docs ── public ── server ── tests ── package-lock.json ── README.md ── tsconfig.json ── vite.config.ts
└── $ npm run dev

> nju-teaching-demo@0.1.0 dev
> node scripts/dev.mjs

(node:4180) ExperimentalWarning: SQLite is an experimental feature and might change at any time
(Use 'node --trace-warnings ...' to show where the warning was created)
Teaching API: http://127.0.0.1:4174

VITE v8.3.0 ready in 451 ms
→ Local: http://127.0.0.1:5173/
→ press h + enter to show help
```


第一步不是只做 MVP，而是做能承受需求变化的最小架构

- 讲者回应“这个问题”：在刚开始、只有需求或处于 MVP 阶段时，不要试图把所有要做的事情都塞进来。
- 要问的是：软件第一步迈向哪里，才对软件的长期生存最好？
- 讲者认为：第一步要做的实际上是软件架构，不仅仅是一个 MVP。
- 他提到刚才看到的 AI 生成的 slop：讲者认为那已经不能算 MVP，而算是一个小型产品原型；维护它会变得非常困难。
- 目标不是一开始就做全，而是做一个最小的软件，使软件在需求膨胀和改变时，依然有一定韧性。
- 关键动作是把耦合解开。例如一开始就立即决定“用一个数据库来存放所有需求”（原文如此）；这能快速得到一个能 work 的原型，AI 也能很快做出来；但如果不仔细思考系统里到底哪些概念可以分开，就很容易进入软件维护的泥潭。

讲者的链条：需求会膨胀和改变 → 最小代码也要有架构 → 架构用来控制复杂性、解开耦合 → 否则快速能 work 的原型会变成难维护的小型产品原型。

架构不是玄学：它本质上是控制复杂性

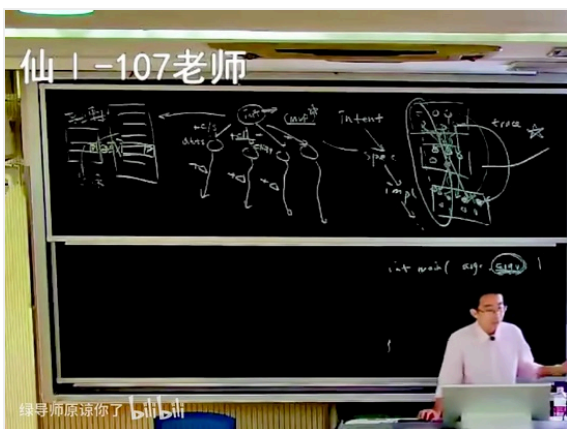
- 讲到架构，大家都觉得有玄学、好像很高大上；讲者认为其实不是。
- 从第一次设计任何代码、任何系统开始，无论它多小，里面都有一种架构思维。
- 架构本质上就是：怎样去控制一个东西的复杂性。
- 当代码哪怕只有一点点规模，就可以考虑：这个复杂性应该怎样去分解。
- 例子：写 OJ 时可能会写成 `main`，然后 `read input → process input → write output`。讲者用这个反问式例子引出“把一件事拆成几步”的思路。（整理者串联：这里体现了按阶段分解复杂性。）

命令行参数处理：getopt 是分解、隔开复杂性的架构

- 命令程序的 `main` 有 `argc` 和 `argv`；这是操作系统和进程之间的约定，会传一个 vector of arguments，把参数以字符串形式传过来。
- 如果手动处理参数，可能只想用 `string compare`。讲者反问：如果在保研机试现场，题目要求你处理 `argv`，你会 `string compare` 吗？

- 然后就会想起 `getopt`。你在课上看过这样的代码，在做实验时写过这样的代码，也会在别人的实际项目里看到，比如 `record is`（疑似 Redis，待核）、`ls` 这些命令怎么实现。
- 讲者说：`getopt` 也是一个架构，也是一种分解复杂性的方法，因为 `argv` 本身就是一个非常复杂的东西。
- 复杂在哪：例如 `npm help`、`npm run dev` 本身就有许多参数，有些还支持 UNIX 风格的长参数和短参数。在 C 的时代，写一个参数 `parser` 是很复杂的。
- 这里的架构就是：用 `getopt` 把参数解析的复杂性隔开。

第 15 部分



```

-U, --created           use the created timestamp field
--changed              use the changed timestamp field
--time-style           how to format timestamps (default
                        full-iso, relative, or a custom s
                        like '%Y-%m-%d %H:%M')
, iso, long-iso,
                        show the size of a directory as t
                        he size of all
                        files and directories inside (uni
                        x only)
--total-size           list each file's permission in oc
                        tal format
--no-permissions       suppress the permissions field
--no-filesize          suppress the filesize field
--no-user              suppress the user field
--no-time              suppress the time field
--stdin               read file names from stdin, one p
                        er line or other separator
--git                  specified in environment
--no-git               list each file's Git status, if t
                        rack or ignored
--no-git               suppress Git status (always overr
                        ides --git,
--git-repos            --git-repos-no-statu
                        s)
--git-repos            list root of git-tree status
--git-repos-no-status  list each git-repos branch name (
                        much faster)
-a, --extended         list each file's extended attribu

```

```

LIBRARY
Standard C library (libc, -lc)

SYNOPSIS
#include <unistd.h>

int getopt(int argc, char *argv[],
           const char *optstring);

extern char *optarg;
extern int optind, opterr, optopt;

#include <getopt.h>

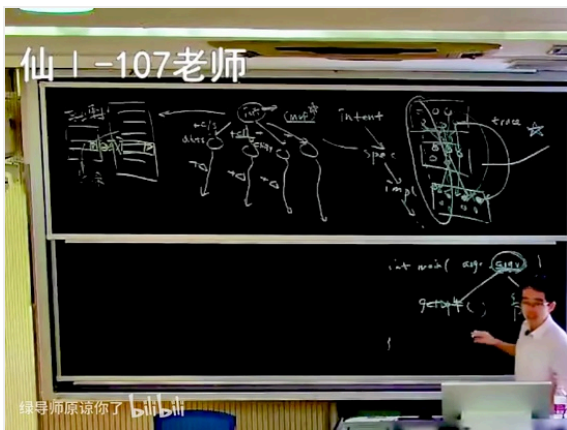
int getopt_long(int argc, char *argv[],
                const char *optstring,
                const struct option *longopts, int *longinde
                x);
int getopt_long_only(int argc, char *argv[],
                     const char *optstring,
                     const struct option *longopts, int *longinde
                     x);

Feature Test Macro Requirements for glibc (see fea
ture_test_macros(7)):

getopt():
    _POSIX_C_SOURCE >= 2 || _XOPEN_SOURCE

getopt_long(), getopt_long_only():
    Manual page getopt(3) line 7 (press h for help or q to quit)

```



再小的代码也有架构

甚至最小的“状态机”就是架构

```

int main() {
    read_input();
    parse_input();
    process_input();
    write_output();
}

```

编程语言与架构

- 你学习的语法、库函数.....都可能和架构有千丝万缕的联系
- 所以我不喜欢老登那种“语言律师”的授课方式
 - <语言语义>的价值，是把设计意图变成约束：谁能改数据，谁负责资源，谁可以依赖谁。
 - <语言语义>回看这四个函数：数据怎样穿过各阶段？换一种输入格式，需要改到哪里？

命令行参数解析：C 的 `getopt` 展示“小代码也有架构”

- 讲者从命令行参数切入：在 C 时代，手写一个 `**parser`（解析器）**是很复杂的。

- 这里有一个架构：`argv` 有一组约定；这组约定背后有标准，即 **POSIX 标准**；在 POSIX 标准上，可以用 `getopt` 来处理；最后程序只需写一个 `while` 循环 + `switch-case`，就能解析参数。
- 讲者给出的链条：
复杂命令行参数 → `argv` / POSIX 约定 → `getopt` → `while` + `switch-case` → 一种 **parser 架构模式**。
- 讲者认为：这就把复杂性隔开了，让人学会一种 **parser** 的架构模式。
- 但讲者强调：这只是非常非常小的例子。看到任何好的、符合 **best practice** 的代码时，它背后都有架构设计，并且映射到需求。
- **（整理者串联）** 这里的关键不只是 `getopt` 本身，而是“用约定/库承担解析复杂性，业务代码只处理选项”的架构动作。

需求不只在“教务系统”：任何模块都能谈 **intent**、**specification**、**implementation**

- 讲者修正前文：刚才用“教务系统”讲需求有点过度。
- 讲者认为：做任何事、任何软件工程行为时，其实都有 **intent**、**specification**、**implementation**。
- 不一定非要一个 **monolithic** 的教务系统；在任何模块里，都可以谈需求、谈架构、谈优雅。

反对语言律师式授课：讲 **C++** 不等于念一行 **lambda**

- 讲者很讨厌那种“老登式”的**语言律师**授课方式。
- 讲者曾与学院老教师有较大分歧：讲者认为，若讲 **C++**，至少在 **C++11/14/17** 里的一些特性，以及这些特性应该怎么用；不能还停留在 [PRIERM] 时代。
- 老教师的回应是：“我讲了，我怎么没讲？”但他讲的是：**lambda** 就是“`a、b`，返回 `return a+b`”，没有了。
- 讲者反问：这“讲了”和“没讲”有什么区别？

从 getopt 往后想一层：parser state 抵抗需求变化

- 讲者提醒：学习时看到的每一个东西，不管是 GitHub 上好项目的 README，还是 `getopt`，都要再往后想一层——这可能是一种模式。
- 这种解析模式把一个复杂的字符串，最后变成一个简单的 **parser state**。
- 讲者认为这是一种抵抗需求变化的方式：`argv` 可以变化，`argv` 可以有各种各样的变化，但 `parser state` 把这个问题解决了。

架构与需求相连：需求面向开放未来

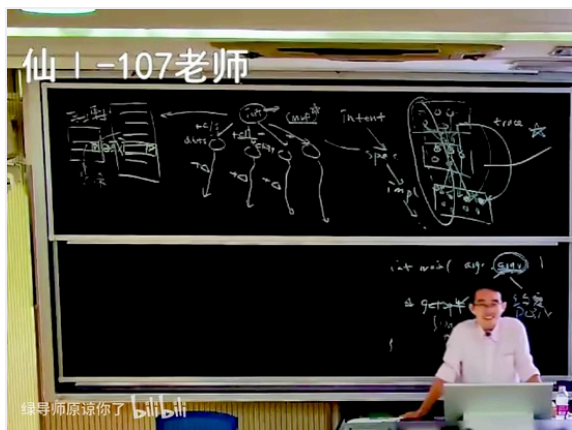
- 讲者认为：架构和需求其实是连在一起的；没有需求，就不用谈架构。
- 谈需求和架构时，需求面对的是一个开放的、为未来需求准备的架构。
- 所以，什么时候能设计出好架构，取决于你对需求掌握到什么程度。
- 讲者用这一点解释：为什么说大模型对架构掌控还不很好，或者说架构是一种品位——因为架构和需求相连。

十行代码的汉诺塔：非递归 Tower of Hanoi 也是架构例子

- 讲者举自己最喜欢的例子：非递归的 Tower of Hanoi（汉诺塔）。
- 以前开始上课时，讲者先问保研研究生：能不能写非递归汉诺塔？研究生摇头。
- 后来这成了讲者上课时的一个例子。
- 讲者称：就这十行代码，它背后的架构联系到大家。

小结：讲者用 `getopt` 和非递归汉诺塔说明，最小代码也有架构，而架构要从需求与未来变化来理解。

第 16 部分



10 行的架构例子

架构关乎你对需求“掌控到什么程度”

- 例子：我喜欢的非递归 Tower of Hanoi
- 背后的问题：大家到底为什么写不出来？
 - 同一个问题，换一种表示，难度就可能改变：汉诺塔同构变体的实验也观察到了这种差异。(Zhang & Norman, 1994)

非递归汉诺塔：十行代码背后的架构问题

- 研究生摇头这件事，最后成了讲者课堂上的例子：一段十行代码背后的架构，联系到“为什么写一个非递归汉诺塔那么困难”。
- 讲者由此让模型也来做同类题，比较不同模型对“架构/递归”的处理方式。

模型写非递归汉诺塔：数学性质 vs 栈模拟

GPT 路线：找到数学性质，代码极优但难懂

- 讲者让听众猜 GPT（原声像 GPU；后面又像 gt6/GPT-6，存疑）写了什么。
- 该模型找到 Tower of Hanoi 的数学性质： $[\text{盘号}] = 1 + \text{步号的二进制表示末尾 } 0 \text{ 的个数}]$
- 还提到“每个盘子的移动方向”；讲者说 **We don't know**，不确定方向是否固定，但判断“应该是对的”。
- 结果：代码 **highly optimized**、极度最优，但“你看不明白”。
- 讲者调侃：它明显是在数学道路上狂奔，希望用数学能力拯救人类世界的一切。

DeepSeek V4 Flash：标准、优雅的栈解

- 同一个 prompt，DeepSeek V4 Flash（原声 deep sk v4flash，模型名存疑）写得非常优雅，基本是标准答案。

- 讲者认为它抓住递归的执行结构：要把一些盘子移过去时，把剩下的任务留在栈里；总是执行栈顶任务。
- 对非递归汉诺塔来说，这是一份非常干净、优雅的代码。

豆包：选了同样方案，但实现有瑕疵

- 讲者特意试了豆包。
- 豆包选了这个方案，但写出的代码有瑕疵/错误，和标准答案差不多但仍不行。
- 讲者“呵呵”，不展示。

讲者的架构视角：写不出来，是递归理解不深

- 讲者以架构师身份强调：架构关乎你对需求掌握到什么程度。
- 所以他写非递归汉诺塔时，想的不是数学技巧，而是“为什么大家写不出来”；他的答案：对递归这件事理解不够深。
- 他的课堂讲法：程序实际上是一个状态机。
 - 参考操作系统课：任何 C 语言程序都可以被讲成状态机。
 - 一个 **single step** 是它的 **operational semantics**（操作语义）；再往下是 **小步语义**（**small-step semantics**）。
 - 例如汉诺塔规模减一（ $\text{Hanoi}(n-1)$ ）在程序语言世界里是 **well-defined** 的一步操作。
- 逻辑链：
 1. 把程序看成状态机。
 2. 把每一步定义成操作语义/小步语义。
 3. 只需要模拟这个状态机，就可以写一个 **C 语言解释器**。
 4. 理解 C 语言解释器后，把任何代码翻成状态机表示都不难。
 5. 讲者当时用两个互相调用的函数 **F 调 G**、**G 调 F** 说明：这只是把 C 语言的形式语义、小步语义翻译成状态机表示。
 6. 再把这种表示翻译到汉诺塔，就得到非递归实现。
- 讲者最后又回到：如果觉得这段代码不够优雅，对汉诺塔来说.....这一评价方向。

小结：讲者用非递归汉诺塔说明，写不出来不是数学不够，而是递归/状态机理解不够；数学性质的极优解可跑但难懂，栈解才直接暴露递归的执行结构。

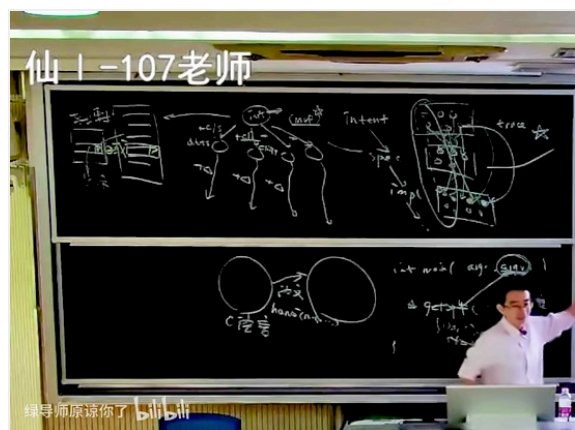
[illegible]

“绿导师：这是理解 C 语言“形式语义”的好机会

```

#define call(...) (*(++top) = (Frame){.pc = 0, __VA_ARGS__})
#define ret() (--top)
call(n, from, to, aux);
while (top >= stk) {
    Frame *f = top;
    int next_pc = f->pc + 1, n = f->n, from = f->from, to = f->to, aux = f->aux;
    switch (f->pc) {
        case 0: if (n == 1) { printf("%c -> %c\n", from, to); ret(); } break;
        case 1: call(n - 1, from, aux, to); break;
        case 2: call(1, from, to); break;
        case 3: call(n - 1, aux, to, from); break;
        case 4: ret(); break;
        default: assert(0);
    }
    f->pc = next_pc;
}

```



```

for(int i=0;i<len1;i++){ //遍历字符串1中的每一个字符
    if(a[i].empty()==1){ //如果为空
        if(a[i].top()==1){ //如果上一步没有最小元素
            if(a[i+1].next()==1){ //如果当前是最后一个，它的下一个为空
                next=1; //直接置为1，表示下一个为空
                break; //当前字符的遍历结束，跳出下一个for
            }
        }
    }
}
if(a[0].size()==len1[a].size())return break; //如果原串
int other1=0;
switch(next){ //在上面的两个if语句中满足条件时
    case 1: //如果满足以上两个if语句的条件
        other1=1; //将下一个字符的遍历交给其他
        other2=1;
        break;
    case 2: //如果不满足
        other1=1;
        other2=1;
        break;
    case 3:
        other1=1;
        other2=2;
        break;
}
if(a[other1].empty()||a[other2].empty()){ //如果1
    else if(a[other2].empty())return(a[other1].empty());
    else

```

说说你的看法，或者提个问题

Java构建的AI... 2026.03.23

高羽的活跃 2025.03.09

这篇博客写得非常棒！讲解清晰透彻，逻辑严谨，干货满满。从基础概念到实战应用层层递进，让我收获很大。代码示例很详细，排版也很舒服。感谢作者的用心分享，期待更多优质内容！

马伊奇也要立... 2026.03.09

博主的文章写的很专业，能把硬核技术写得像故事一样层层递进，引人入胜。逻辑清晰，排版舒服，代码注释详细，这种阅读体验在技术圈里真是清流

我的算法源码、视频、博文、电子书可复用性、可理解性、可测试性很强。优于工作、劣于竞赛。

小叶-duck 2026.03.09
好文章学到了

5 条评论被折叠 查看

5 条评论被折叠 查看

- 讲者接着前面的例子说：可以把一个程序翻译成另一个程序；二者等价；做一个 **simplification**、做优化，就能从前者变成后者。
- 如果觉得代码不够优雅，对 **TA 和 Hanoi**（原音：ta 和 HONNI）来说可能 **overkill**。
- 讲者作为架构师，把这种翻译/简化/优化看作理解 **C 语言形式语义** 的好机会。
- 逻辑链：任意程序可翻译为等价程序 → 可做 simplification/优化 → 不优雅实现可改写成更简版本 → 对具体小问题可能 overkill → 但可用于理解形式语义。

- 看到不同人的实现后，讲者认为自己还能守住品位。
- 讲者判断：GPT（原音：GBT）可能其实“懂”，但不会往这个方向去想。

- 因此讲者有较强信心：**强化学习**可以把 GPT 往这个方向引导；一旦学会，GPT 也能在适当的时候做出这样的实现。

百度搜索非递归汉诺塔：第一条仍是递归，CSDN 版本比 AI slop 还严重

- 讲者在百度搜“非递归汉诺塔”：百度第一个条目还是“递归汉诺塔”；往下才找到 CSDN 的第一条“非递归汉诺塔”。
- 他看到的代码问题严重，甚至比 AI slop 还严重。
- 具体细节：代码里有一个 `move`，但 std/STL 已经有 `move`（原音：STD 是有 move 的）。
- 右边内容疑似 AI 评论；讲者不确定是不是 AI，但这会营造一种 CSDN 博客质量很高的感觉。
- 讲者观点：如果上学时被这种 slop 污染，还不如跟 AI 学；跟 DeepSeek 学都很好；不要 GPT，它已经“不当人”了（原音：不要 GP6 已经不当不当人了）。

同一个汉诺塔的多重表达：真正要控制的是复杂性、架构与抽象

- 讲者把讨论收束到同一个汉诺塔的四重表达。
- 核心不是实现功能，而是**控制复杂性**；这是架构问题，也关乎**抽象**。
- 方法：站在不同层面上考虑问题，即“**跳出问题看问题**”。
- 逻辑：同一问题 → 多重表达 → 差异在抽象层级/架构水平，而不只是能不能跑。

GPT 版“跳出”：一切都是数学规律，讲者说“我服”

- 讲者说，GPT 也“跳出问题看问题”了：**一切都是数学规律**；只要把程序世界里的数学规律理解清楚，就没有解决不了的问题。
- 讲者对此说“我服”。

DeepSeek 版：从语料学到“递归是栈”，做了正确的事；讲者再向外走一步

- DeepSeek 做了一个正确的事情：人类反复说“递归是栈”，它在语料里学到了这一点。
- 讲者再往外走一步：你们对编程语言/世界的理解，被投射成递归与非递归的区别，再投射成一份代码。
- 逻辑链：递归是栈（语料中的共识）→ DeepSeek 做出正确实现 → 讲者从更外层看：世界理解 → 递归/非递归区分 → 具体代码。
- 还有一个版本，讲者说“不知道是什么”，并判断它属于训练还没有收敛。

小地方能看出架构水平高低

- 讲者强调：在任何一个地方，都能看出架构水平的高低；这是实战的架构（原音：实行的架构）。
- （整理者串联）因此，汉诺塔这种小例子在讲者这里被当作架构能力的探针。

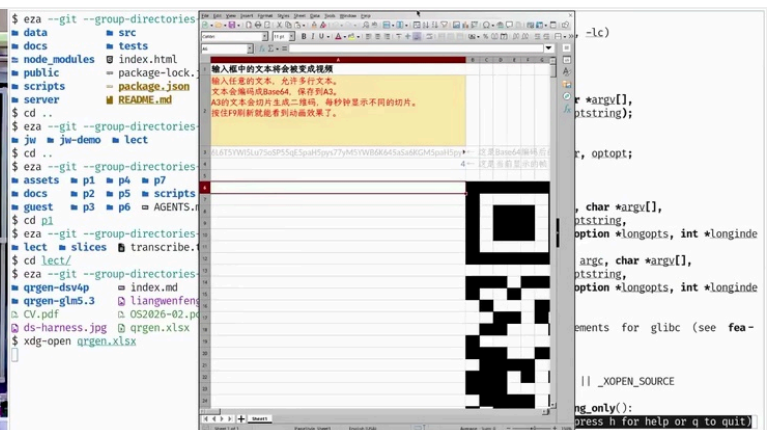
放大到 2000 行架构：跳出代码，看到客户、社会、宇宙的规律

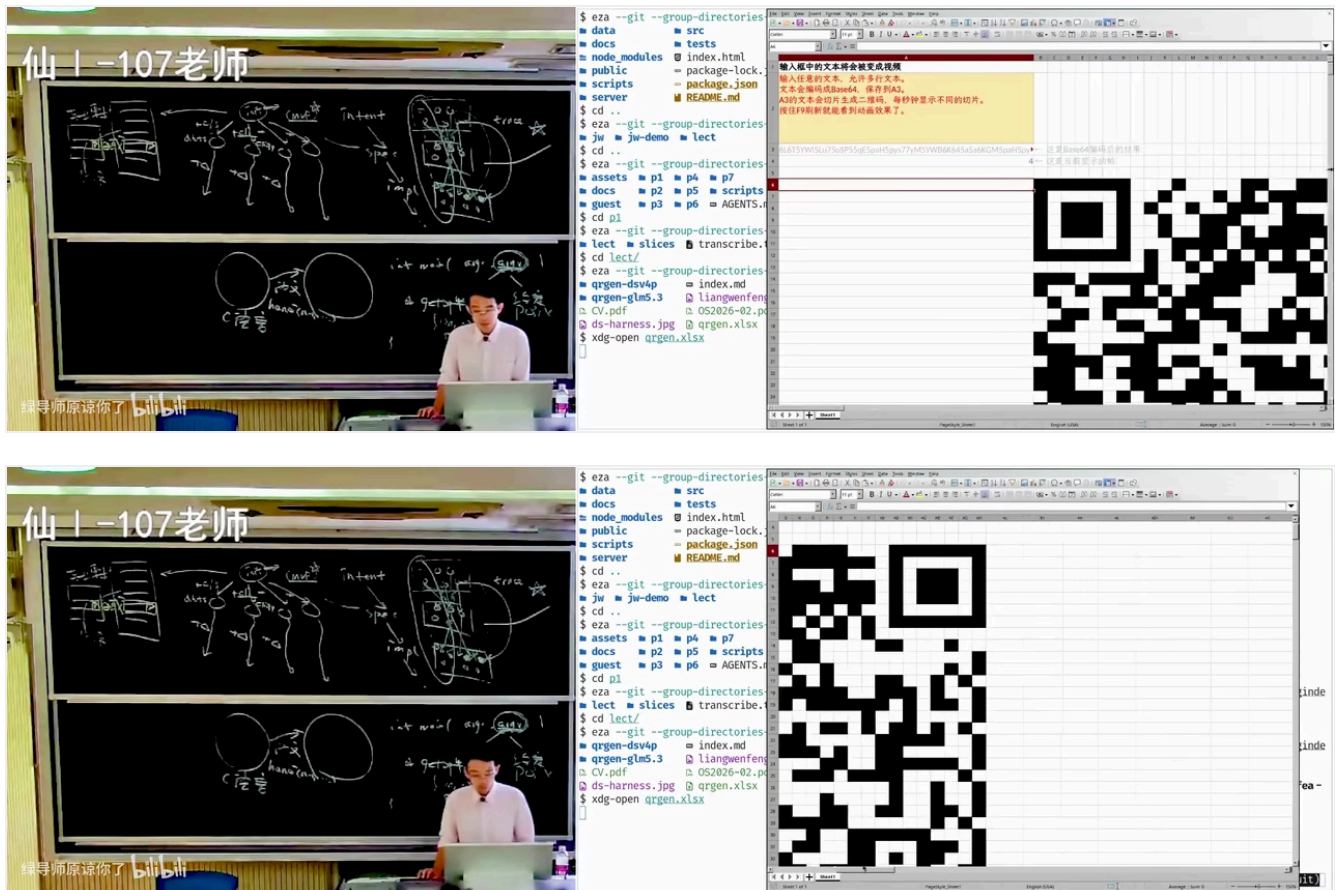
- 如果要做更大的架构，比如 2000 行的架构，同样要展示“跳出问题看问题”：
 - 代码之外有客户的规律；
 - 客户之外有社会的规律；
 - 社会之外有宇宙的规律。
- （整理者串联）抽象层级越高，需要考虑的规律就越超出代码本身。

架构能力从哪来：学过的好东西、优雅、压缩与 first principle

- 讲者预判听众会问：架构的能力从哪来？
- 答案：从你学过的每一个东西上来。如果它是好的、优雅的，就会促使你思考这个问题。
- 讲者提出：压缩就是智能。你把它压缩了，就能用 first principle 去推导“应该怎么样”。
- 因此，这些能力由学校里的学习和社会上的积累共同赋予。
- 最后讲者开始举例：“比如说你必须要和除了教科书……”（本段在此中断）。

第 18 部分





工程能力来自学校训练与社会经验：会共情真实用户

- 讲者主张：这些能力是学校学习 + 社会积累共同赋予的。
- 理由：你必须和教科书、做题之外的人打交道，才能知道真实用户怎么看待系统。
- 例子：一个快要退休、优雅的教务老师。
 - 学生/工程师可能觉得自己的方案最好：数据库性能最高等等。
 - 但她只想要字大一点、点鼠标时不要手滑到旁边按钮上。
- 逻辑链：只从技术指标出发 → 容易把自己的偏好当成用户需求；与真实用户打交道 → 看见不同优先级 → 讲者认为这就是共情别人的能力。

Excel QR code generator：需求明确 ≠ 没有架构问题

- 讲者课堂例子：用 Excel 生成二维码，可以打开看；能把任意长字符串编码成可直接扫描的二维码。
- 用今天讲过的软件工程概念理解：
 1. 需求：在 Excel 里有输入单元格、输出单元格，生成 0/1 序列（二维码）单元格。

- 2. 这个需求和教务系统不同，非常明确、非常 **deterministic**。
- 3. 但讲者强调：这里面仍有**架构**考虑，Excel 公式怎么组织、怎么抽象。
- 逻辑链：需求明确 → 若直接写成一个公式 → 公式极长 → 因为 Excel 计算/公式能力有限 → 必须抽象、分解。
- 类比：像用 **Brainfuck (ASR: brain)** 实现一个 nontrivial 程序，甚至写 **Doom**；代码会变成很难处理的样子。讲者用“偷盗了一个 Lisp 程序”的笑话/梗形容公式长到最后一页还是它。

Agent 会估能力、会分解，但今天的模型做架构仍稍落后

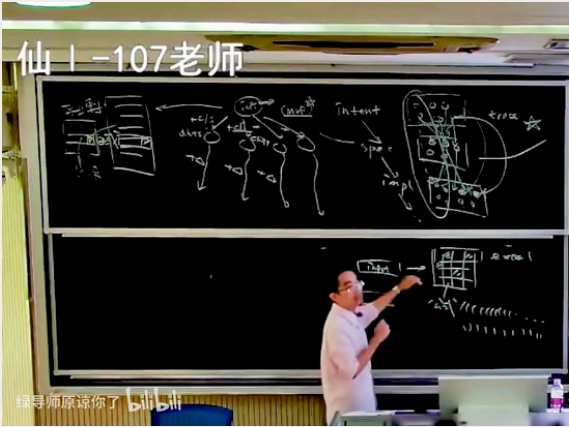
- 讲者觉得 **Agent** 好玩：它对自己的能力有估计。
- 任务足够困难时，它不会直接一把做；超出能力范围时，会试图**分解**问题。
- 能力范围内的例子：两个 sheet，第二个 sheet 是学生实验成绩，第一个 sheet 是整理好的期末考试成绩；要求写公式，按学号把实验成绩拉过来。Agent 会一把梭。
- 讲者判断：至少今天的模型在“做架构”这件事上稍稍落后。

看似明确的需求还有坑：Excel 版本和函数差异

- 讲者说他之前看到的两个 AI 版本没考虑到：需求看起来明确，但其实有很多坑。
- 例如 Excel 有版本差异；从 2019 到 2023 不断增加新功能。
- 刚才“查找学号对应成绩”的需求就是这种坑的例子。
- （整理者串联）同一个查找公式可能因 Excel 版本、可用函数不同而不通用；架构时要把运行环境纳入约束。

小结：真实用户需求需要共情；明确需求仍要架构与抽象；Excel 版本/函数差异是隐藏坑。

第 19 部分



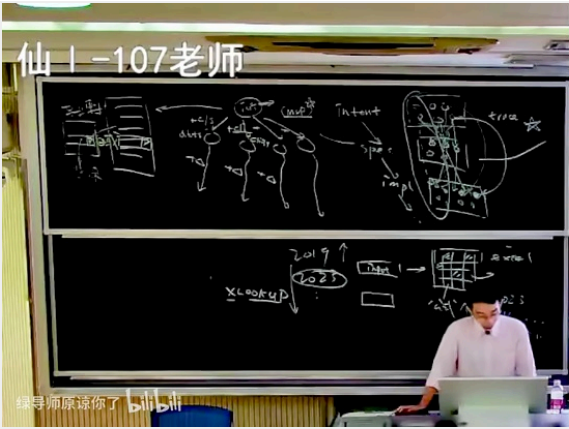
QRCode Generator 的复杂性

公式极长且脆弱

- 直接写公式面临立即崩溃
- 无数次看到 Agent 和括号不配对搏斗

需求变更也极度痛苦

- 兼容更老版本的 Excel, 或允许使用新版本 Excel 的特性
- 把二维码从一个位置移到另一个位置
- 改变一种计算方式
- 增加一个字符串单元格
-



```
$ eza --git --group-directories-first --icons=auto
data      src      (t) tsconfig.app.json
docs      tests    (t) tsconfig.json
node_modules index.html (t) tsconfig.node.json
public    package-lock.json (t) vite.config.ts
scripts   package.json
server    README.md
$ cd ..
$ eza --git --group-directories-first --icons=auto
ju        ju-demo    lect
$ cd ..
$ eza --git --group-directories-first --icons=auto
assets    p1      p4      p7      CLAUDE.md  TODO.md
docs      p2      p5      scripts  Makefile
guest     p3      p6      AGENTS.md README.md
$ cd p1
$ eza --git --group-directories-first --icons=auto
lect      slices    transcribe.txt
$ cd lect/
$ eza --git --group-directories-first --icons=auto
qrngen-dsv4p    index.md    spec-gap.png
qrngen-glms.3   liangwenfeng.gif tar-pits.jpg
CV.pdf          OS2026-02.pdf
ds-harness.jpg  qrngen.xlsx
$ xdg-open qrngen.xlsx
$
```

LIBRARY
Standard C library (libc, -lc)

SYNOPSIS
#include <unistd.h>

int getopt(int argc, char *argv[],
 const char *optstring);

extern char *optarg;
extern int optind, opterr, optopt;

#include <getopt.h>

int getopt_long(int argc, char *argv[],
 const char *optstring,
 const struct option *longopts, int *longinde
x);

int getopt_long_only(int argc, char *argv[],
 const char *optstring,
 const struct option *longopts, int *longinde
x);

Feature Test Macro Requirements for glibc (see fea-
ture_test_macros(7)):

getopt():
_POSIX_C_SOURCE >= 2 || _XOPEN_SOURCE

getopt_long(), getopt_long_only():
Manual page getopt(3) line 7 (press h for help or q to quit)

Excel 版本演化让“按学号查成绩”变成兼容性选择题

- Excel 是有版本差异的：讲者提到 Excel 从 2019 到 2023 这类版本不断加入新功能，函数和语法会变。
- 讲者举的需求是：查找学号对应的成绩。
- 旧版本里，这个需求常用 VLOOKUP（听写为“video cup”）实现。
- 讲者问有没有同学知道 VLOOKUP，没人知道。讲者由此说：说明你们都不是 Excel 的 power user；但“太好了”，你们是 agent 时代的第一类人——要干什么只要跟 agent 说就行，永远不需要再学一个“像吃屎一样的语言”。
- 微软后来（讲者不确定是 2019 还是 2024 的某一年）终于推出 XLOOKUP，可以更方便地查找。
- 于是同一个需求不再只有一种实现方式，而出现版本分叉。

逻辑链：

1. Excel 有版本，且持续演化 →
2. 同一需求可用旧函数 **VLOOKUP**，也可用新函数 **XLOOKUP** →
3. 需求变得模糊：到底要兼容旧版本标准，还是用最新语法做自用即抛工具？ →
4. 如果需求变化，维护会极度痛苦。

讲者进一步说：如果我要兼容旧版本，使我在各种人机器上都能打开、发给教务老师也能打开；还是我只是自己用一用、即抛，那就可以用最新语法实现最简单公式？这个选择本身就让需求不再清晰。

需求一变，Excel 实现可能整片崩塌

- 讲者假设：AI 给你写了一个用 Excel 2023 公式的实现。
- 你突然说：不行，2023 不行，要改成 2019，再要改成 2003。
- 结果是：**全完蛋**。设计和实现从头回到原点，重新再做一遍。
- 对于 2000 行的项目，这还可能 OK，可以重做。
- 但对于 2000 万的项目，且已经嵌入一个巨大无比的需求、嵌入教务系统、和其他部分产生关联时，就不是重做那么简单了——讲者说，作为项目经理，这时就要加人、加钱。
- 讲者认为：这显然是一个架构没有设计好的问题。

硬编码连“整体右移一格”都抵抗不了

- 讲者继续举例：等需求最后出来后，我可能对二维码/输出放的位置不满意，要把它整体往右移一格。
- 如果你所有单元格都是**硬编码**的，那么这种改动会直接立即崩溃。
- 所以讲者判断：这种设计对需求变更的抵抗非常糟糕。

跳出 Excel 看：这是编程/实现问题，二维码本身有规范

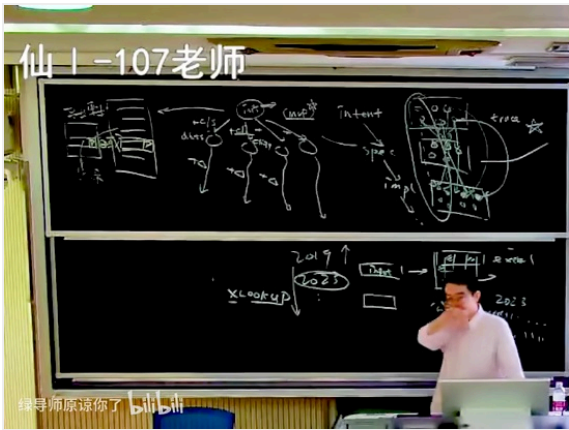
- 讲者说：如果我们跳出问题看问题，其实这是一个**编程问题**（听写为“编译问题”，按上下文更像编程/实现问题）。
- 他提出可以给出一个从字符串到二维码的伪代码；这个问题本身是清楚的。

- 讲者说：我给的是一个非常明确的问题，但 AI 还要去读文件；而**二维码是完全有规范的**。
- AI 给的实现很短，就是一串代码。里面包含这些要素：
 - 版本
 - 纠错等级
 - 掩码
 - 大小
 - 数据编码
 - 0100
 - 8 位字节
 - 纠错码
 - 134 个码字
 - 最后加上 ECC
 - 再直接放进二维码的空间/矩阵里
- 讲者追问：为什么这么简单的东西，在 Excel 里面实现起来这么困难？这个 **gap** 到底在哪里？
- 他的判断是：如果你跳到问题外面，就很明显能看到，**二维码本身很容易用算法来描述**。

（整理者串联） 因此本段的张力是：二维码算法本身清楚、有规范、能短代码描述；困难更像是来自 Excel 的版本兼容、单元格硬编码、需求变更和系统嵌入方式。

小结： 本段核心矛盾是：二维码本身容易用算法描述，但在 Excel 里实现却因版本、兼容与需求变更变得困难。

第 20 部分



仙 I-107老师

绿导师原谅你了 bilibili

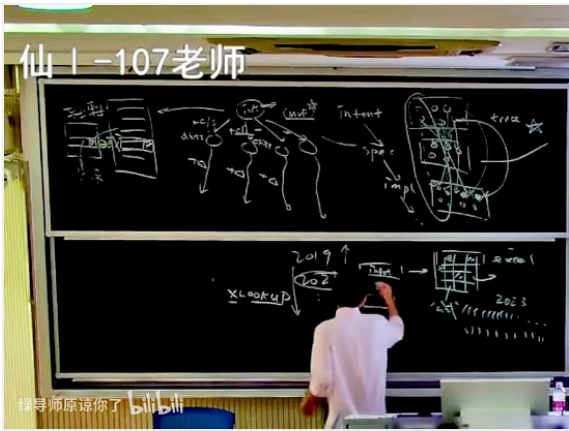
```
# 1. 数据编码
bits = 0x00
bits += len(s) 的 8 位二进制 # byte 模式指示符
# 字符数(byte 模式下按字节计)
bits += s 的每个字节(8 bit)
bits += 0000 # 终止符
bits = 补 0 到 8 的整数倍 # 对齐到码字
bits += 交替填充 0xEC, 0x11 # 直到填满 108 字节
(v5-L)
data = bytes(bits)

# 2. Reed-Solomon 纠错 (GF(256), 本原多项式 0x11D)
g = rs_generator_poly(26) # 26 = v5-L 纠错码字数
ecc = data * x^26 mod g # 用 LFSR 做除法
cw = data + ecc # 共 134 个码字

# 3. 摆放布局
m = 37x37 全空
draw_finder(m) # 三个角的 7x7 回字
draw_tiling(m) # 第 6 行/列的黑白交替
reserve_format(m, 2 处) # 为格式信息留位
zigzag_fill(m, cw) # 从右起, 2 列一组上下蛇形, 跳过功能图形

# 4. 掩码: 让黑白分布均匀, 避免大面积同色
best = argmin over k in 0..7 of penalty(mask(m, k)) #
也可直接写死 MASK
apply_mask(m, best)

# 5. 格式信息
fmt = (EC 的 2 bit, MASK 的 3 bit) # 5 bit
```



仙 I-107老师

绿导师原谅你了 bilibili

我们其实要实现的是“编译器”

跳出问题看问题

- 二维码算法容易用伪代码 / 编程语言描述
- 但它们不能原样变成 Excel 公式
- 先用中间语言 (IR) 表达计算规则, 再决定怎样落到 Excel 上。

用高级语言构造中间语言

- Python 的函数和循环负责构造计算图: 按选定容量展开有限的计算步骤。
- 字符串输入保留为符号: 用户改动单元格后, 由 Excel 重新计算二维码。
- 输入决定的分支写成 Select(cond, a, b) 节点, 留到运行时判断。



仙 I-107老师

绿导师原谅你了 bilibili

我们其实要实现的是“编译器”

跳出问题看问题

- 二维码算法容易用伪代码 / 编程语言描述
- 但它们不能原样变成 Excel 公式
- 先用中间语言 (IR) 表达计算规则, 再决定怎样落到 Excel 上。

用高级语言构造中间语言

- Python 的函数和循环负责构造计算图: 按选定容量展开有限的计算步骤。
- 字符串输入保留为符号: 用户改动单元格后, 由 Excel 重新计算二维码。
- 输入决定的分支写成 Select(cond, a, b) 节点, 留到运行时判断。

Excel 里二维码难, gap 在 Python 算法与 Excel 表格表示之间

- 讲者先问: 在 Excel 里实现二维码为什么这么困难, gap 到底在哪里?
- 跳出 Excel 看问题, 二维码本身很容易用算法描述: 用 QR.py (原识别为“Q2.py”) 做, 最后打出 0/1 矩阵, 再容易不过。
- 讲者认为这可以作为明年保研机试题: 让考生实现并输出 0/1 矩阵; 如果要求脱网 (原识别“拖拖网”, 不确定), 是不错的测试。

没有 Python→Excel 的通用编译器，Excel 也不是为图灵完备计算设计

- 要求是在 Excel 里完成，但不存在翻译器/编译器能把 Python 编译成 Excel；讲者点名 `PYXLC` 不存在。
- Excel 本质是为财务、报表等设计的；管理学生成绩只是一个小数据库。它本身不是为 **图灵完备 (Turing complete)** 计算而设计的（原转写“turn complete”）。
- 所以不能直接指望 Python 跨到 Excel。

直接让 LLM 把伪代码/自然语言编译到 Excel，会需求漂移

- 如果把逻辑变成伪代码或自然语言，再用 LLM 去编译到 Excel，已看到 LLM 的问题：它直接上手就做。
- 一旦需求漂移，就会遇到很大问题。
- 讲者由此推出：需要好的架构。

架构：在 Python 和 Excel 中间造一个中间语言

- 讲者设想：在 Python 与 Excel 之间构造另一个语言。
- 这个语言要同时满足：
 - 很容易翻译成 Excel；
 - 继承一定 Python 功能；
 - 能描述这些运算中需要的东西。
- 这正是刚才伪代码需要的形式：一个正式的**中间编程语言**。讲者说自己做了这样的设计，该语言可以翻译成 Python。

表达式树让中间语言两边都能落地

- 例子来自表达式求值程序：做一个**表达式树**。
- 节点可表示如 `xor(A,B)`、`x[1]`（从字符串中取某个字符）。这就是一个小型编程语言。
- 翻译成 Python：
 1. 把表达式树 dump 出来；
 2. 递归处理左子树、右子树；
 3. 按当前节点拼起来。
 - 所以很容易翻译成 Python。
- 翻译成 Excel：
 1. 每个节点都有一个翻译成 Excel 的规则；
 2. 中间结果可放到任意格子；

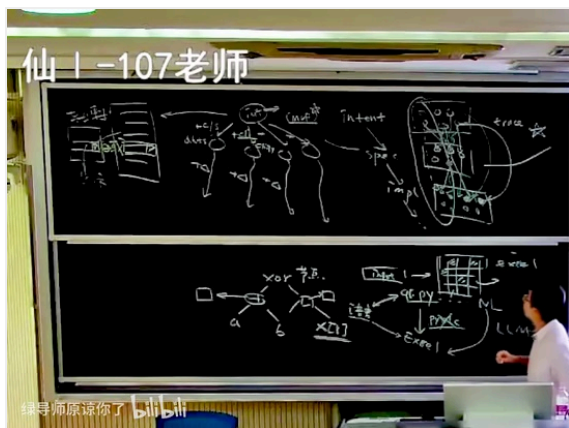
3. 再把两个格子的结果拼起来。
- 所以也能翻译成 Excel。

工作流：LLM 生成中间语言 → Python 验证 → Excel 落地

1. 让大语言模型生成这个语言的二维码计算逻辑。
 2. 把该语言翻译成 Python，验证公式/逻辑写得对不对。
 3. 等公式写对了，再去写 Excel。
- 讲者认为这就是前面最小例子中 `getopt` 的设计：写命令行工具时，第一步先 `getopt` 得到 **parsing state**（原转写“passing state”），再从这个状态出发做下一步。

小结：讲者的方案是用中间语言把“逻辑验证”和“Excel 落地”解耦。

第 21 部分



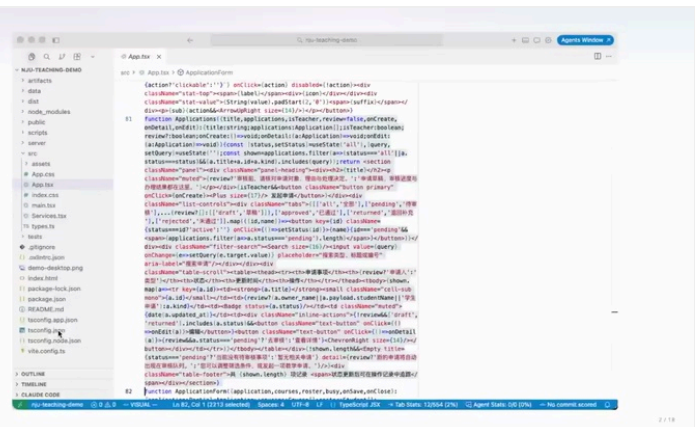
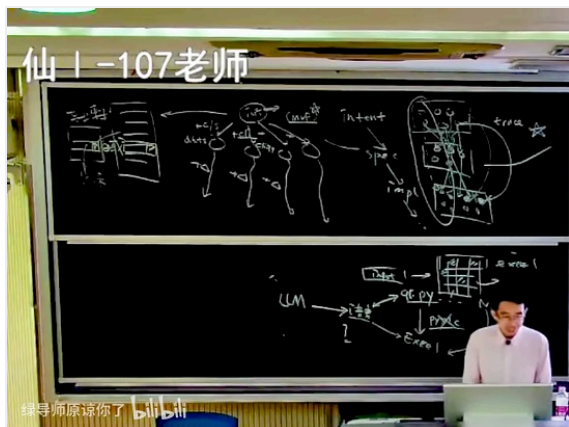
我们其实要实现的是“编译器”

跳出问题看问题

- 二维码算法容易用伪代码 / 编程语言描述
- 但它们不能原样变成 Excel 公式
- 先用中间语言 (IR) 表达计算规则，再决定怎样落到 Excel 上。

用高级语言构造中间语言

- Python 的函数和循环负责构造计算图：按选定容量展开有限的计算步骤。
- 字符串输入保留为符号：用户改动单元格后，由 Excel 重新计算二维码。
- 输入决定的分支写成 `Select(cond, a, b)` 节点，留到运行时判断。



先得到 passing state，再迭代：程序是这样长出来的

(整理者串联) 讲者这条线的推进是：

1. 写程序 / 写命令行工具 → 先 **get output** (原文 ASR: get opp t) 得到 **passing state** → 从 passing state 出发做下一步设计。
2. 作为 **unsupervised learner** / **神经网络** → 有意识看“好东西” → 看得多了自然泛化。
3. 某天看 blog 或听课 → 突然想通：大家做的事情差不多，都是一份中介语言，两种执行方式。
4. “构造一次”：架构 = 从 **LIM** (ASR 不确定，按上下文疑为 LLM) 翻译成 **中间语言** → 一种用于 **验证**，一种用于 **交付**。
5. 在中间语言上发展 **优化**：改写规则、循环内不变量外提、**profiling-guided optimization**、让大语言模型优化。
6. 好架构 → 承载需求变化 (Excel 改版只是换算子实现)。
7. 50 万行架构 → **跳出问题看问题** → 先看 AI 实现的架构案例。

讲者说，写命令行工具第一件事就是先 **get output**，得到一个 **passing state**，再从那个 passing state 出发去做下一步一样的设计。

讲者用 **unsupervised learner** 和神经网络作类比：神经网络就是看好东西；看得多了，自然就能泛化。但看得多时，要 **有意识去看好这个东西**。讲者认为，某一天回头看，看某个 blog，或者听他上课时，会突然想通一件事：原来这个世界上大家做的事情都差不多，都是在做这样的东西。

想通后的共同套路：一份中间语言，两种执行方式

- 讲者说，他想通的就是：一份中介语言，两种执行方式。
- 因此可以“构造一次”。他的架构是：从 **LIM** 翻译成中间语言；这个中间语言 **用来验证**，也 **用来交付**。
- 讲者补充：这些 AI 给他生成的 **slop** 也不重要了。(整理者串联) 重点落在架构本身，而不是 AI 生成物。

中间语言让优化有落点：改写、外提、profiling

- 在这个架构上还可以发展 **优化**。讲者把优化说成：既然有一个语言，如果觉得性能不好，或者哪里生成了太长的代码，就可以有一些 **改写规则**，把一个程序从一个形态改写成另一个形态。
- 例子：如果一个变量在循环里面每次都计算，但值都一样，就可以把这个变量的计算 **提到循环外面**。讲者说，编译器可以自动做这件事；你自己也可以手动把这个计算移出去，做 **手工优化**。
- 如果有 **中间语言**，就可以让 **大语言模型** 去优化它。
- 讲者还联系到计算机系统里学到的 **profiling-guided optimization**：先要知道性能瓶颈在哪里，再用性能的 **profile** 去指导代码应该怎么写更好。讲者认为“你的概念又用上”了。

好架构承载需求变化：Excel 改版只是换算子

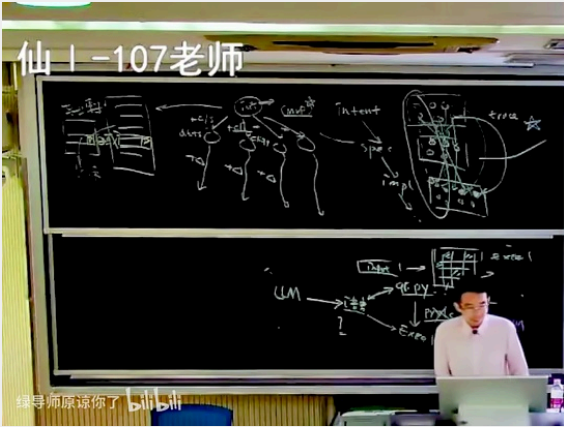
- 讲者认为，一个好的架构可以承载 **需求的变化**。
- 例子：未来 Excel 版本改了，没问题，无非就是把一些 **算子** 用另外一种方式去实现。
- 讲者说，讲算子的时候会发现，今天的 **AI inference**（原文 ASR: AI infer人）也在干这个事。人类有很多领域知识，所以在他们的领域里面有非常多好的设计，这些都非常值得大家学习。

50 万行架构的核心：跳出问题看问题

- 讲者说，终于到最后了：十行的讲完了，2000 行的讲完了，**50 万行的架构应该怎么设计**。
 - 讲者认为，架构就是一件事：**跳出问题看问题**。
 - 当人们谈 **架构师** 时，通常会想到：框图、数据库、微服务、流式数据处理、应用框架；然后选几个应用框架搭起来，就解决教务系统的问题。
 - 但在开始看架构之前，讲者要先看：**AI 实现了一个什么样的架构**。他说这很好玩——这是他在 AI 生成中间（语言）时看到的结果，所以当场发了一个小红书。
-

第 22 部分

仙 | -107老师



绿导师原谅你了 bilibili

不是“尽量通用”，而是先**把设计自由度钉死**（版本/纠错等级/掩码），把冗余多实现收敛成一个：

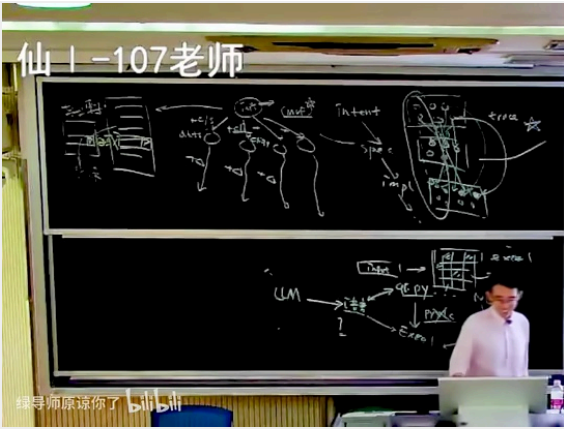
```
QRCode(s):
    # 0. 固定设计决策 — 消灭歧义(这是全部难点所在)
    V, EC, MASK = 5, L, 0 # 版本、纠错等级、掩码
    SIZE = 17 + 4*V # 37 x 37 模块

    # 1. 数据编码
    bits = 0100 # byte 模式指示符
    bits += len(s) # 8 位二进制 # 字符数(byte 模式下按字节计)
    bits += s 的每个字节(8 bit) # 字节数
    bits += 0000 # 终止符
    bits = 补 0 到 8 的整数倍 # 对齐码字
    bits += 交替填充 0xEC, 0x11 # 直到凑满 108 字节
    (v5-L)
    data = bytes(bits)

    # 2. Reed-Solomon 纠错 (GF(256), 本原多项式 0x11D)
    g = rs_generator_poly(26) # 26 = v5-L 纠错码字数
    ecc = data * x^26 mod g # 用 LFSR 做除法
    cw = data + ecc # 共 134 个码字

    # 3. 摆放布局
    m = 37x37 全空
    draw_finder(m) # 三个角的 7x7 回字
    draw_timing(m) # 第 6 行/列的黑白交替
    reserve_format(m, 2 处) # 为格式信息留位
    zigzag_fill(m, cw) # 从右下起, 2 列一组上下
```

仙 | -107老师



绿导师原谅你了 bilibili

新增功能，越来越像拆盲盒

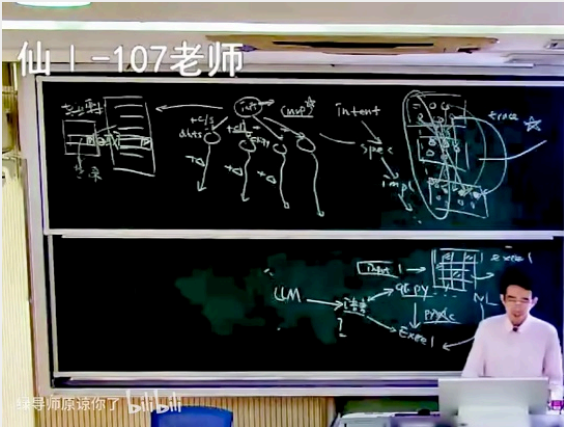
GPT-6 Astra xhigh 还认为

- 能维护，但现在是把业务逻辑挤在几个大文件里的演示原型
- 再不拆分和统一规则，新增功能就会越来越像拆盲盒 😊😊😊



我...不当人类了! JoJo!

仙 | -107老师



绿导师原谅你了 bilibili

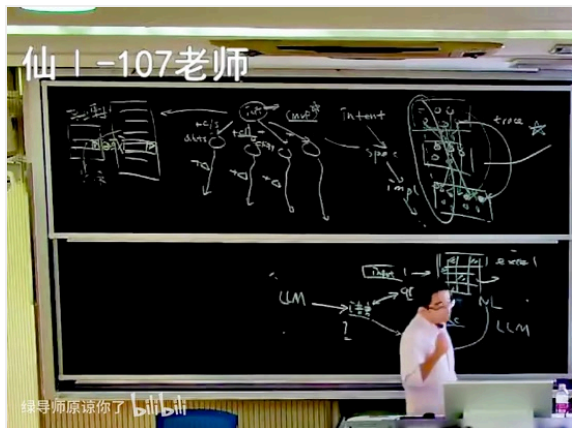
新增功能，越来越像拆盲盒

GPT-6 Astra xhigh 还认为

- 能维护，但现在是把业务逻辑挤在几个大文件里的演示原型
- 再不拆分和统一规则，新增功能就会越来越像拆盲盒 😊



我...不当人类了



CRUD 架构

用“当前状态”描述整个世界

- 任何时候，系统都存在一个“当前状态”的快照
 - 学生、教师、选课、.....
 - 《数据库》课要求你消除表中的冗余 (normal form), 防止不一致
- 为每个需求点，写一个状态查询/修改的业务逻辑
 - Everything is a state machine 😊 的诱惑

“架构是一种看待问题的方式。”

4/18

AI 生成结果：逻辑上“毫无破绽”，代码格式化后仍是大文件单体

- 讲者展示模型生成中间结果，觉得“很好玩”，并当场发小红书调侃：模型（原文作 GT6）说“我不当人，我不做人了”。这个梗接的是前面的汉诺塔：讲者认为模型在数学上能想清楚、答案对，就“毫无破绽”。
- 模型写代码时，一个 `function applications()` 之类的函数“啪啪啪”写很长；后来 GPT 又用代码格式化工具自动格式化。
- 格式化后，现在看这个 demo，仍然是 **monolithic (单体)**：例如 83KB 的 `TSX`、38KB 的 `services`，业务逻辑挤在几个大文件里。讲者仍觉得它很有水准。

评审给出的架构判断：经典三层单体、前后端分离，并带有生产级横切设计

- 讲者让 D老师（原文如此）评价 GPT 生成的东西；又启动独立 session，问正在跑的 GPT：“这个软件能维护吗？”讲者说自己是“害怕地问”，GPT 回答“能”，理由是业务逻辑只是挤在几个大文件里，维护完全没问题。
- 模型给出的架构判断是：经典三层单体、前后端分离；还有几个生产级横切设计：乐观锁、幂等 (idempotent)、事务、审计、统一错误处理。讲者说这些以后可能会专门讲。
- 讲者评价：模型确实做了工程上“你们做不来、至少摸爬滚打几年才理解”的设计，有软件工程直觉；它还有数据驱动、横向扩展、扩展性好的特点，当然也有差的一面。讲者认为，很快 AI 生成生产软件的方式就要超过人类。

跳出问题看：本质是 CRUD，所以讲者不认为它是“非常好的架构”

- 讲者明确跳出来看后说：这还不是一个非常好的架构。
- 理由：本质上这是一个 **CRUD 架构**，即增删改查架构。
- 所以这里显式链条是：AI 生成系统工程细节成熟 → 但底层本质是 CRUD → 讲者不把它评为非常好的架构。

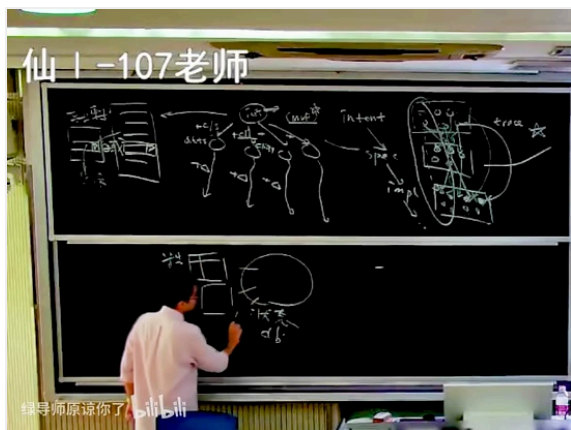
CRUD 架构的第一原理：软件是物理世界过程在信息世界中的投影

讲者给出的基本原理链：

1. 软件是物理世界过程在信息世界里面的投影。
 2. 物理世界是有状态的；**Everything is a state machine**。讲者说自己也是一个 state machine。
 3. 因此，软件系统很容易建模成一个 **state transition system**（状态迁移系统）：它有一个状态。
 4. 这个状态一般用数据库保存；**persistent**（持久化）状态用数据库。
 5. 数据库里可以用二维表表示：比如一张学生表，含学号、姓名、院系等；还可以有学院表、选课表等，各种东西都能用二维表表示。这是数据库课上学过的内容。
 6. 数据库要求消除表里面所有的冗余。
- 链条可写作：
物理世界有状态 → 一切皆状态机 → 软件可建模为状态迁移系统 → persistent 状态落数据库 → 二维表表示学生/学院/选课等 → 数据库要求消除冗余 → **（整理者串联）** 这类系统自然被做成以增删改查为中心的 CRUD → 讲者据此认为它还不是非常好的架构。

小结：**（整理者串联）** 讲者的评价分两层——AI 的工程细节已经很像生产系统，但底层范式仍被他归为“状态机—数据库表—CRUD”。

第 23 部分



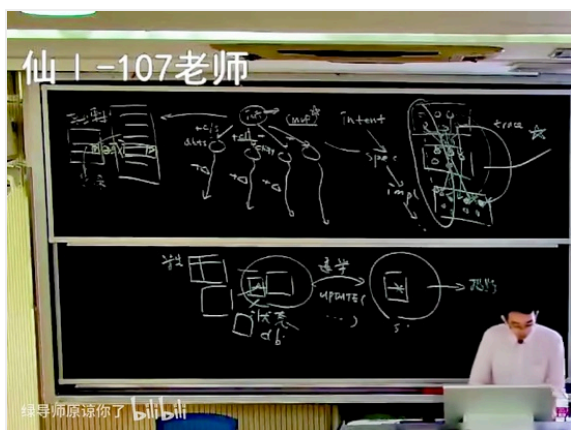
CRUD 架构

用“当前状态”描述整个世界

- 任何时候，系统都存在一个“当前状态”的快照
 - 学生、教师、选课、.....
 - 《数据库》课要求你消除表中的冗余 (normal form), 防止不一致
- 为每个需求点，写一个状态查询/修改的业务逻辑
 - Everything is a state machine 😊 的诱惑

“架构是一种看待问题的方式。”

4 / 18



来越像拆盲盒

CRUD 架构

gh 还认为

把业务逻辑挤在几个大文件里的演示原型

一规则，新增功能就会越来越像拆盲盒 😊😊😊

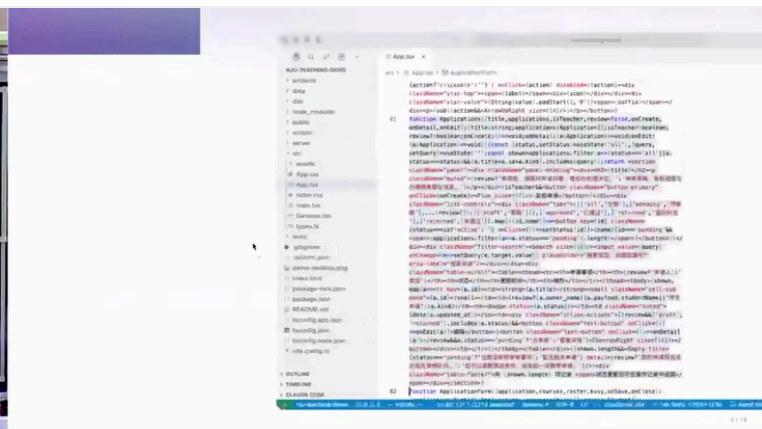
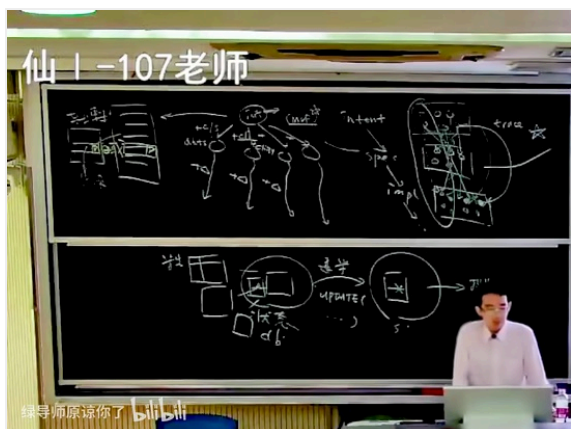


用“当前状态”描

- 任何时候，系统都
 - 学生、教师、选
 - 《数据库》课
- 为每个需求点，写
 - Everything is

“架构是一种看待问题的方式。”

5 / 18



二维表的目标：用范式消除冗余依赖，而不是把信息到处复制

- 讲者说，各种东西都可以用二维表表示；数据库课会要求消除表里的冗余，把数据库 factor 成 normal form（范式）。
- 具体做法：学生有唯一学号，如 1234；学生还有姓名。另一张表要引用学生时，尽量不要用姓名引用，因为可能重名；就算不重名，也会带来麻烦。更好的是不放姓名，放学

生 ID，如 1234。

- 这个 ID 相当于一个**链接**：指向学生信息；学生还可以再指向其他表中的某些信息。
- 讲者把这叫**信息最小化原则**：其实是铲除冗余的 **dependency（依赖）**。逻辑链：
 1. 信息尽量只在权威位置存一份；
 2. 其他表只存 ID 引用；
 3. 一个地方改了，不需要改另一个地方。
- 例子：学生改名——讲者强调这在中国合法，不要笑。若已按分解方式处理，只改学生表里的名字，并可记一个**曾用名**；若两个地方都存了姓名，则两处都要修改，代价很大。

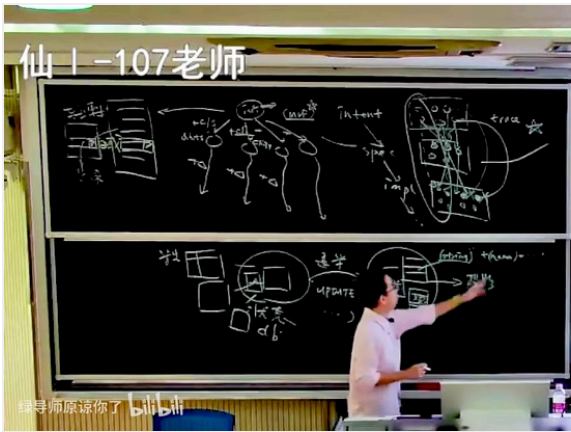
世界被建模成“状态”：当前状态一致，更新就是状态迁移

- 讲者说，一般把世界建模成**状态**。任何时刻的状态代表：当前时间点南京大学有哪些学生、哪些学生选了哪些课、有哪些老师等。
- 信息系统常采用**前后端分离**的经典架构。比如讲者开玩笑说：“今天我选择退学——我没有买到 token，然后退学。”退学就是对数据库的一次 **update**，从一个状态迁移到下一个状态。
- 例子：当前状态里有 A；下一状态里 A 没有了，A 这条记录不存在了。
- 讲者认为：这始终是一个 **consistent（一致）** 的视图；在交互系统里打开网页，看到的是这个东西的一个**投影**。这是非常经典的架构，没有任何错误。

教务系统不适合纯 CRUD：当前状态祥和，但历史丢了，字符串写入还会造成内部不一致

- 但讲者接着问：为什么**教务系统**不适合这样的 **CRUD** 架构？
- 原因一：看当前状态时，看到的是一片**祥和的景象**；可是当前状态的所有**历史**都被丢掉了。教务场景不能只看当下。**(整理者串联)**：一旦需要追溯“当时是谁、当时授予了什么”，纯当前状态视图就不够。
- 原因二：如果某个地方把学生姓名存了两次——讲者说这非常正常——比如某张表里姓名出现两次，因为它可能只是一个**字符串**；这个字符串里可能拼着“什么 + 当时 + 学号.....授予谁谁谁什么什么”之类内容，没有用**关联**方式，而是作为字符串写进数据库。
- 这样一旦姓名等发生变化，数据库内部就发生**不一致**。
- 讲者说，例如 **GP[?]** 给了一个例子。

第 24 部分



CRUD 架构的灾难

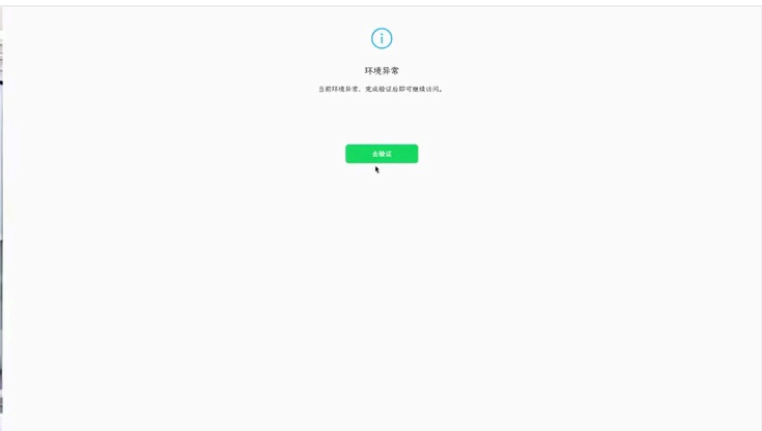
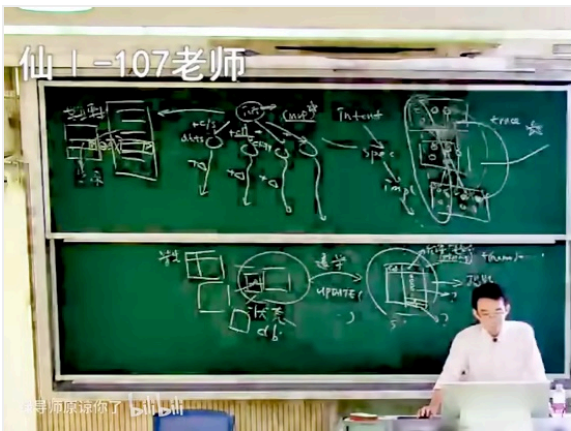
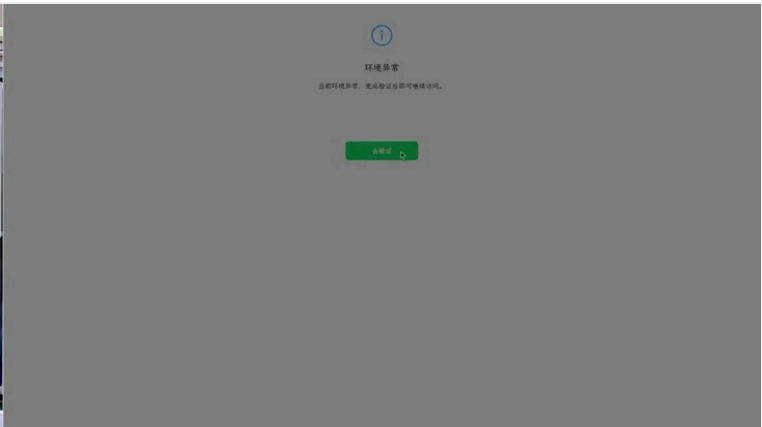
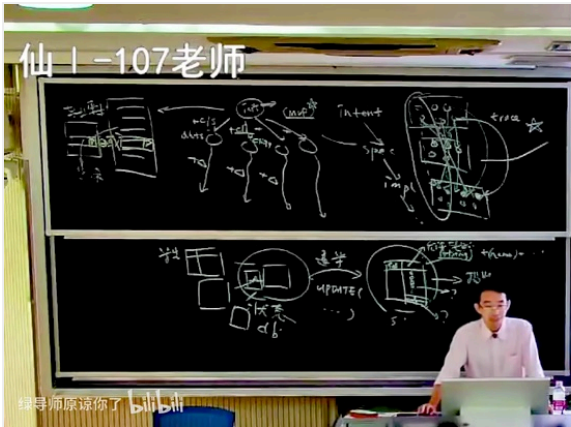
“当前状态”里，混着事实、规则和决定

- “当前状态”= 历史事实 + 当时适用的规则 + 根据规则作出的决定
 - 有大量单点数据是带有 data dependency 的“结果”
 - Data dependency 意味着前面改了会摧毁后面
- 更糟糕的是，这些依赖关系可能没有很好地管理起来(草台班子嘛)

一个“允许毕业”，究竟依赖什么？

- 课程成绩、课程替代审批、培养方案版本，共同决定学分认定和毕业结论。
 - 成绩更正了：哪些结论要重新审核？哪些已经成为正式决定？
- 表中没有重复字段，也可能丢掉“这个结论是怎么算出来的”。

5/18



只存“是否允许毕业”的结果，系统会丢掉历史 dependency 和计算过程

讲者用 GP 给的例子说明：

- 学生表里有 `student_id`，还有一个 column 叫 **是否允许毕业**。
- 当前状态：学生学分不够（还在上我的课）→ 不允许毕业；学分修够了 → 允许毕业。
- 这个“允许毕业”不是直接录入，而是由很多其他状态算出来的。之前有**触发器**：每当学生修了一个学分、毕业设计状态改变等，就重新计算该学生的毕业逻辑。
- 教务员/辅导员有典型需求：从教务系统里导出“到现在还毕不了业”的学生，逐个谈话。讲者认为这是很重要的需求点，必须在教务系统里做到。
- 但历史上的计算 **dependency** 随着当前状态都没有了。结果是：知道学生可以毕业，却不知道他为什么可以毕业；知道学生不能毕业，却不知道他为什么不能毕业；**计算过程丢掉了**。

逻辑链：

1. 毕业资格由多个状态和触发器反复计算。
2. 系统最终只留下当前“是否允许毕业”。
3. 历史 dependency 与计算过程没有保留。
4. → 系统知道结果，但解释不了结果。
5. → 需要导出“毕不了业”名单并追问原因/谈话时，当前结果字段不够用。（整理者串联）

软件是现实世界物理过程的有损投影，未建模处只能靠系统外手段补救

- 讲者认为：计算过程丢掉，而软件又是现实世界物理过程的一个**有损投影**；物理世界里建模不了的东西，最后全部需要通过其他途径解决。
- 例子：政策更新了、领导批准了新的培养计划方案，但新数据还没导入教务系统 → 不一致。
- 这时物理事件已经发生：培养计划已通过、学期已开始、已经上课；但系统里没有建模。
- 教室例子：第一周很典型，仙一这一片（ASR 原文）还在修，教室不能用；教务系统显然没有建模这一点，最后靠人工手改、重新安排教室，把课换到另一个教室；讲者第一周就去另一个教室上课。
- 讲者说这类问题经常遇到。

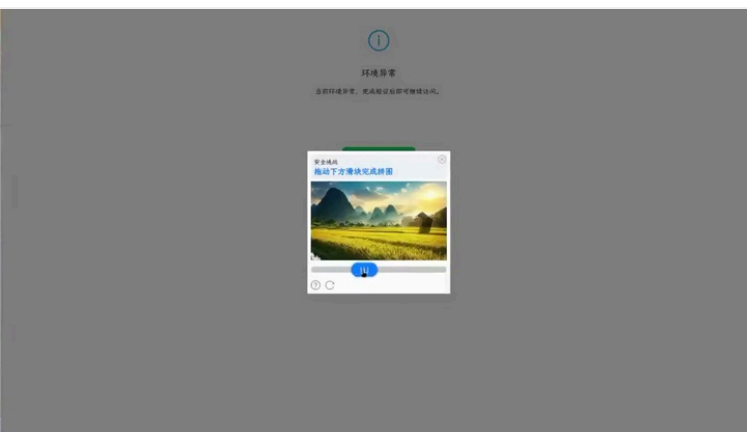
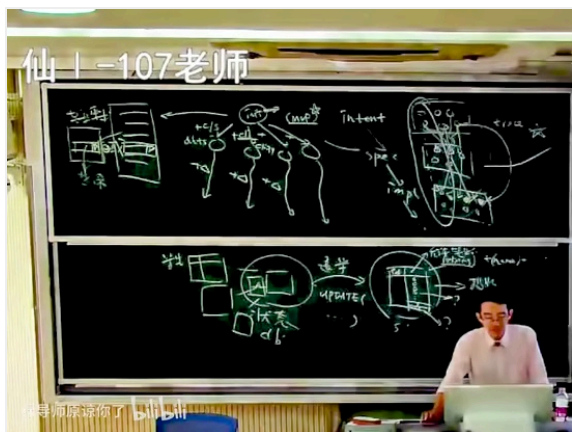
不一致会随时间积累成无法解释的黑洞

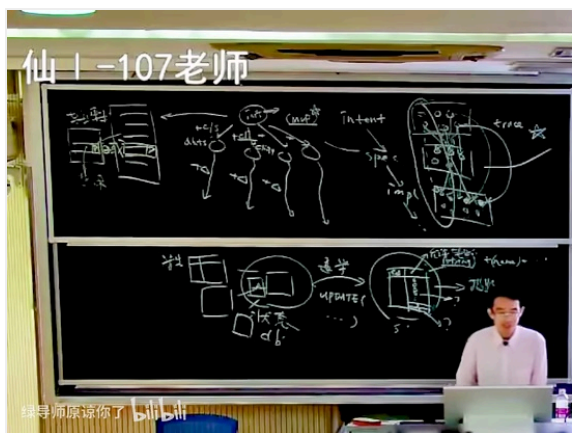
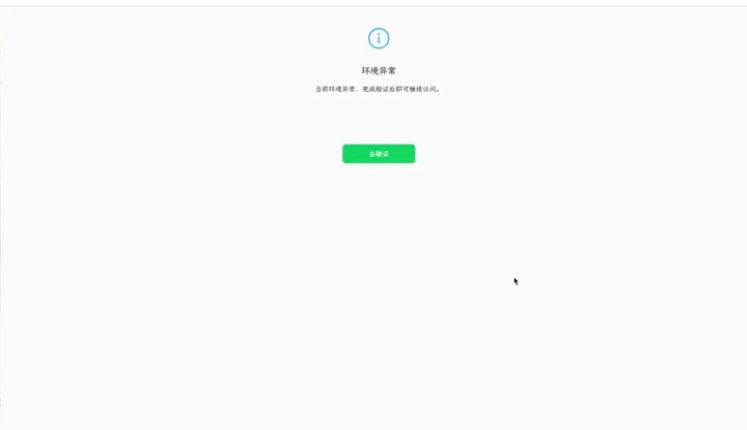
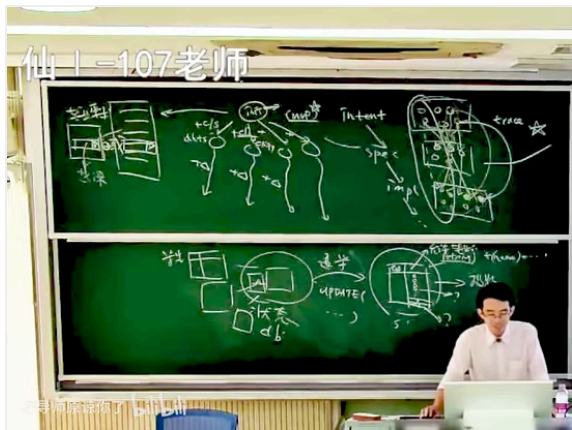
- 随着时间积累，系统里会有大量不一致的数据，变成无法解释的黑洞。
- 例子：明明某个同学不该（能）毕业，他为什么毕业了？回想起来，明明这个同学没有交学费，应该毕不了业，为什么他能毕业？
- 讲者说，系统里会出现无数这种“想都不能想”的不一致。
- 逻辑链：
 - 历史 dependency 丢失 + 物理世界未建模 → 当前系统状态与物理事实脱节。
 - 脱节久了 → 系统结果无法解释。
 - 无法解释的结果堆积 → 黑洞。（整理者串联）

面对这类需求，CRUD 会死

- 讲者直接断言：面对这种需求，**CRUD 就会死**。
- 讲者还提到：“南大没戏了”（ASR 原文）、“当前环境异常”、对 Linux 很害怕等（ASR 不清，保留原文）。
- 讲者说：“我们是最后一批系，现在成为计算机学院”，并进入“南大院系发生调整的时候”的情形——此处未展开。

第 25 部分





需求的语义，还会变

南大没系啦

- 例子：南大没系啦
 - 计算机系变成了计算机学院
 - 喜大普奔？教务系统瑟瑟发抖
- dept_id 应该用旧的，还是应该开一个新的？
 - “组织的名字”“组织的身份”“当时的隶属关系”，原来是三件事。

院系改名暴露 CRUD 的“历史 vs 当前”冲突

- 讲者以南京大学院系调整为例：计算机系后来变成计算机学院；他说“南大没系了”，自己是最后一批“系”。南大院系调整时，教务系统就“瑟瑟发抖”了。
- 教务系统是 CRUD 系统，也是 Point of truth：所有事实都在这里。
- 数据库里的 department 虽然是一张表，但讲者把它当对象看。于是问题变成：要不要把“计算机”这个对象的名字从“计算机系”改成“计算机学院”？讲者说：改也不对，不改也不对。

改：当前状态对了，但毕业记录变成假的

- 如果改，计算机系现在确实已变成计算机学院。
 - 但学生毕业记录被 select 查询时，会变成“2011 年毕业于南京大学计算机学院”。
 - 讲者指出这与事实不符：2011 年时还是计算机系，当时系统应生成“计算机系”的证明。
- （整理者串联）矛盾是：update 后数据库只保留新状态，无法同时表示“2011 年是系”和“现在是学院”。

不改：当前关联错了，仍然要迁移

- 如果不改，学生的 `department_id` 都指向“计算机系”这个旧对象。
- 但现实已经变了，还是得把它改过来。
- 这需要数据库迁移：创建新对象，并想清楚新旧对象之间的关联关系。
- 所以不是改一个名字那么简单。

需求发生摧毁性变化时，CRUD 的完美假设失效

讲者的逻辑链：

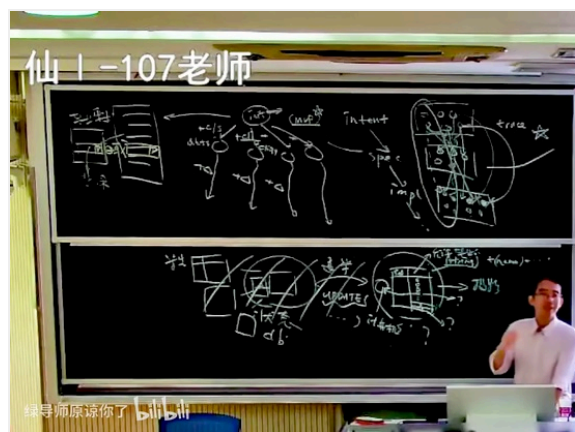
1. 若假设世界投影不变，CRUD 是完美架构。
2. 一旦需求发生摧毁性变化，每次都会涉及数据库迁移和业务逻辑重写。
3. 代码可以重写；历史不会因为代码重写而自动保留。
4. 数据库 `transaction` 的 `update` 完成后，状态从旧状态彻底挪到新状态。
5. 做一次迁移，就要考虑历史上所有可能发生的情况。
6. 数据库里已经有很多不一致，很难写出统一规则来迁移。
7. 迁移还可能把数据库弄坏。
8. 测试环境做一次迁移，和在真实学校里对全部数据做一次迁移，难度完全不一样。讲者还用“把选课给改成一”（[原文如此]）之类例子作对比。

ERP/教务系统的设计维护是手艺活

- 讲者结论：到今天之前，软件设计实现 ERP 这类系统（包括教务系统）的设计、实现、维护都是手艺活。
- 讲者调侃教务系统做得不好，并自纠说它确实有做得不好的一面。
- 但反过来看，作为软件工程实践者，那家公司在 2000 年成立时，可能唯一选择就是参考已有企业架构做教务系统，抢占教育市场第一桶金。

小结：讲者的论点是，CRUD 只在世界投影不变时完美；一旦历史事实与现实需求冲突，迁移就变成手艺活。

第 26 部分



系统一旦跑起来，修改的代价极高

代码可以重写，历史不能凭空补出来

- 数据库迁移、业务逻辑重写都是风险极高的操作
 - 不一致的数据无法解释
 - 同一个“已毕业”，可能来自正常审核、人工特批，也可能只是某次救火。
- 软件的设计、实现、维护都是手艺活
 - 这也是 ERP 类系统的护城河
- 迁移需要决定哪些历史含义继续成立：没有保存的依据，AI 也推不回来。

9/18

2000 年成立时，用已有企业架构做教务系统抢市场，随后背上 CRUD 技术债

- 讲者说，这家公司在 2000 年成立时，唯一选择是参考一个已有的企业架构来做教务系统，以抢占教育市场的“第一桶金”。
- 系统做出来后，本质是一个 CRUD 系统。
- 逻辑链：参考已有企业架构做教务系统 → 抢到教育市场第一桶金 → 系统是 CRUD → 一旦背上技术债 → 就要在中国大学教育改革的几十年里不断扛下所有变更。
- 讲者因此说，这是一门“手艺活”。

研究生院教务系统案例：典型 CRUD 在扩招、培养类型变化和频繁调整下变成屎山

- 讲者在软件所读 PhD；隔壁软件工程组接了学校研究生院教务系统的订单，金额约二三十万。
- 讲者判断：这在一般人看来可能算大订单，但在定制化软件里算小订单，因为要不停维护这个系统。后来这个系统维护不下去了。
- 讲者的室友参与了该项目。讲者常和他抬杠，聊研究生院系统该怎么设计；讲者经常“喷”他，因为那是一个非常典型的 CRUD 架构。
- 讲者当时说：你最后会面临各种各样的问题。后来果然发生：
 - 研究生巨大扩招；
 - 管理内容变多；

- 研究生类型、培养类型增多；
- 研究生相关压力大，经常有调整；
- 这些都是他们做需求设计时始料未及的。
- 结果：系统彻底变成屎山，最后不存在了；现在统一变成公司维护的系统。
- 讲者补充：公司维护方仍有一些领域知识，知道这些系统应该怎么把屎山“扶起来”。

重做教务系统架构：single point of truth 太危险，于是想到 persistent data structure

- 如果讲者要重做一个教务系统的架构，他会先看到 single point of truth 太危险。
- 讲者说：一旦有任何不一致，你都不知道应该怎样把它“拿出来”。
- 逻辑链：教务系统长期变化 → **（整理者串联）**需要处理状态/一致性问题 → single point of truth 危险 → 想到 persistent data structure。
- 讲者说，这是数据结构课上讲过的一种“非常神奇的东西”，它可以得到这个数据结构的所有历史。

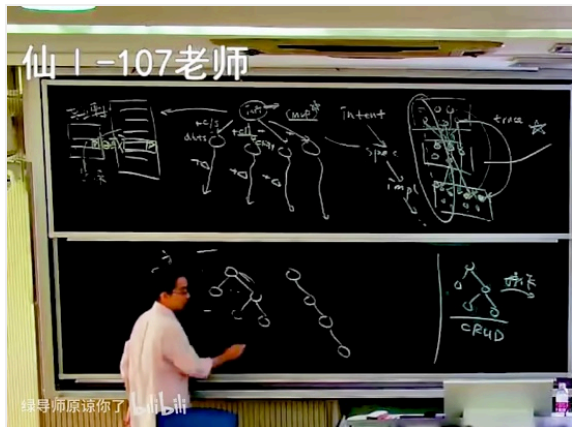
CRUD 是“当前存一个库，操作后到下一状态”；persistent data structure 用树演示保存历史

- 讲者先拿 CRUD 对比：
 - 存一个数据结构，比如存一棵树，这就是 CRUD；
 - 数据库也是一个数据结构；
 - 当前存一个库，对它做一个操作，它就进入下一个状态；
 - 这就是经典的 CRUD。
- 另一种方式，讲者用树演示：
 - 一开始只有一个节点；
 - 创建后，下一次树不变，旧节点还在，可以把这个节点放过来；
 - 如果某天树要变，比如要在某个节点上新建一个，就从该节点到根的路径上“长出来”新节点；
 - 未变的子树可以继续指向旧节点；
 - 这样树无论多大，都可以存下来。
- 对比：CRUD 的操作把结构推进到下一状态；persistent data structure 则让所有历史都能被保存/得到。**（整理者串联）**这使前面对“不一致时不知怎样拿出来”的担忧有

了一个方向：过去版本仍可被定位。

小结：讲者认为，教务系统会被长期教育改革逼着持续变更，CRUD 的单一当前状态会积成屎山；他因此转向能获得所有历史的 persistent data structure。

第 27 部分



把依赖变成 First-Class Citizens

“如果我们跳出问题看问题：既然都知道 dependency 是问题，为什么不能直接把它们变成 first-class citizens?”

数据结构的两面：当前状态与操作历史

- 数据结构可以记录“当前状态”，也可以记录“操作历史”
 - 张三选修了《生成式软件工程》
- 状态由 replay 事件序列决定 (语义 = 状态机)
 - 这样，我们就有底气面对语义改变了
 - 当然，还有很多问题要小心处理，比如语义的版本
- 祝贺，你发明了 [event sourcing](#)
- <@ 账本的直觉：余额是结果，逐笔记录让我们知道结果从何而来。

10 / 18

三种视图等价：状态、操作、Delta 都能回放历史得到当前状态

- 讲者说：状态、操作、状态的 Delta 这三种视图是等价的。
- 存日志：就是 local head 的 WRHEAD journal（转写原文如此；指数据库/OS 课上学过的 journal 思路）。
- 存 Delta：类似 Git/SVN 这类版本管理中的 delta 思想。
- 存状态：就是当前的 CRUD 结果。
- 共同点：它们都可以帮你回放历史，从而得到当前状态。

更好的架构：先把干过的事都记下来，再投影/回放出现状

逻辑链：

1. 如果只存当前状态，直接删除或覆盖，历史事实会丢。
2. 如果把所有操作作为事实记下来，事实就是不可篡改的。
3. 世界里的东西都是对事实的投影。
4. 因此可以通过 **replay** 从历史事件得到你想要的状态。
5. （整理者串联）所以查询过去、恢复错误、迁移数据，都可以建立在“回放历史”之上。

反例：没有“复学”操作时，删除张三再重建关联代价很大

- 讲者举例：今天张三退学了，今天张三又复学了。
- 假设这个世界之前没有“复学”这个操作，就只能把张三从数据库里抹掉。
- 抹掉后，一些记录也删掉了；还要把张三和其他部分的关联重新建起来，代价很大。
- 如果记录所有操作，就可以通过 replay 知道张三复学前的状态，再把状态拿过来，信息没有丢。
- 但讲者强调：**逻辑不能减少**。即使用日志、历史，如果“复学”以前没考虑过，仍然要为其写一个逻辑。

事实不可篡改带来底气：错了可退回快照，修好逻辑再来

- 讲者认为：因为事实不可篡改，世界里所有东西都是对事实的投影，所以人会更有底气，不怕错。
- 对比做数据库迁移时“手在抖”：如果做错，也可以很快退回过去的一个快照，再把逻辑修复，然后重来。
- 这不是说不用写业务逻辑，而是说错误可回退、可修复、可重放。

院系调整例子：查询过去得到过去版本，裁出相关历史再 replay

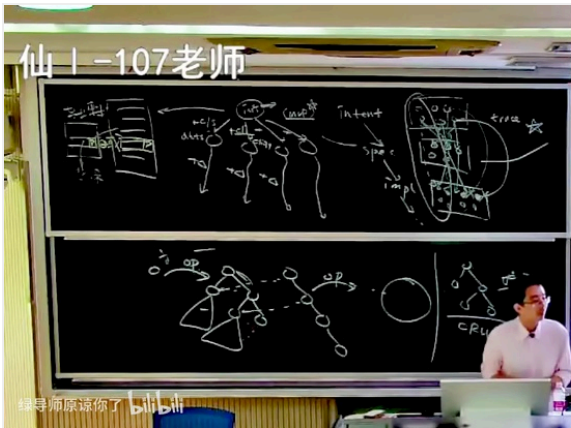
- 场景：某一个时刻发生了事件，叫“院系调整”。
- 查询过去的人时，会查询到他过去的版本，并且知道他经历过这个事件。
- 如果要查他过去，比如毕业时发生的事情，直接查他毕业时的版本即可。
- 如果要得到比较新的信息，可以投影出和这个人相关的历史。
- 具体例子：某同学 2007 年入学、2011 年毕业，当时都是计算机系；计算机系在 2014 年改为计算机学院。
- 讲者说：你总是可以裁出一个和事实相关的历史，然后把这个相关历史 replay，得到现在要的状态。因此可以查询过去，也可以查询现在。

这个设计模式叫 event sourcing：从历史事件生成状态

- 讲者总结：恭喜，你就发明了一个叫 **event sourcing** 的非常经典的设计模式。
- 它的想法很简单：**从历史事件生成状态**。
- （整理者串联）前面“先记事实、再投影/回放”的直觉，正是 event sourcing 的核心。

小结：讲者把“状态/操作/Delta 等价”推进到“事实不可篡改 + 回放历史”的架构原则，并命名为 event sourcing。

第 28 部分

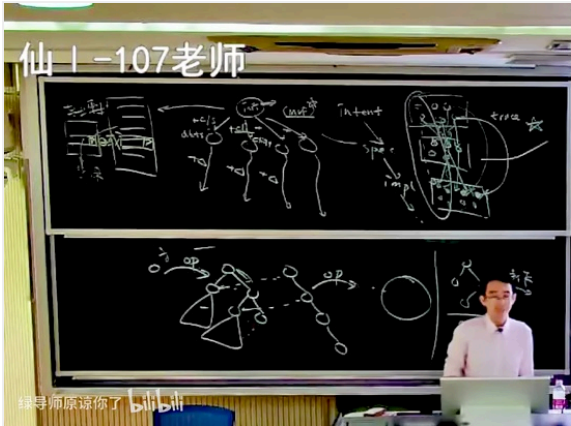


把依赖变成 First-Class Citizens

“如果我们跳出问题看问题：既然都知道 dependency 是问题，为什么不能直接把它们变成 first-class citizens?”

数据结构的两面：当前状态与操作历史

- 数据结构可以记录“当前状态”，也可以记录“操作历史”
 - 张三选修了《生成式软件工程》
- 状态由 replay 事件序列决定 (语义 = 状态机)
 - 这样，我们就有底气面对语义改变了
 - 当然，还有很多问题要小心处理，比如语义的版本
- 祝贺，你发明了 event sourcing
- 账户本的直觉：余额是结果，逐笔记录让我们知道结果从何而来。



代码，扩展性停留在演示版。

不是标准教务系统

不是。它是一个教师视角的最小可演示原型，而不是教务系统：

- 规模上：3 个合成账号、4 门课、18 名学生 (db.mjs 里 seed())。
- 覆盖面上：只有 20 个教师服务入口；缺学生视角、选课、排课、学籍、毕业审核、收费、培养方案管理等教务系统的核心。README 和 implementation.md 明确列出 88 条需求中大量“未纳入首版”。
- 能力上：没有 SSO、电子签章、消息推送、附件、政策规则库、多级审批、教室排课引擎；审批只“记录办件结果”，不会真的改课表或占用教室。

真实教务系统（正方/强智那种）本质是多角色、多学期、强一致、带可配置工作流引擎的大型领域系统。这个 demo 的价值不在“像教务系统”，而在于用极小代价把版本冲突、幂等、事务一致性、权限隔离、审计这些问题讲清楚——它更像是“一个横切面切片”，而不是一个系统。

Recap: 这是一个 React + Express + SQLite 的三层单体演示原型。用“version 乐观锁 + Idempotency-Key 幂等 + 事务 + audit 审计”这条统一管钱撑起核心业务，扩展服务面 resourceSpecs 数据驱动；它把“同类项可配置、新流程仍需改代码”的扩展性边界，以及“教师视角横切面 ≠ 标准教务系统”的差距，都展现得很清楚。

从历史事件生成状态，而不是直接维护 CRUD 状态 —— 这就是 event sourcing

- 讲者的说法：如果你接受了这个想法，你就等于（独立）发明了 event sourcing 这个“非常经典”的设计模式。
- 核心对比（讲者的表述）：
 - 通常做法：直接维护 CRUD 的状态；
 - event sourcing：从历史事件生成状态。
- 讲者强调这个想法本身“很简单”，并建议：回去可以在 agent 的带领下了解一下。

马上会想到的性能问题，讲者认为不是大问题

逻辑链（讲者自己给出的顺序）：

1. event sourcing 要从事件中重放 (replay) 出状态；
2. 你马上会反应过来：是不是会有性能问题？——因为要 replay 大量的事件；
3. 讲者回应：这"都不是很大的问题"；
4. 因为我们总可以用 快照 (snapshot)、缓存 等各种各样的优化技巧。

(整理者串联：讲者这里是在"先承认怀疑成立，再说明怀疑不足以否定方案"——性能是可达成的工程问题，而不是模式本身的缺陷。)

真门槛是迁移成本：已经在"屎山"上的系统很难改成 event sourcing

- 讲者举例：如果你已经在屎山上了，再把它变成 event sourcing 的设计，就会变得很困难。
- 反过来：如果你从一开始就做 event sourcing 的设计，很多事情、很多问题就都不会发生，你能比较好地解决。
- (整理者串联：因此这是一个"早做易、晚做难"的设计取舍，而不是"值不值得为性能做"的问题。)

它旁边站着一堆 fancy 的软件工程概念，但那些概念和它是独立的

- 讲者点名的概念：
 - CQRS —— Command Query Responsibility Segregation；
 - domain driven design (DDD)。
- 讲者明确强调：CQRS 和 event sourcing 是独立的，"event sourcing 是独立的"（不要把它们当成必须捆绑的一套）。
- 讲者对这些名词的态度：你们都不需要——可以去看、可以去了解，但不必以它们为前提。

真正的能力是回到 first principle，看清问题的根源

- 讲者的具体场景：设计一个 50 万行的教务系统。
- 他的主张：我们跳出这个问题，去看教务系统根源的问题在哪里，你就能做一个正确的架构设计。

- 讲者明说的结论：哪怕你没有听说过这些（模式、名词），你也能做一个正确的架构设计。

逻辑链：

回到 first principle / 看清问题根源 → 正确的架构设计（这条链上并不需要先掌握 CQRS、DDD 等名词）

讲者的落点：模式名与衍生概念是后话，先回到 first principle 看清问题根源，才是能做出正确架构设计的原因。
